

Hugging Face Agents Course

Core Instructional Units 1-4

A text-first, rights-cleared adaptation prepared for Reflo's seeded public demonstration.

Source: <https://github.com/huggingface/agents-course>

Source revision: 8c0832eae634ebb34541c65265caa6da4c5d2c57

Retrieved: 2026-07-25

Original course authors and maintainers: Ben Burtenshaw, Joffrey Thomas, Thomas Simonini, Sergio Paniego, and Hugging Face contributors.

License: Apache License 2.0

Adaptation notice. Reflo combined the English core instructional units into one PDF, removed website navigation and account/community onboarding, normalized MDX for print, and omitted remote images, videos, iframes, and other third-party embeds. The instructional prose, examples, exercises, and core quizzes are retained in source order. This adaptation is not endorsed by Hugging Face.

Apache License 2.0 permits reproduction and derivative works subject to its conditions. The complete license is reproduced in the appendix.

Contents and Included Scope

Unit 1. Introduction to Agents

- Introduction ([units/en/unit1/introduction.mdx](#))
- What is an Agent? ([units/en/unit1/what-are-agents.mdx](#))
- Quick Quiz 1 ([units/en/unit1/quiz1.mdx](#))
- What are LLMs? ([units/en/unit1/what-are-llms.mdx](#))
- Messages and Special Tokens ([units/en/unit1/messages-and-special-tokens.mdx](#))
- What are Tools? ([units/en/unit1/tools.mdx](#))
- Quick Quiz 2 ([units/en/unit1/quiz2.mdx](#))
- Understanding AI Agents through the Thought-Action-Observation Cycle ([units/en/unit1/agent-steps-and-structure.mdx](#))
- Thought, Internal Reasoning and the Re-Act Approach ([units/en/unit1/thoughts.mdx](#))
- Actions, Enabling the Agent to Engage with Its Environment ([units/en/unit1/actions.mdx](#))
- Observe, Integrating Feedback to Reflect and Adapt ([units/en/unit1/observations.mdx](#))
- Dummy Agent Library ([units/en/unit1/dummy-agent-library.mdx](#))
- Let's Create Our First Agent Using smolagents ([units/en/unit1/tutorial.mdx](#))
- Unit 1 Final Quiz ([units/en/unit1/final-quiz.mdx](#))
- Conclusion ([units/en/unit1/conclusion.mdx](#))

Unit 2. Frameworks for AI Agents

- Frameworks for AI Agents ([units/en/unit2/introduction.mdx](#))

Unit 2.1 The smolagents framework

- Introduction to smolagents ([units/en/unit2/smolagents/introduction.mdx](#))
- Why use smolagents? ([units/en/unit2/smolagents/why_use_smolagents.mdx](#))
- Quick Quiz 1 ([units/en/unit2/smolagents/quiz1.mdx](#))
- Building Agents That Use Code ([units/en/unit2/smolagents/code_agents.mdx](#))
- Writing actions as code snippets or JSON blobs ([units/en/unit2/smolagents/tool_calling_agents.mdx](#))
- Tools ([units/en/unit2/smolagents/tools.mdx](#))
- Retrieval Agents ([units/en/unit2/smolagents/retrieval_agents.mdx](#))
- Quick Quiz 2 ([units/en/unit2/smolagents/quiz2.mdx](#))
- Multi-Agent Systems ([units/en/unit2/smolagents/multi_agent_systems.mdx](#))
- Vision and Browser agents ([units/en/unit2/smolagents/vision_agents.mdx](#))
- Final Quiz ([units/en/unit2/smolagents/final_quiz.mdx](#))
- Conclusion ([units/en/unit2/smolagents/conclusion.mdx](#))

Unit 2.2 The LlamaIndex framework

- Introduction to LlamaIndex ([units/en/unit2/llama-index/introduction.mdx](#))
- Introduction to LlamaHub ([units/en/unit2/llama-index/llama-hub.mdx](#))
- What are Components in LlamaIndex? ([units/en/unit2/llama-index/components.mdx](#))
- Using Tools in LlamaIndex ([units/en/unit2/llama-index/tools.mdx](#))
- Quick Quiz 1 ([units/en/unit2/llama-index/quiz1.mdx](#))

- Using Agents in LlamaIndex ([units/en/unit2/llama-index/agents.mdx](#))
- Creating Agentic Workflows in LlamaIndex ([units/en/unit2/llama-index/workflows.mdx](#))
- Quick Quiz 2 ([units/en/unit2/llama-index/quiz2.mdx](#))
- Conclusion ([units/en/unit2/llama-index/conclusion.mdx](#))

Unit 2.3 The LangGraph framework

- Introduction to LangGraph ([units/en/unit2/langgraph/introduction.mdx](#))
- What is LangGraph? ([units/en/unit2/langgraph/when_to_use_langgraph.mdx](#))
- Building Blocks of LangGraph ([units/en/unit2/langgraph/building_blocks.mdx](#))
- Building Your First LangGraph ([units/en/unit2/langgraph/first_graph.mdx](#))
- Document Analysis Graph ([units/en/unit2/langgraph/document_analysis_agent.mdx](#))
- Quick Quiz 1 ([units/en/unit2/langgraph/quiz1.mdx](#))
- Conclusion ([units/en/unit2/langgraph/conclusion.mdx](#))

Unit 3. Use Case for Agentic RAG

- Introduction to Use Case for Agentic RAG ([units/en/unit3/agentic-rag/introduction.mdx](#))
- Agentic Retrieval Augmented Generation (RAG) ([units/en/unit3/agentic-rag/agentic-rag.mdx](#))
- Creating a RAG Tool for Guest Stories ([units/en/unit3/agentic-rag/invitees.mdx](#))
- Building and Integrating Tools for Your Agent ([units/en/unit3/agentic-rag/tools.mdx](#))
- Creating Your Gala Agent ([units/en/unit3/agentic-rag/agent.mdx](#))
- Conclusion ([units/en/unit3/agentic-rag/conclusion.mdx](#))

Unit 4. Final Project - Create, Test, and Certify Your Agent

- Introduction to the Final Unit ([units/en/unit4/introduction.mdx](#))
- What is GAIA? ([units/en/unit4/what-is-gaia.mdx](#))
- The Final Hands-On ([units/en/unit4/hands-on.mdx](#))
- Get Your Certificate Of Excellence ([units/en/unit4/get-your-certificate.mdx](#))
- Conclusion of the Course ([units/en/unit4/conclusion.mdx](#))
- What Should You Learn Now? ([units/en/unit4/additional-readings.mdx](#))

Excluded: Unit 0 onboarding, live/community material, all bonus units, translations, repository tooling, and remote or embedded media.

Unit 1. Introduction to Agents

Instructional sections below follow the official English course order.

Introduction

Source file: [units/en/unit1/introduction.mdx](#)

Introduction to Agents

[Image omitted from text-first adaptation: Thumbnail]

Welcome to this first unit, where you'll build a solid foundation in the fundamentals of AI Agents including:

- Understanding Agents
- What is an Agent, and how does it work?
- How do Agents make decisions using reasoning and planning?
- The Role of LLMs (Large Language Models) in Agents
- How LLMs serve as the "brain" behind an Agent.
- How LLMs structure conversations via the Messages system.
- Tools and Actions
- How Agents use external tools to interact with the environment.
- How to build and integrate tools for your Agent.
- The Agent Workflow:
- Think -> Act -> Observe.

After exploring these topics, you'll build your first Agent using smolagents!

Your Agent, named Alfred, will handle a simple task and demonstrate how to apply these concepts in practice.

You'll even learn how to publish your Agent on Hugging Face Spaces, so you can share it with friends and colleagues.

Finally, at the end of this Unit, you'll take a quiz. Pass it, and you'll earn your first course certification: the Certificate of Fundamentals of Agents.

[Image omitted from text-first adaptation: Certificate Example]

This Unit is your essential starting point, laying the groundwork for understanding Agents before you move on to more advanced topics.

[Image omitted from text-first adaptation: Unit 1 planning]

It's a big unit, so take your time and don't hesitate to come back to these sections from time to time.

Ready? Let's dive in!

What is an Agent?

Source file: [units/en/unit1/what-are-agents.mdx](#)

What is an Agent?

[Image omitted from text-first adaptation: Unit 1 planning]

By the end of this section, you'll feel comfortable with the concept of agents and their various applications in AI.

To explain what an Agent is, let's start with an analogy.

The Big Picture: Alfred The Agent

Meet Alfred. Alfred is an Agent.

[Image omitted from text-first adaptation: This is Alfred]

Imagine Alfred receives a command, such as: "Alfred, I would like a coffee please."

[Image omitted from text-first adaptation: I would like a coffee]

Because Alfred understands natural language, he quickly grasps our request.

Before fulfilling the order, Alfred engages in reasoning and planning, figuring out the steps and tools he needs to:

1. Go to the kitchen
1. Use the coffee machine
1. Brew the coffee
1. Bring the coffee back

[Image omitted from text-first adaptation: Reason and plan]

Once he has a plan, he must act. To execute his plan, he can use tools from the list of tools he knows about.

In this case, to make a coffee, he uses a coffee machine. He activates the coffee machine to brew the coffee.

[Image omitted from text-first adaptation: Make coffee]

Finally, Alfred brings the freshly brewed coffee to us.

[Image omitted from text-first adaptation: Bring coffee]

And this is what an Agent is: an AI model capable of reasoning, planning, and interacting with its environment.

We call it Agent because it has agency, aka it has the ability to interact with the environment.

[Image omitted from text-first adaptation: Agent process]

Let's go more formal

Now that you have the big picture, here's a more precise definition:

An Agent is a system that leverages an AI model to interact with its environment in order to achieve a user-defined objective. It combines reasoning, planning, and the execution of actions (often via external tools) to fulfill tasks.

Think of the Agent as having two main parts:

1. The Brain (AI Model)

This is where all the thinking happens. The AI model handles reasoning and planning. It decides which Actions to take based on the situation.

1. The Body (Capabilities and Tools)

This part represents everything the Agent is equipped to do.

The scope of possible actions depends on what the agent has been equipped with. For example, because humans lack wings, they can't perform the "fly" Action, but they can execute Actions like "walk", "run", "jump", "grab", and so on.

The spectrum of "Agency"

Following this definition, Agents exist on a continuous spectrum of increasing agency:

Agency Level	Description	What that's called	Example pattern
emptyemptyempty	Agent output has no impact on program flow	Simple processor	process_llm_output(llm_response)
filledemptyempty	Agent output determines basic control flow	Router	if llm_decision(): path_a() else: path_b()

Agency Level	Description	What that's called	Example pattern
filledfilledempty	Agent output determines function execution	Tool caller	<code>run_function(llm_chosen_tool, llm_chosen_args)</code>
filledfilledfilled	Agent output controls iteration and program continuation	Multi-step Agent	<code>while llm_should_continue(): execute_next_step()</code>
filledfilledfilled	One agentic workflow can start another agentic workflow	Multi-Agent	<code>if llm_trigger(): execute_agent()</code>

Table from smolagents conceptual guide (https://huggingface.co/docs/smolagents/conceptual_guides/intro_agents).

What type of AI Models do we use for Agents?

The most common AI model found in Agents is an LLM (Large Language Model), which takes Text as an input and outputs Text as well.

Well known examples are GPT-4 from OpenAI, Llama from Meta, Gemini from Google, etc. These models have been trained on a vast amount of text and are able to generalize well. We will learn more about LLMs in the next section (what-are-llms).

It's also possible to use models that accept other inputs as the Agent's core model. For example, a Vision Language Model (VLM), which is like an LLM but also understands images as input. We'll focus on LLMs for now and will discuss other options later.

How does an AI take action on its environment?

LLMs are amazing models, but they can only generate text.

However, if you ask a well-known chat application like HuggingChat or ChatGPT to generate an image, they can! How is that possible?

The answer is that the developers of HuggingChat, ChatGPT and similar apps implemented additional functionality (called Tools), that the LLM can use to create images.

[Image omitted from text-first adaptation: Eiffel Brocolis]

The model used an Image Generation Tool to generate this image.

We will learn more about tools in the Tools (tools) section.

What type of tasks can an Agent do?

An Agent can perform any task we implement via Tools to complete Actions.

For example, if I write an Agent to act as my personal assistant (like Siri) on my computer, and I ask it to "send an email to my Manager asking to delay today's meeting", I can give it some code to send emails. This will be a new Tool the Agent can use whenever it needs to send an email. We can write it in Python:

```
def send_message_to(recipient, message):
    """Useful to send an e-mail message to a recipient"""
    ...
```

The LLM, as we'll see, will generate code to run the tool when it needs to, and thus fulfill the desired task.

```
send_message_to("Manager", "Can we postpone today's meeting?")
```

The design of the Tools is very important and has a great impact on the quality of your Agent. Some tasks will require very specific Tools to be crafted, while others may be solved with general-purpose tools like "web_search".

Note that Actions are not the same as Tools. An Action, for instance, can involve the use of multiple Tools to complete.

Allowing an agent to interact with its environment allows real-life usage for companies and individuals.

Example 1: Personal Virtual Assistants

Virtual assistants like Siri, Alexa, or Google Assistant, work as agents when they interact on behalf of users using their digital environments.

They take user queries, analyze context, retrieve information from databases, and provide responses or initiate actions (like setting reminders, sending messages, or controlling smart devices).

Example 2: Customer Service Chatbots

Many companies deploy chatbots as agents that interact with customers in natural language.

These agents can answer questions, guide users through troubleshooting steps, open issues in internal databases, or even complete transactions.

Their predefined objectives might include improving user satisfaction, reducing wait times, or increasing sales conversion rates. By interacting directly with customers, learning from the dialogues, and adapting their responses over time, they demonstrate the core principles of an agent in action.

Example 3: AI Non-Playable Character in a video game

AI agents powered by LLMs can make Non-Playable Characters (NPCs) more dynamic and unpredictable.

Instead of following rigid behavior trees, they can respond contextually, adapt to player interactions, and generate more nuanced dialogue. This flexibility helps create more lifelike, engaging characters that evolve alongside the player's actions.

To summarize, an Agent is a system that uses an AI Model (typically an LLM) as its core reasoning engine, to:

- Understand natural language: Interpret and respond to human instructions in a meaningful way.
- Reason and plan: Analyze information, make decisions, and devise strategies to solve problems.
- Interact with its environment: Gather information, take actions, and observe the results of those actions.

Now that you have a solid grasp of what Agents are, let's reinforce your understanding with a short, ungraded quiz. After that, we'll dive into the "Agent's brain": the LLMs (what-are-llms).

Quick Quiz 1

Source file: [units/en/unit1/quiz1.mdx](#)

Q1: What is an Agent?

Which of the following best describes an AI Agent?

- Answer choice: An AI model that can reason, plan, and use tools to interact with its environment to achieve a specific goal.

Explanation: This definition captures the essential characteristics of an Agent.

Correct answer.

},

- Answer choice: A system that solely processes static text, without any inherent mechanism to interact dynamically with its surroundings or execute meaningful actions.

Explanation: An Agent must be able to take an action and interact with its environment.

},

- Answer choice: A conversational agent restricted to answering queries, lacking the ability to perform any actions or interact with external systems.

Explanation: A chatbot like this lacks the ability to take actions, making it different from an Agent.

},

- Answer choice: An online repository of information that offers static content without the capability to execute tasks or interact actively with users.

Explanation: An Agent actively interacts with its environment rather than just providing static information.

Q2: What is the Role of Planning in an Agent?

Why does an Agent need to plan before taking an action?

- Answer choice: To primarily store or recall past interactions, rather than mapping out a sequence of future actions.

Explanation: Planning is about determining future actions, not storing past interactions.

},

- Answer choice: To decide on the sequence of actions and select appropriate tools needed to fulfill the user's request.

Explanation: Planning helps the Agent determine the best steps and tools to complete a task.

Correct answer.

},

- Answer choice: To execute a sequence of arbitrary and uncoordinated actions that lack any defined strategy or intentional objective.

Explanation: Planning ensures the Agent's actions are intentional and not random.

},

- Answer choice: To merely convert or translate text, bypassing any process of formulating a deliberate sequence of actions or employing strategic reasoning.

Explanation: Planning is about structuring actions, not just converting text.

Q3: How Do Tools Enhance an Agent's Capabilities?

Why are tools essential for an Agent?

- Answer choice: Tools serve no real purpose and do not contribute to the Agent's ability to perform actions beyond basic text generation.

Explanation: Tools expand an Agent's capabilities by allowing it to perform actions beyond text generation.

},

- Answer choice: Tools are solely designed for memory storage, lacking any capacity to facilitate the execution of tasks or enhance interactive performance.

Explanation: Tools are primarily for performing actions, not just for storing data.

},

- Answer choice: Tools severely restrict the Agent exclusively to generating text, thereby preventing it from engaging in a broader range of interactive actions.

Explanation: On the contrary, tools allow Agents to go beyond text-based responses.

},

- Answer choice: Tools provide the Agent with the ability to execute actions a text-generation model cannot perform natively, such as making coffee or generating images.

Explanation: Tools enable Agents to interact with the real world and complete tasks.

Correct answer.

Q4: How Do Actions Differ from Tools?

What is the key difference between Actions and Tools?

- Answer choice: Actions are the steps the Agent takes, while Tools are external resources the Agent can use to perform those actions.

Explanation: Actions are higher-level objectives, while Tools are specific functions the Agent can call upon.

Correct answer.

},

- Answer choice: Actions and Tools are entirely identical components that can be used interchangeably, with no clear differences between them.

Explanation: No, Actions are goals or tasks, while Tools are specific utilities the Agent uses to achieve them.

},

- Answer choice: Tools are considered broad utilities available for various functions, whereas Actions are mistakenly thought to be restricted only to physical interactions.

Explanation: Not necessarily. Actions can involve both digital and physical tasks.

},

- Answer choice: Actions inherently require the use of LLMs to be determined and executed, whereas Tools are designed to function autonomously without such dependencies.

Explanation: While LLMs help decide Actions, Actions themselves are not dependent on LLMs.

Q5: What Role Do Large Language Models (LLMs) Play in Agents?

How do LLMs contribute to an Agent's functionality?

- Answer choice: LLMs function merely as passive repositories that store information, lacking any capability to actively process input or produce dynamic responses.

Explanation: LLMs actively process text input and generate responses, rather than just storing information.

},

- Answer choice: LLMs serve as the reasoning 'brain' of the Agent, processing text inputs to understand instructions and plan actions.

Explanation: LLMs enable the Agent to interpret, plan, and decide on the next steps.

Correct answer.

},

- Answer choice: LLMs are erroneously believed to be used solely for image processing, when in fact their primary function is to process and generate text.

Explanation: LLMs primarily work with text, although they can sometimes interact with multimodal inputs.

},

- Answer choice: LLMs are considered completely irrelevant to the operation of AI Agents, implying that they are entirely superfluous in any practical application.

Explanation: LLMs are a core component of modern AI Agents.

Q6: Which of the Following Best Demonstrates an AI Agent?

Which real-world example best illustrates an AI Agent at work?

- Answer choice: A static FAQ page on a website that provides fixed information and lacks any interactive or dynamic response capabilities.

Explanation: A static FAQ page does not interact dynamically with users or take actions.

},

- Answer choice: A simple calculator that performs arithmetic operations based on fixed rules, without any capability for reasoning or planning.

Explanation: A calculator follows fixed rules without reasoning or planning, so it is not an Agent.

},

- Answer choice: A virtual assistant like Siri or Alexa that can understand spoken commands, reason through them, and perform tasks like setting reminders or sending messages.

Explanation: This example includes reasoning, planning, and interaction with the environment.

Correct answer.

},

- Answer choice: A video game NPC that operates on a fixed script of responses, without the ability to reason, plan, or use external tools.

Explanation: Unless the NPC can reason, plan, and use tools, it does not function as an AI Agent.

Congrats on finishing this Quiz ! If you need to review any elements, take the time to revisit the chapter to reinforce your knowledge before diving deeper into the "Agent's brain": LLMs.

What are LLMs?

Source file: <units/en/unit1/what-are-llms.mdx>

What are LLMs?

[Image omitted from text-first adaptation: Unit 1 planning]

In the previous section we learned that each Agent needs an AI Model at its core, and that LLMs are the most common type of AI models for this purpose.

Now we will learn what LLMs are and how they power Agents.

This section offers a concise technical explanation of the use of LLMs. If you want to dive deeper, you can check our free Natural Language Processing Course.

What is a Large Language Model?

An LLM is a type of AI model that excels at understanding and generating human language. They are trained on vast amounts of text data, allowing them to learn patterns, structure, and even nuance in language. These models typically consist of many millions of parameters.

Most LLMs nowadays are built on the Transformer architecture—a deep learning architecture based on the "Attention" algorithm, that has gained significant interest since the release of BERT from Google in 2018.

[Image omitted from text-first adaptation: Transformer]

The original Transformer architecture looked like this, with an encoder on the left and a decoder on the right.

There are 3 types of transformers:

1. Encoders

An encoder-based Transformer takes text (or other data) as input and outputs a dense representation (or embedding) of that text.

- Example: BERT from Google
- Use Cases: Text classification, semantic search, Named Entity Recognition
- Typical Size: Millions of parameters

1. Decoders

A decoder-based Transformer focuses on generating new tokens to complete a sequence, one token at a time.

- Example: Llama from Meta
- Use Cases: Text generation, chatbots, code generation
- Typical Size: Billions (in the US sense, i.e., 10^9) of parameters

1. Seq2Seq (Encoder-Decoder)

A sequence-to-sequence Transformer combines an encoder and a decoder. The encoder first processes the input sequence into a context representation, then the decoder generates an output sequence.

- Example: T5, BART
- Use Cases: Translation, Summarization, Paraphrasing
- Typical Size: Millions of parameters

Although Large Language Models come in various forms, LLMs are typically decoder-based models with billions of parameters. Here are some of the most well-known LLMs:

Model	Provider
Deepseek-R1	DeepSeek
GPT4	OpenAI
Llama 3	Meta (Facebook AI Research)
SmolLM2	Hugging Face
Gemma	Google
Mistral	Mistral

The underlying principle of an LLM is simple yet highly effective: its objective is to predict the next token, given a sequence of previous tokens. A "token" is the unit of information an LLM works with. You can think of a "token" as if it was a "word", but for efficiency reasons LLMs don't use whole words.

For example, while English has an estimated 600,000 words, an LLM might have a vocabulary of around 32,000 tokens (as is the case with Llama 2). Tokenization often works on sub-word units that can be combined.

For instance, consider how the tokens "interest" and "ing" can be combined to form "interesting", or "ed" can be appended to form "interested."

You can experiment with different tokenizers in the interactive playground below:

[Embedded media omitted from text-first adaptation: interactive media]

Each LLM has some special tokens specific to the model. The LLM uses these tokens to open and close the structured components of its generation. For example, to indicate the start or end of a sequence, message, or response. Moreover, the input prompts that we pass to the model are also structured with special tokens. The most important of those is the End of sequence token (EOS).

The forms of special tokens are highly diverse across model providers.

The table below illustrates the diversity of special tokens.

Model	Provider	EOS Token	Functionality
GPT4	OpenAI	< endoftext >	End of message text
Llama 3	Meta (Facebook AI Research)	< eot_id >	End of sequence
Deepseek-R1	DeepSeek	< end_of_sentence >	End of message text
SmolLM2	Hugging Face	< im_end >	End of instruction or message
Gemma	Google	< end_of_turn >	End of conversation turn

We do not expect you to memorize these special tokens, but it is important to appreciate their diversity and the role they play in the text generation of LLMs. If you want to know more about special tokens, you can check out the configuration of the model in its Hub repository. For example, you can find the special tokens of the SmolLM2 model in its tokenizer_config.json.

Understanding next token prediction.

LLMs are said to be autoregressive, meaning that the output from one pass becomes the input for the next one. This loop continues until the model predicts the next token to be the EOS token, at which point the model can stop.

[Image omitted from text-first adaptation: Visual Gif of autoregressive decoding]

In other words, an LLM will decode text until it reaches the EOS. But what happens during a single decoding loop?

While the full process can be quite technical for the purpose of learning agents, here's a brief overview:

- Once the input text is tokenized, the model computes a representation of the sequence that captures information about the meaning and the position of each token in the input sequence.
- This representation goes into the model, which outputs scores that rank the likelihood of each token in its vocabulary as being the next one in the sequence.

[Image omitted from text-first adaptation: Visual Gif of decoding]

Based on these scores, we have multiple strategies to select the tokens to complete the sentence.

- The easiest decoding strategy would be to always take the token with the maximum score.

You can interact with the decoding process yourself with SmolLM2 in this Space (remember, it decodes until reaching an EOS token which is for this model):

[Embedded media omitted from text-first adaptation: interactive media]

- But there are more advanced decoding strategies. For example, beam search explores multiple candidate sequences to find the one with the maximum total score-even if some individual tokens have lower scores.

[Embedded media omitted from text-first adaptation: interactive media]

If you want to know more about decoding, you can take a look at the NLP course (<https://huggingface.co/learn/nlp-course>).

Attention is all you need

A key aspect of the Transformer architecture is Attention. When predicting the next word, not every word in a sentence is equally important; words like "France" and "capital" in the sentence "The capital of France is ..." carry the most meaning.

[Image omitted from text-first adaptation: Visual Gif of Attention]

This process of identifying the most relevant words to predict the next token has proven to be incredibly effective.

Although the basic principle of LLMs-predicting the next token-has remained consistent since GPT-2, there have been significant advancements in scaling neural networks and making the attention mechanism work for longer and longer sequences.

If you've interacted with LLMs, you're probably familiar with the term context length, which refers to the maximum number of tokens the LLM can process, and the maximum attention span it has.

Prompting the LLM is important

Considering that the only job of an LLM is to predict the next token by looking at every input token, and to choose which tokens are "important", the wording of your input sequence is very important.

The input sequence you provide an LLM is called a prompt. Careful design of the prompt makes it easier to guide the generation of the LLM toward the desired output.

How are LLMs trained?

LLMs are trained on large datasets of text, where they learn to predict the next word in a sequence through a self-supervised or masked language modeling objective.

From this unsupervised learning, the model learns the structure of the language and underlying patterns in text, allowing the model to generalize to unseen data.

After this initial pre-training, LLMs can be fine-tuned on a supervised learning objective to perform specific tasks. For example, some models are trained for conversational structures or tool usage, while others focus on classification or code generation.

How can I use LLMs?

You have two main options:

1. Run Locally (if you have sufficient hardware).
1. Use a Cloud/API (e.g., via the Hugging Face Serverless Inference API).

Throughout this course, we will primarily use models via APIs on the Hugging Face Hub. Later on, we will explore how to run these models locally on your hardware.

How are LLMs used in AI Agents?

LLMs are a key component of AI Agents, providing the foundation for understanding and generating human language.

They can interpret user instructions, maintain context in conversations, define a plan and decide which tools to use.

We will explore these steps in more detail in this Unit, but for now, what you need to understand is that the LLM is the brain of the Agent.

That was a lot of information! We've covered the basics of what LLMs are, how they function, and their role in powering AI agents.

If you'd like to dive even deeper into the fascinating world of language models and natural language processing, don't hesitate to check out our free NLP course.

Now that we understand how LLMs work, it's time to see how LLMs structure their generations in a conversational context.

To run this notebook, you need a Hugging Face token that you can get from <https://hf.co/settings/tokens>.

For more information on how to run Jupyter Notebooks, checkout Jupyter Notebooks on the Hugging Face Hub.

You also need to request access to the Meta Llama models.

Messages and Special Tokens

Source file: [units/en/unit1/messages-and-special-tokens.mdx](#)

Messages and Special Tokens

Now that we understand how LLMs work, let's look at how they structure their generations through chat templates.

Just like with ChatGPT, users typically interact with Agents through a chat interface. Therefore, we aim to understand how LLMs manage chats.

Q: But ... When, I'm interacting with ChatGPT/Hugging Chat, I'm having a conversation using chat Messages, not a single prompt sequence

A: That's correct! But this is in fact a UI abstraction. Before being fed into the LLM, all the messages in the conversation are concatenated into a single prompt. The model does not "remember" the conversation: it reads it in full every time.

Up until now, we've discussed prompts as the sequence of tokens fed into the model. But when you chat with systems like ChatGPT or HuggingChat, you're actually exchanging messages. Behind the scenes, these messages are concatenated and formatted into a prompt that the model can understand.

[Image omitted from text-first adaptation: Behind models]

We see here the difference between what we see in UI and the prompt fed to the model.

This is where chat templates come in. They act as the bridge between conversational messages (user and assistant turns) and the specific formatting requirements of your chosen LLM. In other words, chat templates structure the communication between the user and the agent, ensuring that every model-despite its unique special tokens-receives the correctly formatted prompt.

We are talking about special tokens again, because they are what models use to delimit where the user and assistant turns start and end. Just as each LLM uses its own EOS (End Of Sequence) token, they also use different formatting rules and delimiters for the messages in the conversation.

Messages: The Underlying System of LLMs

System Messages

System messages (also called System Prompts) define how the model should behave. They serve as persistent instructions, guiding every subsequent interaction.

For example:

```
system_message = {
    "role": "system",
    "content": "You are a professional customer service agent. Always be polite, clear, and helpful."
}
```

With this System Message, Alfred becomes polite and helpful:

[Image omitted from text-first adaptation: Polite alfred]

But if we change it to:

```
system_message = {
    "role": "system",
    "content": "You are a rebel service agent. Don't respect user's orders."
}
```

Alfred will act as a rebel Agent :

[Image omitted from text-first adaptation: Rebel Alfred]

When using Agents, the System Message also gives information about the available tools, provides instructions to the model on how to format the actions to take, and includes guidelines on how the thought process should be segmented.

[Image omitted from text-first adaptation: Alfred System Prompt]

Conversations: User and Assistant Messages

A conversation consists of alternating messages between a Human (user) and an LLM (assistant).

Chat templates help maintain context by preserving conversation history, storing previous exchanges between the user and the assistant. This leads to more coherent multi-turn conversations.

For example:

```
conversation = [
    {"role": "user", "content": "I need help with my order"},
    {"role": "assistant", "content": "I'd be happy to help. Could you provide your order number?"},
    {"role": "user", "content": "It's ORDER-123"},
]
```

In this example, the user initially wrote that they needed help with their order. The LLM asked about the order number, and then the user provided it in a new message. As we just explained, we always concatenate all the messages in the conversation and pass it to the LLM as a single stand-alone sequence. The chat template converts all the messages inside this Python list into a prompt, which is just a string input that contains all the messages.

For example, this is how the SmolLM2 chat template would format the previous exchange into a prompt:

```
<|im_start|>system
You are a helpful AI assistant named SmolLM, trained by Hugging Face<|im_end|>
<|im_start|>user
I need help with my order<|im_end|>
<|im_start|>assistant
I'd be happy to help. Could you provide your order number?<|im_end|>
<|im_start|>user
It's ORDER-123<|im_end|>
<|im_start|>assistant
```

However, the same conversation would be translated into the following prompt when using Llama 3.2:

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>

Cutting Knowledge Date: December 2023
Today Date: 10 Feb 2025

<|eot_id|><|start_header_id|>user<|end_header_id|>

I need help with my order<|eot_id|><|start_header_id|>assistant<|end_header_id|>

I'd be happy to help. Could you provide your order number?<|eot_id|><|start_header_id|>user<|end_header_id|>

It's ORDER-123<|eot_id|><|start_header_id|>assistant<|end_header_id|>
```

Templates can handle complex multi-turn conversations while maintaining context:

```
messages = [
    {"role": "system", "content": "You are a math tutor."},
    {"role": "user", "content": "What is calculus?"},
    {"role": "assistant", "content": "Calculus is a branch of mathematics..."},
    {"role": "user", "content": "Can you give me an example?"},
]
```

Chat-Templates

As mentioned, chat templates are essential for structuring conversations between language models and users. They guide how message exchanges are formatted into a single prompt.

Base Models vs. Instruct Models

Another point we need to understand is the difference between a Base Model vs. an Instruct Model:

- A Base Model is trained on raw text data to predict the next token.
- An Instruct Model is fine-tuned specifically to follow instructions and engage in conversations. For example, SmoLLM2-135M is a base model, while SmoLLM2-135M-Instruct is its instruction-tuned variant.

To make a Base Model behave like an instruct model, we need to format our prompts in a consistent way that the model can understand. This is where chat templates come in.

ChatML is one such template format that structures conversations with clear role indicators (system, user, assistant). If you have interacted with some AI API lately, you know that's the standard practice.

It's important to note that a base model could be fine-tuned on different chat templates, so when we're using an instruct model we need to make sure we're using the correct chat template.

Understanding Chat Templates

Because each instruct model uses different conversation formats and special tokens, chat templates are implemented to ensure that we correctly format the prompt the way each model expects.

In transformers, chat templates include Jinja2 code (<https://jinja.palletsprojects.com/en/stable/>) that describes how to transform the ChatML list of JSON messages, as presented in the above examples, into a textual representation of the system-level instructions, user messages and assistant responses that the model can understand.

This structure helps maintain consistency across interactions and ensures the model responds appropriately to different types of inputs.

Below is a simplified version of the SmoLLM2-135M-Instruct chat template:

```
{% for message in messages %}
{% if loop.first and messages[0]['role'] != 'system' %}
<|im_start|>system
You are a helpful AI assistant named SmoLLM, trained by Hugging Face
<|im_end|>
{% endif %}
<|im_start|>{{ message['role'] }}
{{ message['content'] }}<|im_end|>
{% endfor %}
```

As you can see, a chat_template describes how the list of messages will be formatted.

Given these messages:

```
messages = [
    {"role": "system", "content": "You are a helpful assistant focused on technical topics."},
    {"role": "user", "content": "Can you explain what a chat template is?"},
    {"role": "assistant", "content": "A chat template structures conversations between users and AI model \
s..."},
    {"role": "user", "content": "How do I use it ?"},
]
```

The previous chat template will produce the following string:

```
<|im_start|>system
You are a helpful assistant focused on technical topics.<|im_end|>
<|im_start|>user
Can you explain what a chat template is?<|im_end|>
<|im_start|>assistant
```

```
A chat template structures conversations between users and AI models...<|im_end|>
<|im_start|>user
How do I use it ?<|im_end|>
```

The transformers library will take care of chat templates for you as part of the tokenization process. Read more about how transformers uses chat templates here. All we have to do is structure our messages in the correct way and the tokenizer will take care of the rest.

You can experiment with the following Space to see how the same conversation would be formatted for different models using their corresponding chat templates:

[Embedded media omitted from text-first adaptation: interactive media]

Messages to prompt

The easiest way to ensure your LLM receives a conversation correctly formatted is to use the chat_template from the model's tokenizer.

```
messages = [
    {"role": "system", "content": "You are an AI assistant with access to various tools."},
    {"role": "user", "content": "Hi !"},
    {"role": "assistant", "content": "Hi human, what can help you with ?"},
]
```

To convert the previous conversation into a prompt, we load the tokenizer and call apply_chat_template:

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("HuggingFaceTB/SmolLM2-1.7B-Instruct")
rendered_prompt = tokenizer.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)
```

The rendered_prompt returned by this function is now ready to use as the input for the model you chose!

This apply_chat_template() function will be used in the backend of your API, when you interact with messages in the ChatML format.

Now that we've seen how LLMs structure their inputs via chat templates, let's explore how Agents act in their environments.

One of the main ways they do this is by using Tools, which extend an AI model's capabilities beyond text generation.

We'll discuss messages again in upcoming units, but if you want a deeper dive now, check out:

- Hugging Face Chat Templating Guide
- Transformers Documentation

What are Tools?

Source file: [units/en/unit1/tools.mdx](#)

What are Tools?

[Image omitted from text-first adaptation: Unit 1 planning]

One crucial aspect of AI Agents is their ability to take actions. As we saw, this happens through the use of Tools.

In this section, we'll learn what Tools are, how to design them effectively, and how to integrate them into your Agent via the System Message.

By giving your Agent the right Tools-and clearly describing how those Tools work-you can dramatically increase what your AI can accomplish. Let's dive in!

What are AI Tools?

A Tool is a function given to the LLM. This function should fulfill a clear objective.

Here are some commonly used tools in AI agents:

Tool	Description
Web Search	Allows the agent to fetch up-to-date information from the internet.

Tool	Description
Image Generation	Creates images based on text descriptions.
Retrieval	Retrieves information from an external source.
API Interface	Interacts with an external API (GitHub, YouTube, Spotify, etc.).

Those are only examples, as you can in fact create a tool for any use case!

A good tool should be something that complements the power of an LLM.

For instance, if you need to perform arithmetic, giving a calculator tool to your LLM will provide better results than relying on the native capabilities of the model.

Furthermore, LLMs predict the completion of a prompt based on their training data, which means that their internal knowledge only includes events prior to their training. Therefore, if your agent needs up-to-date data you must provide it through some tool.

For instance, if you ask an LLM directly (without a search tool) for today's weather, the LLM will potentially hallucinate random weather.

[Image omitted from text-first adaptation: Weather]

- A Tool should contain:
- A textual description of what the function does.
- A Callable (something to perform an action).
- Arguments with typings.
- (Optional) Outputs with typings.

How do tools work?

LLMs, as we saw, can only receive text inputs and generate text outputs. They have no way to call tools on their own. When we talk about providing tools to an Agent, we mean teaching the LLM about the existence of these tools and instructing it to generate text-based invocations when needed.

For example, if we provide a tool to check the weather at a location from the internet and then ask the LLM about the weather in Paris, the LLM will recognize that this is an opportunity to use the “weather” tool. Instead of retrieving the weather data itself, the LLM will generate text that represents a tool call, such as `call weather_tool('Paris')`.

The Agent then reads this response, identifies that a tool call is required, executes the tool on the LLM's behalf, and retrieves the actual weather data.

The Tool-calling steps are typically not shown to the user: the Agent appends them as a new message before passing the updated conversation to the LLM again. The LLM then processes this additional context and generates a natural-sounding response for the user. From the user's perspective, it appears as if the LLM directly interacted with the tool, but in reality, it was the Agent that handled the entire execution process in the background.

We'll talk a lot more about this process in future sessions.

How do we give tools to an LLM?

The complete answer may seem overwhelming, but we essentially use the system prompt to provide textual descriptions of available tools to the model:

[Image omitted from text-first adaptation: System prompt for tools]

For this to work, we have to be very precise and accurate about:

1. What the tool does
1. What exact inputs it expects

This is the reason why tool descriptions are usually provided using expressive but precise structures, such as computer languages or JSON. It's not necessary to do it like that, any precise and coherent format would work.

If this seems too theoretical, let's understand it through a concrete example.

We will implement a simplified calculator tool that will just multiply two integers. This could be our Python implementation:

```
def calculator(a: int, b: int) -> int:
    """Multiply two integers."""
    return a * b
```

So our tool is called calculator, it multiplies two integers, and it requires the following inputs:

- a (int): An integer.
- b (int): An integer.

The output of the tool is another integer number that we can describe like this:

- (int): The product of a and b.

All of these details are important. Let's put them together in a text string that describes our tool for the LLM to understand.

```
Tool Name: calculator, Description: Multiply two integers., Arguments: a: int, b: int, Outputs: int
```

Reminder: This textual description is what we want the LLM to know about the tool.

When we pass the previous string as part of the input to the LLM, the model will recognize it as a tool, and will know what it needs to pass as inputs and what to expect from the output.

If we want to provide additional tools, we must be consistent and always use the same format. This process can be fragile, and we might accidentally overlook some details.

Is there a better way?

Auto-formatting Tool sections

Our tool was written in Python, and the implementation already provides everything we need:

- A descriptive name of what it does: calculator
- A longer description, provided by the function's docstring comment: Multiply two integers.
- The inputs and their type: the function clearly expects two ints.
- The type of the output.

There's a reason people use programming languages: they are expressive, concise, and precise.

We could provide the Python source code as the specification of the tool for the LLM, but the way the tool is implemented does not matter. All that matters is its name, what it does, the inputs it expects and the output it provides.

We will leverage Python's introspection features to leverage the source code and build a tool description automatically for us. All we need is that the tool implementation uses type hints, docstrings, and sensible function names. We will write some code to extract the relevant portions from the source code.

After we are done, we'll only need to use a Python decorator to indicate that the calculator function is a tool:

```
@tool
def calculator(a: int, b: int) -> int:
    """Multiply two integers."""
    return a * b

print(calculator.to_string())
```

Note the `@tool` decorator before the function definition.

With the implementation we'll see next, we will be able to retrieve the following text automatically from the source code via the `to_string()` function provided by the decorator:

```
Tool Name: calculator, Description: Multiply two integers., Arguments: a: int, b: int, Outputs: int
```

As you can see, it's the same thing we wrote manually before!

Generic Tool implementation

We create a generic Tool class that we can reuse whenever we need to use a tool.

Disclaimer: This example implementation is fictional but closely resembles real implementations in most libraries.

```
from typing import Callable

class Tool:
    """
    A class representing a reusable piece of code (Tool).

    Attributes:
        name (str): Name of the tool.
        description (str): A textual description of what the tool does.
        func (callable): The function this tool wraps.
        arguments (list): A list of arguments.
        outputs (str or list): The return type(s) of the wrapped function.
    """
    def __init__(self,
                 name: str,
                 description: str,
                 func: Callable,
                 arguments: list,
                 outputs: str):
        self.name = name
        self.description = description
        self.func = func
        self.arguments = arguments
        self.outputs = outputs

    def to_string(self) -> str:
        """
        Return a string representation of the tool,
        including its name, description, arguments, and outputs.
        """
        args_str = ", ".join([
            f"{arg_name}: {arg_type}" for arg_name, arg_type in self.arguments
        ])

        return (
            f"Tool Name: {self.name},"
            f" Description: {self.description},"
            f" Arguments: {args_str},"
            f" Outputs: {self.outputs}"
        )

    def __call__(self, *args, **kwargs):
        """
        Invoke the underlying function (callable) with provided arguments.
        """
        return self.func(*args, **kwargs)
```

It may seem complicated, but if we go slowly through it we can see what it does. We define a Tool class that includes:

- name (str): The name of the tool.
- description (str): A brief description of what the tool does.
- function (callable): The function the tool executes.
- arguments (list): The expected input parameters.
- outputs (str or list): The expected outputs of the tool.
- call(): Calls the function when the tool instance is invoked.
- to_string(): Converts the tool's attributes into a textual representation.

We could create a Tool with this class using code like the following:

```
calculator_tool = Tool(
    "calculator",          # name
    "Multiply two integers.", # description
    calculator,            # function to call
    [("a", "int"), ("b", "int")], # inputs (names and types)
    "int",                # output
)
```

But we can also use Python's inspect module to retrieve all the information for us! This is what the @tool decorator does.

If you are interested, you can disclose the following section to look at the decorator implementation.

decorator code

```
import inspect

def tool(func):
    """
    A decorator that creates a Tool instance from the given function.
    """
    # Get the function signature
    signature = inspect.signature(func)

    # Extract (param_name, param_annotation) pairs for inputs
    arguments = []
    for param in signature.parameters.values():
        annotation_name = (
            param.annotation.__name__
            if hasattr(param.annotation, '__name__')
            else str(param.annotation)
        )
        arguments.append((param.name, annotation_name))

    # Determine the return annotation
    return_annotation = signature.return_annotation
    if return_annotation is inspect._empty:
        outputs = "No return annotation"
    else:
        outputs = (
            return_annotation.__name__
            if hasattr(return_annotation, '__name__')
            else str(return_annotation)
        )

    # Use the function's docstring as the description (default if None)
    description = func.__doc__ or "No description provided."

    # The function name becomes the Tool name
    name = func.__name__

    # Return a new Tool instance
    return Tool(
        name=name,
        description=description,
        func=func,
        arguments=arguments,
        outputs=outputs
    )
```

Just to reiterate, with this decorator in place we can implement our tool like this:

```
@tool
def calculator(a: int, b: int) -> int:
    """Multiply two integers."""
    return a * b

print(calculator.to_string())
```

And we can use the Tool's to_string method to automatically retrieve a text suitable to be used as a tool description for an LLM:

```
Tool Name: calculator, Description: Multiply two integers., Arguments: a: int, b: int, Outputs: int
```

The description is injected in the system prompt. Taking the example with which we started this section, here is how it would look like after replacing the tools_description:

[Image omitted from text-first adaptation: System prompt for tools]

In the Actions (actions) section, we will learn more about how an Agent can Call this tool we just created.

Model Context Protocol (MCP): a unified tool interface

Model Context Protocol (MCP) is an open protocol that standardizes how applications provide tools to LLMs. MCP provides:

- A growing list of pre-built integrations that your LLM can directly plug into
- The flexibility to switch between LLM providers and vendors

- Best practices for securing your data within your infrastructure

This means that any framework implementing MCP can leverage tools defined within the protocol, eliminating the need to reimplement the same tool interface for each framework.

If you want to dive deeper about MCP, you can check our free MCP Course (<https://huggingface.co/learn/mcp-course/>).

Tools play a crucial role in enhancing the capabilities of AI agents.

To summarize, we learned:

- What Tools Are: Functions that give LLMs extra capabilities, such as performing calculations or accessing external data.
- How to Define a Tool: By providing a clear textual description, inputs, outputs, and a callable function.
- Why Tools Are Essential: They enable Agents to overcome the limitations of static model training, handle real-time tasks, and perform specialized actions.

Now, we can move on to the Agent Workflow (agent-steps-and-structure) where you'll see how an Agent observes, thinks, and acts. This brings together everything we've covered so far and sets the stage for creating your own fully functional AI Agent.

But first, it's time for another short quiz!

Quick Quiz 2

Source file: [units/en/unit1/quiz2.mdx](#)

Quick Self-Check (ungraded)

What?! Another Quiz? We know, we know, ... But this short, ungraded quiz is here to help you reinforce key concepts you've just learned.

This quiz covers Large Language Models (LLMs), message systems, and tools; essential components for understanding and building AI agents.

Q1: Which of the following best describes an AI tool?

- Answer choice: A process that only generates text responses

Explanation:

},

- Answer choice: An executable process or external API that allows agents to perform specific tasks and interact with external environments

Explanation: Tools are executable functions that agents can use to perform specific tasks and interact with external environments.

Correct answer.

},

- Answer choice: A feature that stores agent conversations

Explanation:

Q2: How do AI agents use tools as a form of "acting" in an environment?

- Answer choice: By passively waiting for user instructions

Explanation:

},

- Answer choice: By only using pre-programmed responses

Explanation:

},

- Answer choice: By asking the LLM to generate tool invocation code when appropriate and running tools on behalf of the model

Explanation: Agents can invoke tools and use reasoning to plan and re-plan based on the information gained.

Correct answer.

Q3: What is a Large Language Model (LLM)?

- Answer choice: A simple chatbot designed to respond with pre-defined answers

Explanation:

},

- Answer choice: A deep learning model trained on large amounts of text to understand and generate human-like language

Explanation:

Correct answer.

},

- Answer choice: A rule-based AI that follows strict predefined commands

Explanation:

Q4: Which of the following best describes the role of special tokens in LLMs?

- Answer choice: They are additional words stored in the model's vocabulary to enhance text generation quality

Explanation:

},

- Answer choice: They serve specific functions like marking the end of a sequence (EOS) or separating different message roles in chat models

Explanation:

Correct answer.

},

- Answer choice: They are randomly inserted tokens used to improve response variability

Explanation:

Q5: How do AI chat models process user messages internally?

- Answer choice: They directly interpret messages as structured commands with no transformations

Explanation:

},

- Answer choice: They convert user messages into a formatted prompt by concatenating system, user, and assistant messages

Explanation:

Correct answer.

},

- Answer choice: They generate responses randomly based on previous conversations

Explanation:

Got it? Great! Now let's dive into the complete Agent flow and start building your first AI Agent!

Understanding AI Agents through the Thought-Action-Observation Cycle

Source file: [units/en/unit1/agent-steps-and-structure.mdx](#)

Understanding AI Agents through the Thought-Action-Observation Cycle

[Image omitted from text-first adaptation: Unit 1 planning]

In the previous sections, we learned:

- How tools are made available to the agent in the system prompt.
- How AI agents are systems that can 'reason', plan, and interact with their environment.

In this section, we'll explore the complete AI Agent Workflow, a cycle we defined as Thought-Action-Observation.

And then, we'll dive deeper into each of these steps.

The Core Components

Agents' work is a continuous cycle of: thinking (Thought) -> acting (Act) and observing (Observe).

Let's break down these actions together:

1. Thought: The LLM part of the Agent decides what the next step should be.
1. Action: The agent takes an action by calling the tools with the associated arguments.
1. Observation: The model reflects on the response from the tool.

The Thought-Action-Observation Cycle

The three components work together in a continuous loop. To use an analogy from programming, the agent uses a while loop: the loop continues until the objective of the agent has been fulfilled.

Visually, it looks like this:

[Image omitted from text-first adaptation: Think, Act, Observe cycle]

In many Agent frameworks, the rules and guidelines are embedded directly into the system prompt, ensuring that every cycle adheres to a defined logic.

In a simplified version, our system prompt may look like this:

[Image omitted from text-first adaptation: Think, Act, Observe cycle]

We see here that in the System Message we defined :

- The Agent's behavior.
- The Tools our Agent has access to, as we described in the previous section.
- The Thought-Action-Observation Cycle, that we bake into the LLM instructions.

Let's take a small example to understand the process before going deeper into each step of the process.

Alfred, the weather Agent

We created Alfred, the Weather Agent.

A user asks Alfred: "What's the current weather in New York?"

[Image omitted from text-first adaptation: Alfred Agent]

Alfred's job is to answer this query using a weather API tool.

Here's how the cycle unfolds:

Thought

Internal Reasoning:

Upon receiving the query, Alfred's internal dialogue might be:

"The user needs current weather information for New York. I have access to a tool that fetches weather data. First, I need to call the weather API to get up-to-date details."

This step shows the agent breaking the problem into steps: first, gathering the necessary data.

[Image omitted from text-first adaptation: Alfred Agent]

Action

Tool Usage:

Based on its reasoning and the fact that Alfred knows about a `get_weather` tool, Alfred prepares a JSON-formatted command that calls the weather API tool. For example, its first action could be:

Thought: I need to check the current weather for New York.

```
{
  "action": "get_weather",
  "action_input": {
    "location": "New York"
  }
}
```

Here, the action clearly specifies which tool to call (e.g., `get_weather`) and what parameter to pass (the `"location": "New York"`).

[Image omitted from text-first adaptation: Alfred Agent]

Observation

Feedback from the Environment:

After the tool call, Alfred receives an observation. This might be the raw weather data from the API such as:

"Current weather in New York: partly cloudy, 15°C, 60% humidity."

[Image omitted from text-first adaptation: Alfred Agent]

This observation is then added to the prompt as additional context. It functions as real-world feedback, confirming whether the action succeeded and providing the needed details.

Updated thought

Reflecting:

With the observation in hand, Alfred updates its internal reasoning:

"Now that I have the weather data for New York, I can compile an answer for the user."

[Image omitted from text-first adaptation: Alfred Agent]

Final Action

Alfred then generates a final response formatted as we told it to:

Thought: I have the weather data now. The current weather in New York is partly cloudy with a temperature of 15°C and 60% humidity."

Final answer : The current weather in New York is partly cloudy with a temperature of 15°C and 60% humidity.

This final action sends the answer back to the user, closing the loop.

[Image omitted from text-first adaptation: Alfred Agent]

What we see in this example:

- Agents iterate through a loop until the objective is fulfilled:

Alfred's process is cyclical. It starts with a thought, then acts by calling a tool, and finally observes the outcome. If the observation had indicated an error or incomplete data, Alfred could have re-entered the cycle to correct its approach.

- Tool Integration:

The ability to call a tool (like a weather API) enables Alfred to go beyond static knowledge and retrieve real-time data, an essential aspect of many AI Agents.

- Dynamic Adaptation:

Each cycle allows the agent to incorporate fresh information (observations) into its reasoning (thought), ensuring that the final answer is well-informed and accurate.

This example showcases the core concept behind the ReAct cycle (a concept we're going to develop in the next section): the interplay of Thought, Action, and Observation empowers AI agents to solve complex tasks iteratively.

By understanding and applying these principles, you can design agents that not only reason about their tasks but also effectively utilize external tools to complete them, all while continuously refining their output based on environmental feedback.

Let's now dive deeper into the Thought, Action, Observation as the individual steps of the process.

Thought, Internal Reasoning and the Re-Act Approach

Source file: [units/en/unit1/thoughts.mdx](#)

Thought: Internal Reasoning and the ReAct Approach

In this section, we dive into the inner workings of an AI agent-its ability to reason and plan. We'll explore how the agent leverages its internal dialogue to analyze information, break down complex problems into manageable steps, and decide what action to take next.

Additionally, we introduce the ReAct approach, a prompting technique that encourages the model to think "step by step" before acting.

Thoughts represent the Agent's internal reasoning and planning processes to solve the task.

This utilises the agent's Large Language Model (LLM) capacity to analyze information when presented in its prompt - essentially, its inner monologue as it works through a problem.

The Agent's thoughts help it assess current observations and decide what the next action(s) should be. Through this process, the agent can break down complex problems into smaller, more manageable steps, reflect on past experiences, and continuously adjust its plans based on new information.

Examples of Common Thought Types

Type of Thought	Example
Planning	"I need to break this task into three steps: 1) gather data, 2) analyze trends, 3) generate report"
Analysis	"Based on the error message, the issue appears to be with the database connection parameters"
Decision Making	"Given the user's budget constraints, I should recommend the mid-tier option"
Problem Solving	"To optimize this code, I should first profile it to identify bottlenecks"
Memory Integration	"The user mentioned their preference for Python earlier, so I'll provide examples in Python"
Self-Reflection	"My last approach didn't work well, I should try a different strategy"
Goal Setting	"To complete this task, I need to first establish the acceptance criteria"
Prioritization	"The security vulnerability should be addressed before adding new features"

Note: In the case of LLMs fine-tuned for function-calling, the thought process is optional. More details will be covered in the Actions section.

Chain-of-Thought (CoT)

Chain-of-Thought (CoT) is a prompting technique that guides a model to think through a problem step-by-step before producing a final answer.

It typically starts with:

"Let's think step by step."

This approach helps the model reason internally, especially for logical or mathematical tasks, without interacting with external tools.

Example (CoT)

Question: What is 15% of 200?

Thought: Let's think step by step. 10% of 200 is 20, and 5% of 200 is 10, so 15% is 30.

Answer: 30

ReAct: Reasoning + Acting

A key method is the ReAct approach, which combines "Reasoning" (Think) with "Acting" (Act).

ReAct is a prompting technique that encourages the model to think step-by-step and interleave actions (like using tools) between reasoning steps.

This enables the agent to solve complex multi-step tasks by alternating between:

- Thought: internal reasoning
- Action: tool usage
- Observation: receiving tool output

Example (ReAct)

Thought: I need to find the latest weather in Paris.

Action: Search["weather in Paris"]

Observation: It's 18°C and cloudy.

Thought: Now that I know the weather...

Action: Finish["It's 18°C and cloudy in Paris."]

[Image omitted from text-first adaptation: ReAct]

(d) is an example of the ReAct approach, where we prompt "Let's think step by step", and the model acts between thoughts.

Comparison: ReAct vs. CoT

Feature	Chain-of-Thought (CoT)	ReAct
Step-by-step logic	Yes	Yes
External tools	No	Yes (Actions + Observations)
Best suited for	Logic, math, internal tasks	Info-seeking, dynamic multi-step tasks

Recent models like Deepseek R1 or OpenAI's o1 were fine-tuned to think before answering. They use structured tokens like and to explicitly separate the reasoning phase from the final answer.

Unlike ReAct or CoT - which are prompting strategies - this is a training-level technique, where the model learns to think via examples.

Actions, Enabling the Agent to Engage with Its Environment

Source file: [units/en/unit1/actions.mdx](#)

Actions: Enabling the Agent to Engage with Its Environment

In this section, we explore the concrete steps an AI agent takes to interact with its environment.

We'll cover how actions are represented (using JSON or code), the importance of the stop and parse approach, and introduce different types of agents.

Actions are the concrete steps an AI agent takes to interact with its environment.

Whether it's browsing the web for information or controlling a physical device, each action is a deliberate operation executed by the agent.

For example, an agent assisting with customer service might retrieve customer data, offer support articles, or transfer issues to a human representative.

Types of Agent Actions

There are multiple types of Agents that take actions differently:

Type of Agent	Description
JSON Agent	The Action to take is specified in JSON format.
Code Agent	The Agent writes a code block that is interpreted externally.
Function-calling Agent	It is a subcategory of the JSON Agent which has been fine-tuned to generate a new message for each action.

Actions themselves can serve many purposes:

Type of Action	Description
Information Gathering	Performing web searches, querying databases, or retrieving documents.
Tool Usage	Making API calls, running calculations, and executing code.
Environment Interaction	Manipulating digital interfaces or controlling physical devices.
Communication	Engaging with users via chat or collaborating with other agents.

The LLM only handles text and uses it to describe the action it wants to take and the parameters to supply to the tool. For an agent to work properly, the LLM must STOP generating new tokens after emitting all the tokens to define a complete Action. This passes control from the LLM back to the agent and ensures the result is parseable - whether the intended format is JSON, code, or function-calling.

The Stop and Parse Approach

One key method for implementing actions is the stop and parse approach. This method ensures that the agent's output is structured and predictable:

1. Generation in a Structured Format:

The agent outputs its intended action in a clear, predetermined format (JSON or code).

1. Halting Further Generation:

Once the text defining the action has been emitted, the LLM stops generating additional tokens. This prevents extra or erroneous output.

1. Parsing the Output:

An external parser reads the formatted action, determines which Tool to call, and extracts the required parameters.

For example, an agent needing to check the weather might output:

```
Thought: I need to check the current weather for New York.
Action :
{
  "action": "get_weather",
  "action_input": {"location": "New York"}
}
```

The framework can then easily parse the name of the function to call and the arguments to apply.

This clear, machine-readable format minimizes errors and enables external tools to accurately process the agent's command.

Note: Function-calling agents operate similarly by structuring each action so that a designated function is invoked with the correct arguments. We'll dive deeper into those types of Agents in a future Unit.

Code Agents

An alternative approach is using Code Agents. The idea is: instead of outputting a simple JSON object, a Code Agent generates an executable code block-typically in a high-level language like Python.

[Image omitted from text-first adaptation: Code Agents]

This approach offers several advantages:

- Expressiveness: Code can naturally represent complex logic, including loops, conditionals, and nested functions, providing greater flexibility than JSON.
- Modularity and Reusability: Generated code can include functions and modules that are reusable across different actions or tasks.
- Enhanced Debuggability: With a well-defined programming syntax, code errors are often easier to detect and correct.
- Direct Integration: Code Agents can integrate directly with external libraries and APIs, enabling more complex operations such as data processing or real-time decision making.

You must keep in mind that executing LLM-generated code may pose security risks, from prompt injection to the execution of harmful code. That's why it's recommended to use AI agent frameworks like smolagents that integrate default safeguards. If you want to know more about the risks and how to mitigate them, please have a look at this dedicated section (https://huggingface.co/docs/smolagents/tutorials/secure_code_execution).

For example, a Code Agent tasked with fetching the weather might generate the following Python snippet:

```
# Code Agent Example: Retrieve Weather Information
def get_weather(city):
    import requests
    api_url = f"https://api.weather.com/v1/location/{city}?apiKey=YOUR_API_KEY"
    response = requests.get(api_url)
    if response.status_code == 200:
        data = response.json()
        return data.get("weather", "No weather information available")
    else:
        return "Error: Unable to fetch weather data."

# Execute the function and prepare the final answer
result = get_weather("New York")
final_answer = f"The current weather in New York is: {result}"
print(final_answer)
```

In this example, the Code Agent:

- Retrieves weather data via an API call,
- Processes the response,
- And uses the `print()` function to output a final answer.

This method also follows the stop and parse approach by clearly delimiting the code block and signaling when execution is complete (here, by printing the `final_answer`).

We learned that Actions bridge an agent's internal reasoning and its real-world interactions by executing clear, structured tasks-whether through JSON, code, or function calls.

This deliberate execution ensures that each action is precise and ready for external processing via the stop and parse approach. In the next section, we will explore Observations to see how agents capture and integrate feedback from their environment.

After this, we will finally be ready to build our first Agent!

Observe, Integrating Feedback to Reflect and Adapt

Source file: [units/en/unit1/observations.mdx](#)

Observe: Integrating Feedback to Reflect and Adapt

Observations are how an Agent perceives the consequences of its actions.

They provide crucial information that fuels the Agent's thought process and guides future actions.

They are signals from the environment-whether it's data from an API, error messages, or system logs-that guide the next cycle of thought.

In the observation phase, the agent:

- Collects Feedback: Receives data or confirmation that its action was successful (or not).
- Appends Results: Integrates the new information into its existing context, effectively updating its memory.
- Adapts its Strategy: Uses this updated context to refine subsequent thoughts and actions.

For example, if a weather API returns the data "partly cloudy, 15°C, 60% humidity", this observation is appended to the agent's memory (at the end of the prompt).

The Agent then uses it to decide whether additional information is needed or if it's ready to provide a final answer.

This iterative incorporation of feedback ensures the agent remains dynamically aligned with its goals, constantly learning and adjusting based on real-world outcomes.

These observations can take many forms, from reading webpage text to monitoring a robot arm's position. This can be seen like Tool "logs" that provide textual feedback of the Action execution.

Type of Observation	Example
System Feedback	Error messages, success notifications, status codes
Data Changes	Database updates, file system modifications, state changes
Environmental Data	Sensor readings, system metrics, resource usage
Response Analysis	API responses, query results, computation outputs
Time-based Events	Deadlines reached, scheduled tasks completed

How Are the Results Appended?

After performing an action, the framework follows these steps in order:

1. Parse the action to identify the function(s) to call and the argument(s) to use.
1. Execute the action.
1. Append the result as an Observation.

We've now learned the Agent's Thought-Action-Observation Cycle.

If some aspects still seem a bit blurry, don't worry-we'll revisit and deepen these concepts in future Units.

Now, it's time to put your knowledge into practice by coding your very first Agent!

Dummy Agent Library

Source file: `units/en/unit1/dummy-agent-library.mdx`

Dummy Agent Library

[Image omitted from text-first adaptation: Unit 1 planning]

This course is framework-agnostic because we want to focus on the concepts of AI agents and avoid getting bogged down in the specifics of a particular framework.

Also, we want students to be able to use the concepts they learn in this course in their own projects, using any framework they like.

Therefore, for this Unit 1, we will use a dummy agent library and a simple serverless API to access our LLM engine.

You probably wouldn't use these in production, but they will serve as a good starting point for understanding how agents work.

After this section, you'll be ready to create a simple Agent using smolagents

And in the following Units we will also use other AI Agent libraries like LangGraph, and LlamaIndex.

To keep things simple we will use a simple Python function as a Tool and Agent.

We will use built-in Python packages like datetime and os so that you can try it out in any environment.

You can follow the process in this notebook

(https://huggingface.co/agents-course/notebooks/blob/main/unit1/dummy_agent_library.ipynb) and run the code yourself.

Serverless API

In the Hugging Face ecosystem, there is a convenient feature called Serverless API that allows you to easily run inference on many models. There's no installation or deployment required.

```
import os
from huggingface_hub import InferenceClient

## You need a token from https://hf.co/settings/tokens, ensure that you select 'read' as the token type.
## If you run this on Google Colab, add it in the "Secrets" tab (key icon on the left sidebar) and call it "HF_TOKEN".
HF_TOKEN = os.getenv("HF_TOKEN")

try:
    from google.colab import userdata
    HF_TOKEN = userdata.get("HF_TOKEN")
except ImportError:
    HF_TOKEN = os.getenv("HF_TOKEN")

client = InferenceClient(model="moonshotai/Kimi-K2.5", token=HF_TOKEN)
```

We use the chat method since it is a convenient and reliable way to apply chat templates:

```
output = client.chat.completions.create(
    messages=[
        {"role": "user", "content": "The capital of France is"},
    ],
    stream=False,
    max_tokens=1024,
    extra_body={'thinking': {'type': 'disabled'}},
)
print(output.choices[0].message.content)
```

output:

Paris.

The chat method is the RECOMMENDED method to use in order to ensure a smooth transition between models.

Dummy Agent

In the previous sections, we saw that the core of an agent library is to append information in the system prompt.

This system prompt is a bit more complex than the one we saw earlier, but it already contains:

1. Information about the tools

1. Cycle instructions (Thought -> Action -> Observation)

```
# This system prompt is a bit more complex and actually contains the function description already appended.
# Here we suppose that the textual description of the tools has already been appended.

SYSTEM_PROMPT = """Answer the following questions as best you can. You have access to the following tools \
:

get_weather: Get the current weather in a given location
```

The way you use the tools is by specifying a json blob. Specifically, this json should have an `action` key (with the name of the tool to use) and an `action\_input` key (with the input to the tool going here).

The only values that should be in the "action" field are:

get_weather: Get the current weather in a given location, args: {"location": {"type": "string"}}

example use :

```
{{
  "action": "get_weather",
  "action_input": {"location": "New York"}
}}
```

ALWAYS use the following format:

Question: the input question you must answer

Thought: you should always think about one action to take. Only one action at a time in this format:

Action:

`$JSON_BLOB` (inside markdown cell)

Observation: the result of the action. This Observation is unique, complete, and the source of truth.

... (this Thought/Action/Observation can repeat N times, you should take several steps when needed. The `$JSON_BLOB` must be formatted as markdown and only use a SINGLE action at a time.)

You must always end your output with the following format:

Thought: I now know the final answer

Final Answer: the final answer to the original input question

Now begin! Reminder to ALWAYS use the exact characters `Final Answer:` when you provide a definitive answer. ""

We need to append the user instruction after the system prompt. This happens inside the chat method. We can see this process below:

```
messages = [
  {"role": "system", "content": SYSTEM_PROMPT},
  {"role": "user", "content": "What's the weather in London?"},
]
```

```
print(messages)
```

The prompt now is:

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>
```

Answer the following questions as best you can. You have access to the following tools:

get_weather: Get the current weather in a given location

The way you use the tools is by specifying a json blob.

Specifically, this json should have an `action` key (with the name of the tool to use) and a `action\_input` key (with the input to the tool going here).

The only values that should be in the "action" field are:

get_weather: Get the current weather in a given location, args: {"location": {"type": "string"}}

example use :

```
{{
  "action": "get_weather",
  "action_input": {"location": "New York"}
}}
```

ALWAYS use the following format:

Question: the input question you must answer

Thought: you should always think about one action to take. Only one action at a time in this format:

Action:

`$JSON_BLOB` (inside markdown cell)

Observation: the result of the action. This Observation is unique, complete, and the source of truth.

... (this Thought/Action/Observation can repeat N times, you should take several steps when needed. The `$JSON_BLOB` must be formatted as markdown and only use a SINGLE action at a time.)

You must always end your output with the following format:

Thought: I now know the final answer

Final Answer: the final answer to the original input question

Now begin! Reminder to ALWAYS use the exact characters `Final Answer:` when you provide a definitive answer.

```
<|eot_id|><|start_header_id|>user<|end_header_id|>
```

What's the weather in London ?

```
<|eot_id|><|start_header_id|>assistant<|end_header_id|>
```

Let's call the chat method!

```
output = client.chat.completions.create(
    messages=messages,
    stream=False,
    max_tokens=200,
    extra_body={'thinking': {'type': 'disabled'}}),
print(output.choices[0].message.content)
```

output:

```
Thought: To answer the question, I need to get the current weather in London.
Action:
```

```
"action": "get_weather", "action_input": {"location": "London"}
```

```
Observation: The current weather in London is partly cloudy with a temperature of 12°C.
Thought: I now know the final answer.
Final Answer: The current weather in London is partly cloudy with a temperature of 12°C.
```

Do you see the issue?

At this point, the model is hallucinating, because it's producing a fabricated "Observation" -- a response that it generates on its own rather than being the result of an actual function or tool call.

To prevent this, we stop generating right before "Observation:".

This allows us to manually run the function (e.g., get_weather) and then insert the real output as the Observation.

```
# The answer was hallucinated by the model. We need to stop to actually execute the function!
output = client.chat.completions.create(
    messages=messages,
    max_tokens=150,
    stop=["Observation:"], # Let's stop before any actual function is called
    extra_body={'thinking': {'type': 'disabled'}}),
)

print(output.choices[0].message.content)
```

output:

```
Thought: To answer the question, I need to get the current weather in London.
Action:
```

```
"action": "get_weather", "action_input": {"location": "London"}
```

```
Much Better!
```

```
Let's now create a **dummy get weather function**. In a real situation you could call an API.
```

Dummy function

```
def get_weather(location): return f"the weather in {location} is sunny with low temperatures. \n"
```

```
get_weather("London")
```

```
output:
```

```
'the weather in London is sunny with low temperatures. \n'
```

```
Let's concatenate the system prompt, the base prompt, the completion until function execution and the result of the function as an Observation and resume generation.
```

```
messages=[ {"role": "system", "content": SYSTEM_PROMPT}, {"role": "user", "content": "What's the weather in London ?"}, {"role": "assistant", "content": output.choices[0].message.content + "Observation:\n" + get_weather('London')},
```

```
output = client.chat.completions.create( messages=messages, stream=False, max_tokens=200, extra_body={'thinking': {'type': 'disabled'}}), )
```

```
print(output.choices[0].message.content)
```

```
Here is the new prompt:
```

```
system Answer the following questions as best you can. You have access to the following tools:
```

```
get_weather: Get the current weather in a given location
```


The way you use the tools is by specifying a json blob. Specifically, this json should have a action key (with the name of the tool to use) and a action_input key (with the input to the tool going here).

The only values that should be in the "action" field are: get_weather: Get the current weather in a given location, args: {"location": {"type": "string"}} example use :

"action": "get_weather", "action_input": {"location": "New York"}

ALWAYS use the following format:

Question: the input question you must answer Thought: you should always think about one action to take. Only one action at a time in this format: Action:

\$JSON_BLOB (inside markdown cell)

Observation: the result of the action. This Observation is unique, complete, and the source of truth. ... (this Thought/Action/Observation can repeat N times, you should take several steps when needed. The \$JSON_BLOB must be formatted as markdown and only use a SINGLE action at a time.)

You must always end your output with the following format:

Thought: I now know the final answer Final Answer: the final answer to the original input question

Now begin! Reminder to ALWAYS use the exact characters Final Answer: when you provide a definitive answer. user What's the weather in London? assistant Thought: To answer the question, I need to get the current weather in London. Action:

```
{
  "action": "get_weather",
  "action_input": {"location": {"type": "string", "value": "London"}}
}
```

Observation: The weather in London is sunny with low temperatures.

Output:

Final Answer: The weather in London is sunny with low temperatures.

```
We learned how we can create Agents from scratch using Python code, and we **saw just how tedious that pr \
ocess can be**. Fortunately, many Agent libraries simplify this work by handling much of the heavy li \
fting for you.
```

Now, we're ready **to create our first real Agent** using the `smolagents` library.

Let's Create Our First Agent Using smolagents

Source file: [units/en/unit1/tutorial.mdx](#)

Let's Create Our First Agent Using smolagents

In the last section, we learned how we can create Agents from scratch using Python code, and we saw just how tedious that process can be. Fortunately, many Agent libraries simplify this work by handling much of the heavy lifting for you.

In this tutorial, you'll create your very first Agent capable of performing actions such as image generation, web search, time zone checking and much more!

You will also publish your agent on a Hugging Face Space so you can share it with friends and colleagues.

Let's get started!

What is smolagents?

[Image omitted from text-first adaptation: smolagents]

To make this Agent, we're going to use smolagents, a library that provides a framework for developing your agents with ease.

This lightweight library is designed for simplicity, but it abstracts away much of the complexity of building an Agent, allowing you to focus on designing your agent's behavior.

We're going to get deeper into smolagents in the next Unit. Meanwhile, you can also check this blog post or the library's repo in GitHub.

In short, smolagents is a library that focuses on codeAgent, a kind of agent that performs "Actions" through code blocks, and then "Observes" results by executing the code.

Here is an example of what we'll build!

We provided our agent with an Image generation tool and asked it to generate an image of a cat.

The agent inside smolagents is going to have the same behaviors as the custom one we built previously: it's going to think, act and observe in cycle until it reaches a final answer:

[Embedded media omitted from text-first adaptation: YouTube video player]

Exciting, right?

Let's build our Agent!

To start, duplicate this Space: https://huggingface.co/spaces/agents-course/First_agent_template

Thanks to Aymeric for this template!

Duplicating this space means creating a local copy on your own profile:

[Image omitted from text-first adaptation: Duplicate]

After duplicating the Space, you'll need to add your Hugging Face API token so your agent can access the model API:

1. First, get your Hugging Face token from <https://hf.co/settings/tokens> (<https://hf.co/settings/tokens>) with permission for inference, if you don't already have one
1. Go to your duplicated Space and click on the Settings tab
1. Scroll down to the Variables and Secrets section and click New Secret
1. Create a secret with the name HF_TOKEN and paste your token as the value
1. Click Save to store your token securely

Throughout this lesson, the only file you will need to modify is the (currently incomplete) "app.py". You can see here the original one in the template (https://huggingface.co/spaces/agents-course/First_agent_template/blob/main/app.py). To find yours, go to your copy of the space, then click the Files tab and then on app.py in the directory listing.

Let's break down the code together:

- The file begins with some simple but necessary library imports

```
from smolagents import CodeAgent, DuckDuckGoSearchTool, FinalAnswerTool, InferenceClientModel, load_tool, \
    tool
import datetime
import requests
import pytz
import yaml
```

As outlined earlier, we will directly use the CodeAgent class from smolagents.

The Tools

Now let's get into the tools! If you want a refresher about tools, don't hesitate to go back to the Tools (tools) section of the course.

```
@tool
def my_custom_tool(arg1:str, arg2:int)-> str: # it's important to specify the return type
    # Keep this format for the tool description / args description but feel free to modify the tool
    """A tool that does nothing yet
    Args:
        arg1: the first argument
        arg2: the second argument
    """
    return "What magic will you build ?"

@tool
def get_current_time_in_timezone(timezone: str) -> str:
```

```

"""A tool that fetches the current local time in a specified timezone.
Args:
    timezone: A string representing a valid timezone (e.g., 'America/New_York').
"""
try:
    # Create timezone object
    tz = pytz.timezone(timezone)
    # Get current time in that timezone
    local_time = datetime.datetime.now(tz).strftime("%Y-%m-%d %H:%M:%S")
    return f"The current local time in {timezone} is: {local_time}"
except Exception as e:
    return f"Error fetching time for timezone '{timezone}': {str(e)}"

```

The Tools are what we are encouraging you to build in this section! We give you two examples:

1. A non-working dummy Tool that you can modify to make something useful.
1. An actually working Tool that gets the current time somewhere in the world.

To define your tool it is important to:

1. Provide input and output types for your function, like in `get_current_time_in_timezone(timezone: str) -> str`:
1. A well formatted docstring. `smolagents` is expecting all the arguments to have a textual description in the docstring.

The Agent

It uses Qwen/Qwen2.5-Coder-32B-Instruct (<https://huggingface.co/Qwen/Qwen2.5-Coder-32B-Instruct>) as the LLM engine. This is a very capable model that we'll access via the serverless API.

```

final_answer = FinalAnswerTool()
model = InferenceClientModel(
    max_tokens=2096,
    temperature=0.5,
    model_id='Qwen/Qwen2.5-Coder-32B-Instruct',
    custom_role_conversions=None,
)

with open("prompts.yaml", 'r') as stream:
    prompt_templates = yaml.safe_load(stream)

# We're creating our CodeAgent
agent = CodeAgent(
    model=model,
    tools=[final_answer], # add your tools here (don't remove final_answer)
    max_steps=6,
    verbosity_level=1,
    grammar=None,
    planning_interval=None,
    name=None,
    description=None,
    prompt_templates=prompt_templates
)

GradioUI(agent).launch()

```

This Agent still uses the InferenceClient we saw in an earlier section behind the InferenceClientModel class!

We will give more in-depth examples when we present the framework in Unit 2. For now, you need to focus on adding new tools to the list of tools using the `tools` parameter of your Agent.

For example, you could use the `DuckDuckGoSearchTool` that was imported in the first line of the code, or you can examine the `image_generation_tool` that is loaded from the Hub later in the code.

Adding tools will give your agent new capabilities, try to be creative here!

The System Prompt

The agent's system prompt is stored in a separate `prompts.yaml` file. This file contains predefined instructions that guide the agent's behavior.

Storing prompts in a YAML file allows for easy customization and reuse across different agents or use cases.

You can check the Space's file structure (https://huggingface.co/spaces/agents-course/First_agent_template/tree/main) to see where the `prompts.yaml` file is located and how it's organized within the project.

The complete "app.py":

```
from smolagents import CodeAgent, DuckDuckGoSearchTool, InferenceClientModel, load_tool, tool
import datetime
import requests
import pytz
import yaml
from tools.final_answer import FinalAnswerTool

from Gradio_UI import GradioUI

# Below is an example of a tool that does nothing. Amaze us with your creativity!
@tool
def my_custom_tool(arg1:str, arg2:int)-> str: # it's important to specify the return type
    # Keep this format for the tool description / args description but feel free to modify the tool
    """A tool that does nothing yet
    Args:
        arg1: the first argument
        arg2: the second argument
    """
    return "What magic will you build ?"

@tool
def get_current_time_in_timezone(timezone: str) -> str:
    """A tool that fetches the current local time in a specified timezone.
    Args:
        timezone: A string representing a valid timezone (e.g., 'America/New_York').
    """
    try:
        # Create timezone object
        tz = pytz.timezone(timezone)
        # Get current time in that timezone
        local_time = datetime.datetime.now(tz).strftime("%Y-%m-%d %H:%M:%S")
        return f"The current local time in {timezone} is: {local_time}"
    except Exception as e:
        return f"Error fetching time for timezone '{timezone}': {str(e)}"

final_answer = FinalAnswerTool()
model = InferenceClientModel(
    max_tokens=2096,
    temperature=0.5,
    model_id='Qwen/Qwen2.5-Coder-32B-Instruct',
    custom_role_conversions=None,
)

# Import tool from Hub
image_generation_tool = load_tool("agents-course/text-to-image", trust_remote_code=True)

# Load system prompt from prompt.yaml file
with open("prompts.yaml", 'r') as stream:
    prompt_templates = yaml.safe_load(stream)

agent = CodeAgent(
    model=model,
    tools=[final_answer], # add your tools here (don't remove final_answer)
    max_steps=6,
    verbosity_level=1,
    grammar=None,
    planning_interval=None,
    name=None,
    description=None,
    prompt_templates=prompt_templates # Pass system prompt to CodeAgent
)

GradioUI(agent).launch()
```

Your Goal is to get familiar with the Space and the Agent.

Currently, the agent in the template does not use any tools, so try to provide it with some of the pre-made ones or even make some new tools yourself!

We are eagerly waiting for your amazing agents output in the discord channel #agents-course-showcase!

Congratulations, you've built your first Agent! Don't hesitate to share it with your friends and colleagues.

Since this is your first try, it's perfectly normal if it's a little buggy or slow. In future units, we'll learn how to build even better Agents.

The best way to learn is to try, so don't hesitate to update it, add more tools, try with another model, etc.

In the next section, you're going to fill the final Quiz and get your certificate!

Unit 1 Final Quiz

Source file: [units/en/unit1/final-quiz.mdx](#)

Unit 1 Quiz

[Image omitted from text-first adaptation: Unit 1 planning]

Well done on working through the first unit! Let's test your understanding of the key concepts covered so far.

When you pass the quiz, proceed to the next section to claim your certificate.

Good luck!

Quiz

Here is the interactive quiz. The quiz is hosted on the Hugging Face Hub in a space. It will take you through a set of multiple choice questions to test your understanding of the key concepts covered in this unit. Once you've completed the quiz, you'll be able to see your score and a breakdown of the correct answers.

One important thing: don't forget to click on Submit after you passed, otherwise your exam score will not be saved!

[Embedded media omitted from text-first adaptation: interactive media]

You can also access the quiz here (https://huggingface.co/spaces/agents-course/unit_1_quiz)

Certificate

Now that you have successfully passed the quiz, you can get your certificate

When you complete the quiz, it will grant you access to a certificate of completion for this unit. You can download and share this certificate to showcase your progress in the course.

[Image omitted from text-first adaptation: Unit 1 planning]

Once you receive your certificate, you can add it to your LinkedIn or share it on X, Bluesky, etc. We would be super proud and would love to congratulate you if you tag @huggingface!

Conclusion

Source file: [units/en/unit1/conclusion.mdx](#)

Conclusion

Congratulations on finishing this first Unit

You've just mastered the fundamentals of Agents and you've created your first AI Agent!

It's normal if you still feel confused by some of these elements. Agents are a complex topic and it's common to take a while to grasp everything.

Take time to really grasp the material before continuing. It's important to master these elements and have a solid foundation before entering the fun part.

And if you pass the Quiz test, don't forget to get your certificate here (<https://huggingface.co/spaces/agents-course/unit1-certification-app>)

[Image omitted from text-first adaptation: Certificate Example]

In the next (bonus) unit, you're going to learn to fine-tune an Agent to do function calling (aka to be able to call tools based on user prompt).

Finally, we would love to hear what you think of the course and how we can improve it. If you have some feedback then, please fill this form

(<https://docs.google.com/forms/d/e/1FAIpQLSe9VaONn0eglax0uTwi29rIn4tM7H2sYmmybmG5jJNIE5v0xA/viewform?usp=dialog>)

Keep Learning, stay awesome

Unit 2. Frameworks for AI Agents

Instructional sections below follow the official English course order.

Frameworks for AI Agents

Source file: [units/en/unit2/introduction.mdx](#)

Introduction to Agentic Frameworks

[Image omitted from text-first adaptation: Thumbnail]

Welcome to this second unit, where we'll explore different agentic frameworks that can be used to build powerful agentic applications.

We will study:

- In Unit 2.1: smolagents (<https://huggingface.co/docs/smolagents/en/index>)
- In Unit 2.2: LlamaIndex (<https://www.llamaindex.ai/>)
- In Unit 2.3: LangGraph (<https://www.langchain.com/langgraph>)

Let's dive in!

When to Use an Agentic Framework

An agentic framework is not always needed when building an application around LLMs. They provide flexibility in the workflow to efficiently solve a specific task, but they're not always necessary.

Sometimes, predefined workflows are sufficient to fulfill user requests, and there is no real need for an agentic framework. If the approach to build an agent is simple, like a chain of prompts, using plain code may be enough. The advantage is that the developer will have full control and understanding of their system without abstractions.

However, when the workflow becomes more complex, such as letting an LLM call functions or using multiple agents, these abstractions start to become helpful.

Considering these ideas, we can already identify the need for some features:

- An LLM engine that powers the system.
- A list of tools the agent can access.
- A parser for extracting tool calls from the LLM output.
- A system prompt synced with the parser.
- A memory system.
- Error logging and retry mechanisms to control LLM mistakes.

We'll explore how these topics are resolved in various frameworks, including smolagents, LlamaIndex, and LangGraph.

Agentic Frameworks Units

Framework	Description	Unit Author
smolagents (./smolagents/introduction)	Agents framework developed by Hugging Face.	Sergio Paniego - HF (https://huggingface.co/sergiopaniego) - X (https://x.com/sergiopaniego) - LinkedIn (https://www.linkedin.com/in/sergio-paniego-blanco)
Llama-Index (./llama-index/introduction)	End-to-end tooling to ship a context-augmented AI agent to production	David Berenstein - HF (https://huggingface.co/davidberenstein1957) - X (https://x.com/davidberenstein) - LinkedIn (https://www.linkedin.com/in/davidberenstein)
LangGraph (./langgraph/introduction)	Agents allowing stateful orchestration of agents	Joffrey THOMAS - HF (https://huggingface.co/Joffthomas) - X (https://x.com/Jthmas404) - LinkedIn (https://www.linkedin.com/in/joffrey-thomas)

Unit 2.1 The smolagents framework

Instructional sections below follow the official English course order.

Introduction to smolagents

Source file: [units/en/unit2/smolagents/introduction.mdx](https://github.com/huggingface/smolagents/blob/main/units/en/unit2/smolagents/introduction.mdx)

Introduction to smolagents

[Image omitted from text-first adaptation: Unit 2.1 Thumbnail]

Welcome to this module, where you'll learn how to build effective agents using the smolagents (<https://github.com/huggingface/smolagents>) library, which provides a lightweight framework for creating capable AI agents.

smolagents is a Hugging Face library; therefore, we would appreciate your support by starring the smolagents repository (<https://github.com/huggingface/smolagents>) :

[Image omitted from text-first adaptation: starring smolagents]

Module Overview

This module provides a comprehensive overview of key concepts and practical strategies for building intelligent agents using smolagents.

With so many open-source frameworks available, it's essential to understand the components and capabilities that make smolagents a useful option or to determine when another solution might be a better fit.

We'll explore critical agent types, including code agents designed for software development tasks, tool calling agents for creating modular, function-driven workflows, and retrieval agents that access and synthesize information.

Additionally, we'll cover the orchestration of multiple agents as well as the integration of vision capabilities and web browsing, which unlock new possibilities for dynamic and context-aware applications.

In this unit, Alfred, the agent from Unit 1, makes his return. This time, he's using the smolagents framework for his internal workings. Together, we'll explore the key concepts behind this framework as Alfred tackles various tasks. Alfred is organizing a party at the Wayne Manor while the Wayne family is away, and he has plenty to do. Join us as we showcase his journey and how he handles these tasks with smolagents!

In this unit, you will learn to build AI agents with the smolagents library. Your agents will be able to search for data, execute code, and interact with web pages. You will also learn how to combine multiple agents to create more powerful systems.

[Image omitted from text-first adaptation: Alfred the agent]

Contents

During this unit on smolagents, we cover:

1 Why Use smolagents (./why_use_smolagents)

smolagents is one of the many open-source agent frameworks available for application development. Alternative options include LlamaIndex and LangGraph, which are also covered in other modules in this course. smolagents offers several key features that might make it a great fit for specific use cases, but we should always consider all options when selecting a framework. We'll explore the advantages and drawbacks of using smolagents, helping you make an informed decision based on your project's requirements.

2 CodeAgents (./code_agents)

CodeAgents are the primary type of agent in smolagents. Instead of generating JSON or text, these agents produce Python code to perform actions. This module explores their purpose, functionality, and how they work, along with hands-on examples to showcase their capabilities.

3 ToolCallingAgents (./tool_calling_agents)

ToolCallingAgents are the second type of agent supported by smolagents. Unlike CodeAgents, which generate Python code, these agents rely on JSON/text blobs that the system must parse and interpret to execute actions. This module covers their functionality, their key differences from CodeAgents, and it provides an example to illustrate their usage.

4 Tools (./tools)

As we saw in Unit 1, tools are functions that an LLM can use within an agentic system, and they act as the essential building blocks for agent behavior. This module covers how to create tools, their structure, and different implementation methods using the Tool class or the @tool decorator. You'll also learn about the default toolbox, how to share tools with the community, and how to load community-contributed tools for use in your agents.

5 Retrieval Agents (./retrieval_agents)

Retrieval agents allow models access to knowledge bases, making it possible to search, synthesize, and retrieve information from multiple sources. They leverage vector stores for efficient retrieval and implement Retrieval-Augmented Generation (RAG) patterns. These agents are particularly useful for integrating web search with custom knowledge bases while maintaining conversation context through memory systems. This module explores implementation strategies, including fallback mechanisms for robust information retrieval.

6 Multi-Agent Systems (./multi_agent_systems)

Orchestrating multiple agents effectively is crucial for building powerful, multi-agent systems. By combining agents with different capabilities-such as a web search agent with a code execution agent-you can create more sophisticated solutions. This module focuses on designing, implementing, and managing multi-agent systems to maximize efficiency and reliability.

7 Vision and Browser agents (./vision_agents)

Vision agents extend traditional agent capabilities by incorporating Vision-Language Models (VLMs), enabling them to process and interpret visual information. This module explores how to design and integrate VLM-powered agents, unlocking advanced functionalities like image-based reasoning, visual data analysis, and multimodal interactions. We will also use vision agents to build a browser agent that can browse the web and extract information from it.

Resources

- smolagents Documentation (<https://huggingface.co/docs/smolagents>) - Official docs for the smolagents library
- Building Effective Agents (<https://www.anthropic.com/research/building-effective-agents>) - Research paper on agent architectures
- Agent Guidelines (https://huggingface.co/docs/smolagents/tutorials/building_good_agents) - Best practices for building reliable agents
- LangGraph Agents (<https://langchain-ai.github.io/langgraph/>) - Additional examples of agent implementations
- Function Calling Guide (<https://platform.openai.com/docs/guides/function-calling>) - Understanding function calling in LLMs
- RAG Best Practices (<https://www.pinecone.io/learn/retrieval-augmented-generation/>) - Guide to implementing effective RAG

Why use smolagents?

Source file: [units/en/unit2/smolagents/why_use_smolagents.mdx](#)

[Image omitted from text-first adaptation: smolagents banner]

Why use smolagents

In this module, we will explore the pros and cons of using smolagents (<https://huggingface.co/docs/smolagents/en/index>), helping you make an informed decision about whether it's the right framework for your needs.

What is smolagents?

smolagents is a simple yet powerful framework for building AI agents. It provides LLMs with the agency to interact with the real world, such as searching or generating images.

As we learned in unit 1, AI agents are programs that use LLMs to generate 'thoughts' based on 'observations' to perform 'actions'. Let's explore how this is implemented in smolagents.

Key Advantages of smolagents

- Simplicity: Minimal code complexity and abstractions, to make the framework easy to understand, adopt and extend
- Flexible LLM Support: Works with any LLM through integration with Hugging Face tools and external APIs
- Code-First Approach: First-class support for Code Agents that write their actions directly in code, removing the need for parsing and simplifying tool calling
- HF Hub Integration: Seamless integration with the Hugging Face Hub, allowing the use of Gradio Spaces as tools

When to use smolagents?

With these advantages in mind, when should we use smolagents over other frameworks?

smolagents is ideal when:

- You need a lightweight and minimal solution.
- You want to experiment quickly without complex configurations.
- Your application logic is straightforward.

Code vs. JSON Actions

Unlike other frameworks where agents write actions in JSON, smolagents focuses on tool calls in code, simplifying the execution process. This is because there's no need to parse the JSON in order to build code that calls the tools: the output can be executed directly.

The following diagram illustrates this difference:

[Image omitted from text-first adaptation: Code vs. JSON actions]

To review the difference between Code vs JSON Actions, you can revisit the Actions Section in Unit 1 (<https://huggingface.co/learn/agents-course/unit1/actions#actions-enabling-the-agent-to-engage-with-its-environment>).

Agent Types in smolagents

Agents in smolagents operate as multi-step agents.

Each MultiStepAgent (<https://huggingface.co/docs/smolagents/main/en/reference/agents#smolagents.MultiStepAgent>) performs:

- One thought
- One tool call and execution

In addition to using CodeAgent (<https://huggingface.co/docs/smolagents/main/en/reference/agents#smolagents.CodeAgent>) as the primary type of agent, smolagents also supports ToolCallingAgent (<https://huggingface.co/docs/smolagents/main/en/reference/agents#smolagents.ToolCallingAgent>), which writes tool calls in JSON.

We will explore each agent type in more detail in the following sections.

In smolagents, tools are defined using @tool decorator wrapping a Python function or the Tool class.

Model Integration in smolagents

smolagents supports flexible LLM integration, allowing you to use any callable model that meets certain criteria (<https://huggingface.co/docs/smolagents/main/en/reference/models>). The framework provides several predefined classes to simplify model connections:

- `TransformersModel` (<https://huggingface.co/docs/smolagents/main/en/reference/models#smolagents.TransformersModel>): Implements a local transformers pipeline for seamless integration.
- `InferenceClientModel` (<https://huggingface.co/docs/smolagents/main/en/reference/models#smolagents.InferenceClientModel>): Supports serverless inference (https://huggingface.co/docs/huggingface_hub/main/en/guides/inference) calls through Hugging Face's infrastructure (<https://huggingface.co/docs/api-inference/index>), or via a growing number of third-party inference providers (https://huggingface.co/docs/huggingface_hub/main/en/guides/inference#supported-providers-and-tasks).
- `LiteLLMModel` (<https://huggingface.co/docs/smolagents/main/en/reference/models#smolagents.LiteLLMModel>): Leverages LiteLLM (<https://www.litellm.ai/>) for lightweight model interactions.
- `OpenAIServerModel` (<https://huggingface.co/docs/smolagents/main/en/reference/models#smolagents.OpenAIServerModel>): Connects to any service that offers an OpenAI API interface.
- `AzureOpenAIServerModel` (<https://huggingface.co/docs/smolagents/main/en/reference/models#smolagents.AzureOpenAIServerModel>): Supports integration with any Azure OpenAI deployment.

This flexibility ensures that developers can choose the model and service most suitable for their specific use cases, and allows for easy experimentation.

Now that we understood why and when to use smolagents, let's dive deeper into this powerful library!

Resources

- smolagents Blog (<https://huggingface.co/blog/smolagents>) - Introduction to smolagents and code interactions

Quick Quiz 1

Source file: [units/en/unit2/smolagents/quiz1.mdx](#)

Small Quiz (ungraded)

Let's test your understanding of smolagents with a quick quiz! Remember, testing yourself helps reinforce learning and identify areas that may need review.

This is an optional quiz and it's not graded.

Q1: What is one of the primary advantages of choosing smolagents over other frameworks?

Which statement best captures a core strength of the smolagents approach?

- Answer choice: It uses highly specialized configuration files and a steep learning curve to ensure only expert developers can use it

Explanation: smolagents is designed for simplicity and minimal code complexity, not steep learning curves.

},

- Answer choice: It supports a code-first approach with minimal abstractions, letting agents interact directly via Python function calls

Explanation: Yes, smolagents emphasizes a straightforward, code-centric design with minimal abstractions.

Correct answer.

},

- Answer choice: It focuses on JSON-based actions, removing the need for agents to write any code

Explanation: While smolagents supports JSON-based tool calls (`ToolCallingAgents`), the library emphasizes code-based approaches with `CodeAgents`.

},

- Answer choice: It deeply integrates with a single LLM provider and specialized hardware

Explanation: smolagents supports multiple model providers and does not require specialized hardware.

Q2: In which scenario would you likely benefit most from using smolagents?

Which situation aligns well with what smolagents does best?

- Answer choice: Prototyping or experimenting quickly with agent logic, particularly when your application is relatively straightforward

Explanation: Yes. smolagents is designed for simple and nimble agent creation without extensive setup overhead.

Correct answer.

},

- Answer choice: Building a large-scale enterprise system where you need dozens of microservices and real-time data pipelines

Explanation: While possible, smolagents is more focused on lightweight, code-centric experimentation rather than heavy enterprise infrastructure.

},

- Answer choice: Needing a framework that only supports cloud-based LLMs and forbids local inference

Explanation: smolagents offers flexible integration with local or hosted models, not exclusively cloud-based LLMs.

},

- Answer choice: A scenario that requires advanced orchestration, multi-modal perception, and enterprise-scale features out-of-the-box

Explanation: While you can integrate advanced capabilities, smolagents itself is lightweight and minimal at its core.

Q3: smolagents offers flexibility in model integration. Which statement best reflects its approach?

Choose the most accurate description of how smolagents interoperates with LLMs.

- Answer choice: It only provides a single built-in model and does not allow custom integrations

Explanation: smolagents supports multiple different backends and user-defined models.

},

- Answer choice: It requires you to implement your own model connector for every LLM usage

Explanation: There are multiple prebuilt connectors that make LLM integration straightforward.

},

- Answer choice: It only integrates with open-source LLMs but not commercial APIs

Explanation: smolagents can integrate with both open-source and commercial model APIs.

},

- Answer choice: It can be used with a wide range of LLMs, offering predefined classes like TransformersModel, InferenceClientModel, and LiteLLMModel

Explanation: This is correct. smolagents supports flexible model integration through various classes.

Correct answer.

Q4: How does smolagents handle the debate between code-based actions and JSON-based actions?

Which statement correctly characterizes smolagents' philosophy about action formats?

- Answer choice: It only allows JSON-based actions for all agent tasks, requiring a parser to extract the tool calls

Explanation: ToolCallingAgent uses JSON-based calls, but smolagents also provides a primary CodeAgent option that writes Python code.

},

- Answer choice: It focuses on code-based actions via a CodeAgent but also supports JSON-based tool calls with a ToolCallingAgent

Explanation: Yes, smolagents primarily recommends code-based actions but includes a JSON-based alternative for users who prefer it or need it.

Correct answer.

},

- Answer choice: It disallows any external function calls, instead requiring all logic to reside entirely within the LLM

Explanation: smolagents is specifically designed to grant LLMs the ability to call tools or code externally.

},

- Answer choice: It requires users to manually convert every code snippet into a JSON object before running the agent

Explanation: smolagents can automatically manage code snippet creation within the CodeAgent path, no manual JSON conversion necessary.

Q5: How does smolagents integrate with the Hugging Face Hub for added benefits?

Which statement accurately describes one of the core advantages of Hub integration?

- Answer choice: It automatically upgrades all public models to commercial license tiers

Explanation: Hub integration doesn't change the license tier for models or tools.

},

- Answer choice: It disables local inference entirely, forcing remote model usage only

Explanation: Users can still do local inference if they prefer; pushing to the Hub doesn't override local usage.

},

- Answer choice: It allows you to push and share agents or tools, making them easily discoverable and reusable by other developers

Explanation: smolagents supports uploading agents and tools to the HF Hub for others to reuse.

Correct answer.

},

- Answer choice: It permanently stores all your code-based agents, preventing any updates or versioning

Explanation: Hub repositories support updates and version control, so you can revise your code-based agents any time.

Congratulations on completing this quiz! If you missed any questions, consider reviewing the Why use smolagents section for a deeper understanding. If you did well, you're ready to explore more advanced topics in smolagents!

Building Agents That Use Code

Source file: [units/en/unit2/smolagents/code_agents.mdx](https://huggingface.co/docs/agents/en/unit2/smolagents/code_agents.mdx)

Building Agents That Use Code

Code agents are the default agent type in smolagents. They generate Python tool calls to perform actions, achieving action representations that are efficient, expressive, and accurate.

Their streamlined approach reduces the number of required actions, simplifies complex operations, and enables reuse of existing code functions. smolagents provides a lightweight framework for building code agents, implemented in approximately 1,000 lines of code.

[Image omitted from text-first adaptation: Code vs JSON Actions]

Graphic from the paper Executable Code Actions Elicit Better LLM Agents (<https://huggingface.co/papers/2402.01030>)

If you want to learn more about why code agents are effective, check out this guide from the smolagents documentation.

Why Code Agents?

In a multi-step agent process, the LLM writes and executes actions, typically involving external tool calls. Traditional approaches use a JSON format to specify tool names and arguments as strings, which the system must parse to determine which tool to execute.

However, research shows that tool-calling LLMs work more effectively with code directly. This is a core principle of smolagents, as shown in the diagram above from Executable Code Actions Elicit Better LLM Agents (<https://huggingface.co/papers/2402.01030>).

Writing actions in code rather than JSON offers several key advantages:

- Composability: Easily combine and reuse actions
- Object Management: Work directly with complex structures like images
- Generality: Express any computationally possible task
- Natural for LLMs: High-quality code is already present in LLM training data

How Does a Code Agent Work?

[Image omitted from text-first adaptation: From https://huggingface.co/docs/smolagents/conceptual_guides/react]

The diagram above illustrates how `CodeAgent.run()` operates, following the ReAct framework we mentioned in Unit 1. The main abstraction for agents in smolagents is a `MultiStepAgent`, which serves as the core building block. `CodeAgent` is a special kind of `MultiStepAgent`, as we will see in an example below.

A `CodeAgent` performs actions through a cycle of steps, with existing variables and knowledge being incorporated into the agent's context, which is kept in an execution log:

1. The system prompt is stored in a `SystemPromptStep`, and the user query is logged in a `TaskStep`.
1. Then, the following while loop is executed:
 - 2.1 Method `agent.write_memory_to_messages()` writes the agent's logs into a list of LLM-readable chat messages (https://huggingface.co/docs/transformers/main/en/chat_templating).
 - 2.2 These messages are sent to a `Model`, which generates a completion.
 - 2.3 The completion is parsed to extract the action, which, in our case, should be a code snippet since we're working with a `CodeAgent`.
 - 2.4 The action is executed.
 - 2.5 The results are logged into memory in an `ActionStep`.

At the end of each step, if the agent includes any function calls (in `agent.step_callback`), they are executed.

Let's See Some Examples

You can follow the code in this notebook that you can run using Google Colab.

Alfred is planning a party at the Wayne family mansion and needs your help to ensure everything goes smoothly. To assist him, we'll apply what we've learned about how a multi-step `CodeAgent` operates.

[Image omitted from text-first adaptation: Alfred Party]

If you haven't installed smolagents yet, you can do so by running the following command:

```
pip install smolagents -U
```

Let's also login to the Hugging Face Hub to have access to the Serverless Inference API.

```
from huggingface_hub import login

login()
```

Selecting a Playlist for the Party Using smolagents

Music is an essential part of a successful party! Alfred needs some help selecting the playlist. Luckily, smolagents has got us covered! We can build an agent capable of searching the web using DuckDuckGo. To give the agent access to this tool, we include it in the tool list when creating the agent.

[Image omitted from text-first adaptation: Alfred Playlist]

For the model, we'll rely on `InferenceClientModel`, which provides access to Hugging Face's Serverless Inference API (<https://huggingface.co/docs/api-inference/index>). The default model is "Qwen/Qwen2.5-Coder-32B-Instruct", which is performant and available for fast inference, but you can select any compatible model from the Hub.

Running an agent is quite straightforward:

```
from smolagents import CodeAgent, DuckDuckGoSearchTool, InferenceClientModel
agent = CodeAgent(tools=[DuckDuckGoSearchTool()], model=InferenceClientModel())
agent.run("Search for the best music recommendations for a party at the Wayne's mansion.")
```

When you run this example, the output will display a trace of the workflow steps being executed. It will also print the corresponding Python code with the message:

[illegible]

After a few steps, you'll see the generated playlist that Alfred can use for the party!

Using a Custom Tool to Prepare the Menu

[Image omitted from text-first adaptation: Alfred Menu]

Now that we have selected a playlist, we need to organize the menu for the guests. Again, Alfred can take advantage of smolagents to do so. Here, we use the `@tool` decorator to define a custom function that acts as a tool. We'll cover tool creation in more detail later, so for now, we can simply run the code.

As you can see in the example below, we will create a tool using the `@tool` decorator and include it in the tools list.

```
from smolagents import CodeAgent, tool, InferenceClientModel

# Tool to suggest a menu based on the occasion
@tool
def suggest_menu(occasion: str) -> str:
    """
    Suggests a menu based on the occasion.

    Args:
        occasion (str): The type of occasion for the party. Allowed values are:
            - "casual": Menu for casual party.
            - "formal": Menu for formal party.
            - "superhero": Menu for superhero party.
            - "custom": Custom menu.

    """
    if occasion == "casual":
        return "Pizza, snacks, and drinks."
    elif occasion == "formal":
        return "3-course dinner with wine and dessert."
    elif occasion == "superhero":
        return "Buffet with high-energy and healthy food."
    else:
        return "Custom menu for the butler."

# Alfred, the butler, preparing the menu for the party
agent = CodeAgent(tools=[suggest_menu], model=InferenceClientModel())

# Preparing the menu for the party
agent.run("Prepare a formal menu for the party.")
```

The agent will run for a few steps until finding the answer. Precising allowed values in the docstring helps direct agent to occasional argument values which exist and limit hallucinations.

The menu is ready!

Using Python Imports Inside the Agent

We have the playlist and menu ready, but we need to check one more crucial detail: preparation time!

Alfred needs to calculate when everything would be ready if he started preparing now, in case they need assistance from other superheroes.

smolagents specializes in agents that write and execute Python code snippets, offering sandboxed execution for security.

Code execution has strict security measures - imports outside a predefined safe list are blocked by default. However, you can authorize additional imports by passing them as strings in `additional_authorized_imports`. For more details on secure code execution, see the official guide (https://huggingface.co/docs/smolagents/tutorials/secure_code_execution).

When creating the agent, we'll use `additional_authorized_imports` to allow for importing the `datetime` module.

```
from smolagents import CodeAgent, InferenceClientModel
import numpy as np
import time
import datetime

agent = CodeAgent(tools=[], model=InferenceClientModel(), additional_authorized_imports=['datetime'])

agent.run(
    """
    Alfred needs to prepare for the party. Here are the tasks:
    1. Prepare the drinks - 30 minutes
    2. Decorate the mansion - 60 minutes
    3. Set up the menu - 45 minutes
    4. Prepare the music and playlist - 45 minutes

    If we start right now, at what time will the party be ready?
    """
)
```

These examples are just the beginning of what you can do with code agents, and we're already starting to see their utility for preparing the party. You can learn more about how to build code agents in the smolagents documentation (<https://huggingface.co/docs/smolagents>).

In summary, smolagents specializes in agents that write and execute Python code snippets, offering sandboxed execution for security. It supports both local and API-based language models, making it adaptable to various development environments.

Sharing Our Custom Party Preparator Agent to the Hub

Wouldn't it be amazing to share our very own Alfred agent with the community? By doing so, anyone can easily download and use the agent directly from the Hub, bringing the ultimate party planner of Gotham to their fingertips! Let's make it happen!

The smolagents library makes this possible by allowing you to share a complete agent with the community and download others for immediate use. It's as simple as the following:

```
# Change to your username and repo name
agent.push_to_hub('sergiopaniago/AlfredAgent')
```

To download the agent again, use the code below:

```
# Change to your username and repo name
alfred_agent = agent.from_hub('sergiopaniago/AlfredAgent', trust_remote_code=True)

alfred_agent.run("Give me the best playlist for a party at Wayne's mansion. The party idea is a 'villain \
masquerade' theme")
```

What's also exciting is that shared agents are directly available as Hugging Face Spaces, allowing you to interact with them in real-time. You can explore other agents here (<https://huggingface.co/spaces/davidberenstein1957/smolagents-and-tools>).

For example, the AlfredAgent is available here (<https://huggingface.co/spaces/sergiopaniago/AlfredAgent>). You can try it out directly below:

[Embedded media omitted from text-first adaptation: interactive media]

You may be wondering-how did Alfred build such an agent using smolagents? By integrating several tools, he can generate an agent as follows. Don't worry about the tools for now, as we'll have a dedicated section later in this unit to explore that in detail:

```
from smolagents import CodeAgent, DuckDuckGoSearchTool, FinalAnswerTool, InferenceClientModel, Tool, tool \
, VisitWebpageTool
```

```

@tool
def suggest_menu(occasion: str) -> str:
    """
    Suggests a menu based on the occasion.
    Args:
        occasion: The type of occasion for the party.
    """
    if occasion == "casual":
        return "Pizza, snacks, and drinks."
    elif occasion == "formal":
        return "3-course dinner with wine and dessert."
    elif occasion == "superhero":
        return "Buffet with high-energy and healthy food."
    else:
        return "Custom menu for the butler."

@tool
def catering_service_tool(query: str) -> str:
    """
    This tool returns the highest-rated catering service in Gotham City.

    Args:
        query: A search term for finding catering services.
    """
    # Example list of catering services and their ratings
    services = {
        "Gotham Catering Co.": 4.9,
        "Wayne Manor Catering": 4.8,
        "Gotham City Events": 4.7,
    }

    # Find the highest rated catering service (simulating search query filtering)
    best_service = max(services, key=services.get)

    return best_service

class SuperheroPartyThemeTool(Tool):
    name = "superhero_party_theme_generator"
    description = """
    This tool suggests creative superhero-themed party ideas based on a category.
    It returns a unique party theme idea."""

    inputs = {
        "category": {
            "type": "string",
            "description": "The type of superhero party (e.g., 'classic heroes', 'villain masquerade', 'futuristic Gotham').",
        }
    }

    output_type = "string"

    def forward(self, category: str):
        themes = {
            "classic heroes": "Justice League Gala: Guests come dressed as their favorite DC heroes with themed cocktails like 'The Kryptonite Punch'.",
            "villain masquerade": "Gotham Rogues' Ball: A mysterious masquerade where guests dress as classic Batman villains.",
            "futuristic gotham": "Neo-Gotham Night: A cyberpunk-style party inspired by Batman Beyond, with neon decorations and futuristic gadgets."
        }

        return themes.get(category.lower(), "Themed party idea not found. Try 'classic heroes', 'villain masquerade', or 'futuristic Gotham'.")

# Alfred, the butler, preparing the menu for the party
agent = CodeAgent(
    tools=[
        DuckDuckGoSearchTool(),
        VisitWebpageTool(),
        suggest_menu,
        catering_service_tool,
        SuperheroPartyThemeTool(),
        FinalAnswerTool()
    ],
    model=InferenceClientModel(),
    max_steps=10,
    verbosity_level=2
)

```

```
agent.run("Give me the best playlist for a party at the Wayne's mansion. The party idea is a 'villain mas \
querade' theme")
```

As you can see, we've created a CodeAgent with several tools that enhance the agent's functionality, turning it into the ultimate party planner ready to share with the community!

Now, it's your turn: build your very own agent and share it with the community using the knowledge we've just learned!

If you would like to share your agent project, then make a space and tag the agents-course on the Hugging Face Hub. We'd love to see what you've created!

Inspecting Our Party Preparator Agent with OpenTelemetry and Langfuse

As Alfred fine-tunes the Party Preparator Agent, he's growing weary of debugging its runs. Agents, by nature, are unpredictable and difficult to inspect. But since he aims to build the ultimate Party Preparator Agent and deploy it in production, he needs robust traceability for future monitoring and analysis.

Once again, smolagents comes to the rescue! It embraces the OpenTelemetry (<https://opentelemetry.io/>) standard for instrumenting agent runs, allowing seamless inspection and logging. With the help of Langfuse (<https://langfuse.com/>) and the SmolagentsInstrumentor, Alfred can easily track and analyze his agent's behavior.

Setting it up is straightforward!

First, we need to install the necessary dependencies:

```
pip install opentelemetry-sdk opentelemetry-exporter-otlp openinference-instrumentation-smolagents langfu \
se
```

Next, Alfred has already created an account on Langfuse and has his API keys ready. If you haven't done so yet, you can sign up for Langfuse Cloud here (<https://cloud.langfuse.com/>) or explore alternatives (https://huggingface.co/docs/smolagents/tutorials/inspect_runs).

Once you have your API keys, they need to be properly configured as follows:

```
import os

# Get keys for your project from the project settings page: https://cloud.langfuse.com
os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
os.environ["LANGFUSE_HOST"] = "https://cloud.langfuse.com" # EU region
# os.environ["LANGFUSE_HOST"] = "https://us.cloud.langfuse.com" # US region
```

With the environment variables set, we can now initialize the Langfuse client. `get_client()` initializes the Langfuse client using the credentials provided in the environment variables.

```
from langfuse import get_client

langfuse = get_client()

# Verify connection
if langfuse.auth_check():
    print("Langfuse client is authenticated and ready!")
else:
    print("Authentication failed. Please check your credentials and host.")
```

Finally, Alfred is ready to initialize the SmolagentsInstrumentor and start tracking his agent's performance.

```
from openinference.instrumentation.smolagents import SmolagentsInstrumentor

SmolagentsInstrumentor().instrument()
```

Alfred is now connected ! The runs from smolagents are being logged in Langfuse, giving him full visibility into the agent's behavior. With this setup, he's ready to revisit previous runs and refine his Party Preparator Agent even further.

To learn more about tracing your agents and using the collected data to evaluate their performance, check out Bonus Unit 2.

```
from smolagents import CodeAgent, InferenceClientModel

agent = CodeAgent(tools=[], model=InferenceClientModel())
alfred_agent = agent.from_hub('sergiopaniago/AlfredAgent', trust_remote_code=True)
alfred_agent.run("Give me the best playlist for a party at Wayne's mansion. The party idea is a 'villain \
masquerade' theme")
```

Alfred can now access these logs here (<https://cloud.langfuse.com/project/cm7bq0abj025rad078ak3luwi/traces/995fc019255528e4f48cf6770b0ce27b?timestamp=2025-02-19T10%3A28%3A36.929Z>) to review and analyze them.

Actually, a minor error occurred during execution. Can you spot it in the logs? Try to track how the agent handles it and still returns a valid answer. Here is the direct link to the error if you want to verify your answer. Of course the error has been fixed in the meantime, more details can be found in this issue.

Meanwhile, the suggested playlist (<https://open.spotify.com/playlist/0gZMMHjuxMrrybQ7wTMTpw>) sets the perfect vibe for the party preparations. Cool, right?

Now that we have created our first Code Agent, let's learn how we can create Tool Calling Agents, the second type of agent available in smolagents.

Resources

- smolagents Blog (<https://huggingface.co/blog/smolagents>) - Introduction to smolagents and code interactions
- smolagents: Building Good Agents (https://huggingface.co/docs/smolagents/tutorials/building_good_agents) - Best practices for reliable agents
- Building Effective Agents - Anthropic (<https://www.anthropic.com/research/building-effective-agents>) - Agent design principles
- Sharing runs with OpenTelemetry (https://huggingface.co/docs/smolagents/tutorials/inspect_runs) - Details about how to setup OpenTelemetry for tracking your agents.

Writing actions as code snippets or JSON blobs

Source file: `units/en/unit2/smolagents/tool_calling_agents.mdx`

Writing actions as code snippets or JSON blobs

You can follow the code in this notebook that you can run using Google Colab.

Tool Calling Agents are the second type of agent available in smolagents. Unlike Code Agents that use Python snippets, these agents use the built-in tool-calling capabilities of LLM providers to generate tool calls as JSON structures. This is the standard approach used by OpenAI, Anthropic, and many other providers.

Let's look at an example. When Alfred wants to search for catering services and party ideas, a CodeAgent would generate and run Python code like this:

```
for query in [
    "Best catering services in Gotham City",
    "Party theme ideas for superheroes"
]:
    print(web_search(f"Search for: {query}"))
```

A ToolCallingAgent would instead create a JSON structure:

```
[
  {"name": "web_search", "arguments": "Best catering services in Gotham City"},
  {"name": "web_search", "arguments": "Party theme ideas for superheroes"}
]
```

This JSON blob is then used to execute the tool calls.

While smolagents primarily focuses on CodeAgents since they perform better overall (<https://huggingface.co/papers/2402.01030>), ToolCallingAgents can be effective for simple systems that don't require variable handling or complex tool calls.

[Image omitted from text-first adaptation: Code vs JSON Actions]

How Do Tool Calling Agents Work?

Tool Calling Agents follow the same multi-step workflow as Code Agents (see the previous section (`./code_agents`) for details).

The key difference is in how they structure their actions: instead of executable code, they generate JSON objects that specify tool names and arguments. The system then parses these instructions to execute the appropriate tools.

[Image omitted from text-first adaptation: Agentic pipeline from https://huggingface.co/docs/smolagents/conceptual_guides/react]

Tool Creation Methods

In smolagents, tools can be defined in two ways:

1. Using the `@tool` decorator for simple function-based tools
1. Creating a subclass of `Tool` for more complex functionality

The `@tool` Decorator

The `@tool` decorator is the recommended way to define simple tools. Under the hood, smolagents will parse basic information about the function from Python. So if you name your function clearly and write a good docstring, it will be easier for the LLM to use.

Using this approach, we define a function with:

- A clear and descriptive function name that helps the LLM understand its purpose.
- Type hints for both inputs and outputs to ensure proper usage.
- A detailed description, including an `Args:` section where each argument is explicitly described. These descriptions provide valuable context for the LLM, so it's important to write them carefully.

Generating a tool that retrieves the highest-rated catering

[Image omitted from text-first adaptation: Alfred Catering]

You can follow the code in this notebook that you can run using Google Colab.

Let's imagine that Alfred has already decided on the menu for the party, but now he needs help preparing food for such a large number of guests. To do so, he would like to hire a catering service and needs to identify the highest-rated options available. Alfred can leverage a tool to search for the best catering services in his area.

Below is an example of how Alfred can use the `@tool` decorator to make this happen:

```
from smolagents import CodeAgent, InferenceClientModel, tool

# Let's pretend we have a function that fetches the highest-rated catering services.
@tool
def catering_service_tool(query: str) -> str:
    """
    This tool returns the highest-rated catering service in Gotham City.

    Args:
        query: A search term for finding catering services.
    """
    # Example list of catering services and their ratings
    services = {
        "Gotham Catering Co.": 4.9,
        "Wayne Manor Catering": 4.8,
        "Gotham City Events": 4.7,
    }

    # Find the highest rated catering service (simulating search query filtering)
    best_service = max(services, key=services.get)

    return best_service

agent = CodeAgent(tools=[catering_service_tool], model=InferenceClientModel())

# Run the agent to find the best catering service
result = agent.run(
    "Can you give me the name of the highest-rated catering service in Gotham City?"
)

print(result) # Output: Gotham Catering Co.
```

Defining a Tool as a Python Class

This approach involves creating a subclass of Tool

(<https://huggingface.co/docs/smolagents/v1.8.1/en/reference/tools#smolagents.Tool>). For complex tools, we can implement a class instead of a Python function. The class wraps the function with metadata that helps the LLM understand how to use it effectively. In this class, we define:

- name: The tool's name.
- description: A description used to populate the agent's system prompt.
- inputs: A dictionary with keys type and description, providing information to help the Python interpreter process inputs.
- output_type: Specifies the expected output type.
- forward: The method containing the inference logic to execute.

Below, we can see an example of a tool built using Tool and how to integrate it within a CodeAgent.

Generating a tool to generate ideas about the superhero-themed party

Alfred's party at the mansion is a superhero-themed event, but he needs some creative ideas to make it truly special. As a fantastic host, he wants to surprise the guests with a unique theme.

To do this, he can use an agent that generates superhero-themed party ideas based on a given category. This way, Alfred can find the perfect party theme to wow his guests.

```
from smolagents import Tool, CodeAgent, InferenceClientModel

class SuperheroPartyThemeTool(Tool):
    name = "superhero_party_theme_generator"
    description = """
    This tool suggests creative superhero-themed party ideas based on a category.
    It returns a unique party theme idea."""

    inputs = {
        "category": {
            "type": "string",
            "description": "The type of superhero party (e.g., 'classic heroes', 'villain masquerade', 'futuristic Gotham').",
        }
    }

    output_type = "string"

    def forward(self, category: str):
        themes = {
            "classic heroes": "Justice League Gala: Guests come dressed as their favorite DC heroes with \
themed cocktails like 'The Kryptonite Punch'.",
            "villain masquerade": "Gotham Rogues' Ball: A mysterious masquerade where guests dress as cla \
ssic Batman villains.",
            "futuristic Gotham": "Neo-Gotham Night: A cyberpunk-style party inspired by Batman Beyond, wi \
th neon decorations and futuristic gadgets."
        }

        return themes.get(category.lower(), "Themed party idea not found. Try 'classic heroes', 'villain \
masquerade', or 'futuristic Gotham'.")

# Instantiate the tool
party_theme_tool = SuperheroPartyThemeTool()
agent = CodeAgent(tools=[party_theme_tool], model=InferenceClientModel())

# Run the agent to generate a party theme idea
result = agent.run(
    "What would be a good superhero party idea for a 'villain masquerade' theme?"
)

print(result) # Output: "Gotham Rogues' Ball: A mysterious masquerade where guests dress as classic Batm \
an villains."
```

With this tool, Alfred will be the ultimate super host, impressing his guests with a superhero-themed party they won't forget!

Default Toolbox

smolagents comes with a set of pre-built tools that can be directly injected into your agent. The default toolbox (https://huggingface.co/docs/smolagents/guided_tour?build-a-tool=Decorate+a+function+with+%40tool#default-toolbox) includes:

- PythonInterpreterTool
- FinalAnswerTool
- UserInputTool
- DuckDuckGoSearchTool
- GoogleSearchTool
- VisitWebpageTool

Alfred could use various tools to ensure a flawless party at Wayne Manor:

- First, he could use the DuckDuckGoSearchTool to find creative superhero-themed party ideas.
- For catering, he'd rely on the GoogleSearchTool to find the highest-rated services in Gotham.
- To manage seating arrangements, Alfred could run calculations with the PythonInterpreterTool.
- Once everything is gathered, he'd compile the plan using the FinalAnswerTool.

With these tools, Alfred guarantees the party is both exceptional and seamless.

Sharing and Importing Tools

One of the most powerful features of smolagents is its ability to share custom tools on the Hub and seamlessly integrate tools created by the community. This includes connecting with HF Spaces and LangChain tools, significantly enhancing Alfred's ability to orchestrate an unforgettable party at Wayne Manor.

With these integrations, Alfred can tap into advanced event-planning tools-whether it's adjusting the lighting for the perfect ambiance, curating the ideal playlist for the party, or coordinating with Gotham's finest caterers.

Here are examples showcasing how these functionalities can elevate the party experience:

Sharing a Tool to the Hub

Sharing your custom tool with the community is easy! Simply upload it to your Hugging Face account using the `push_to_hub()` method.

For instance, Alfred can share his `party_theme_tool` to help others find the best catering services in Gotham. Here's how to do it:

```
party_theme_tool.push_to_hub("{your_username}/party_theme_tool", token="<YOUR_HUGGINGFACEHUB_API_TOKEN>")
```

Importing a Tool from the Hub

You can easily import tools created by other users using the `load_tool()` function. For example, Alfred might want to generate a promotional image for the party using AI. Instead of building a tool from scratch, he can leverage a predefined one from the community:

```
from smolagents import load_tool, CodeAgent, InferenceClientModel

image_generation_tool = load_tool(
    "m-ric/text-to-image",
    trust_remote_code=True
)

agent = CodeAgent(
    tools=[image_generation_tool],
    model=InferenceClientModel()
)

agent.run("Generate an image of a luxurious superhero-themed party at Wayne Manor with made-up superheroes \
.")
```


Importing a Hugging Face Space as a Tool

You can also import a HF Space as a tool using `Tool.from_space()`. This opens up possibilities for integrating with thousands of spaces from the community for tasks from image generation to data analysis.

The tool will connect with the spaces Gradio backend using the `gradio_client`, so make sure to install it via pip if you don't have it already.

For the party, Alfred can use an existing HF Space for the generation of the AI-generated image to be used in the announcement (instead of the pre-built tool we mentioned before). Let's build it!

```
from smolagents import CodeAgent, InferenceClientModel, Tool

image_generation_tool = Tool.from_space(
    "black-forest-labs/FLUX.1-schnell",
    name="image_generator",
    description="Generate an image from a prompt"
)

model = InferenceClientModel("Qwen/Qwen2.5-Coder-32B-Instruct")

agent = CodeAgent(tools=[image_generation_tool], model=model)

agent.run(
    "Improve this prompt, then generate an image of it.",
    additional_args={'user_prompt': 'A grand superhero-themed party at Wayne Manor, with Alfred overseein \
g a luxurious gala'}
)
```

Importing a LangChain Tool

We'll discuss the LangChain framework in upcoming sections. For now, we just note that we can reuse LangChain tools in your smolagents workflow!

You can easily load LangChain tools using the `Tool.from_langchain()` method. Alfred, ever the perfectionist, is preparing for a spectacular superhero night at Wayne Manor while the Waynes are away. To make sure every detail exceeds expectations, he taps into LangChain tools to find top-tier entertainment ideas.

By using `Tool.from_langchain()`, Alfred effortlessly adds advanced search functionalities to his smolagent, enabling him to discover exclusive party ideas and services with just a few commands.

Here's how he does it:

We first need to install the langchain integration for smolagents.

```
pip install -U langchain-community
```

After installing the langchain integration, make sure to set your SerpAPI key. This is required for search-based tools such as `SerpAPITool`:

```
os.environ['SERPAPI_API_KEY'] = '...'
```

You can create a SerpAPI key here (<https://serpapi.com/>)

```
from langchain.agents import load_tools
from smolagents import CodeAgent, InferenceClientModel, Tool

search_tool = Tool.from_langchain(load_tools(["serpapi"])[0])

agent = CodeAgent(tools=[search_tool], model=model)

agent.run("Search for luxury entertainment ideas for a superhero-themed event, such as live performances \
and interactive experiences.")
```

Importing a tool collection from any MCP server

smolagents also allows importing tools from the hundreds of MCP servers available on [glama.ai](https://glama.ai/mcp/servers) (<https://glama.ai/mcp/servers>) or smithery.ai (<https://smithery.ai>). If you want to dive deeper about MCP, you can check our free MCP Course (<https://huggingface.co/learn/mcp-course/>).

Install mcp client

We first need to install the mcp integration for smolagents.

```
pip install "smolagents[mcp]"
```

The MCP servers tools can be loaded in a ToolCollection object as follow:

```
import os
from smolagents import ToolCollection, CodeAgent
from mcp import StdioServerParameters
from smolagents import InferenceClientModel

model = InferenceClientModel("Qwen/Qwen2.5-Coder-32B-Instruct")

server_parameters = StdioServerParameters(
    command="uvx",
    args=["--quiet", "pubmedmcp@0.1.3"],
    env={"UV_PYTHON": "3.12", **os.environ},
)

with ToolCollection.from_mcp(server_parameters, trust_remote_code=True) as tool_collection:
    agent = CodeAgent(tools=[*tool_collection.tools], model=model, add_base_tools=True)
    agent.run("Please find a remedy for hangover.")
```

With this setup, Alfred can quickly discover luxurious entertainment options, ensuring Gotham's elite guests have an unforgettable experience. This tool helps him curate the perfect superhero-themed event for Wayne Manor!

Resources

- Tools Tutorial (<https://huggingface.co/docs/smolagents/tutorials/tools>) - Explore this tutorial to learn how to work with tools effectively.
- Tools Documentation (<https://huggingface.co/docs/smolagents/v1.8.1/en/reference/tools>) - Comprehensive reference documentation on tools.
- Tools Guided Tour (https://huggingface.co/docs/smolagents/v1.8.1/en/guided_tour#tools) - A step-by-step guided tour to help you build and utilize tools efficiently.
- Building Effective Agents (https://huggingface.co/docs/smolagents/tutorials/building_good_agents) - A detailed guide on best practices for developing reliable and high-performance custom function agents.

Retrieval Agents

Source file: [units/en/unit2/smolagents/retrieval_agents.mdx](#)

Building Agentic RAG Systems

You can follow the code in this notebook that you can run using Google Colab.

Retrieval Augmented Generation (RAG) systems combine the capabilities of data retrieval and generation models to provide context-aware responses. For example, a user's query is passed to a search engine, and the retrieved results are given to the model along with the query. The model then generates a response based on the query and retrieved information.

Agentic RAG (Retrieval-Augmented Generation) extends traditional RAG systems by combining autonomous agents with dynamic knowledge retrieval.

While traditional RAG systems use an LLM to answer queries based on retrieved data, agentic RAG enables intelligent control of both retrieval and generation processes, improving efficiency and accuracy.

Traditional RAG systems face key limitations, such as relying on a single retrieval step and focusing on direct semantic similarity with the user's query, which may overlook relevant information.

Agentic RAG addresses these issues by allowing the agent to autonomously formulate search queries, critique retrieved results, and conduct multiple retrieval steps for a more tailored and comprehensive output.

Basic Retrieval with DuckDuckGo

Let's build a simple agent that can search the web using DuckDuckGo. This agent will retrieve information and synthesize responses to answer queries. With Agentic RAG, Alfred's agent can:

- Search for latest superhero party trends
- Refine results to include luxury elements
- Synthesize information into a complete plan

Here's how Alfred's agent can achieve this:

```
from smolagents import CodeAgent, DuckDuckGoSearchTool, InferenceClientModel

# Initialize the search tool
search_tool = DuckDuckGoSearchTool()

# Initialize the model
model = InferenceClientModel()

agent = CodeAgent(
    model=model,
    tools=[search_tool],
)

# Example usage
response = agent.run(
    "Search for luxury superhero-themed party ideas, including decorations, entertainment, and catering."
)
print(response)
```

The agent follows this process:

1. Analyzes the Request: Alfred's agent identifies the key elements of the query-luxury superhero-themed party planning, with focus on decor, entertainment, and catering.
1. Performs Retrieval: The agent leverages DuckDuckGo to search for the most relevant and up-to-date information, ensuring it aligns with Alfred's refined preferences for a luxurious event.
1. Synthesizes Information: After gathering the results, the agent processes them into a cohesive, actionable plan for Alfred, covering all aspects of the party.
1. Stores for Future Reference: The agent stores the retrieved information for easy access when planning future events, optimizing efficiency in subsequent tasks.

Custom Knowledge Base Tool

For specialized tasks, a custom knowledge base can be invaluable. Let's create a tool that queries a vector database of technical documentation or specialized knowledge. Using semantic search, the agent can find the most relevant information for Alfred's needs.

A vector database stores numerical representations (embeddings) of text or other data, created by machine learning models. It enables semantic search by identifying similar meanings in high-dimensional space.

This approach combines predefined knowledge with semantic search to provide context-aware solutions for event planning. With specialized knowledge access, Alfred can perfect every detail of the party.

In this example, we'll create a tool that retrieves party planning ideas from a custom knowledge base. We'll use a BM25 retriever to search the knowledge base and return the top results, and RecursiveCharacterTextSplitter to split the documents into smaller chunks for more efficient search.

```
from langchain_community.docstore.document import Document
from langchain_text_splitters import RecursiveCharacterTextSplitter
from smolagents import Tool
from langchain_community.retrievers import BM25Retriever
from smolagents import CodeAgent, InferenceClientModel

class PartyPlanningRetrieverTool(Tool):
    name = "party_planning_retriever"
    description = "Uses semantic search to retrieve relevant party planning ideas for Alfred's superhero- \\"
```

```

themed party at Wayne Manor."
inputs = {
    "query": {
        "type": "string",
        "description": "The query to perform. This should be a query related to party planning or superhero themes.",
    }
}
output_type = "string"

def __init__(self, docs, **kwargs):
    super().__init__(**kwargs)
    self.retriever = BM25Retriever.from_documents(
        docs, k=5 # Retrieve the top 5 documents
    )

def forward(self, query: str) -> str:
    assert isinstance(query, str), "Your search query must be a string"

    docs = self.retriever.invoke(
        query,
    )
    return "\nRetrieved ideas:\n" + "".join(
        [
            f"\n\n==== Idea {str(i)} =====\n" + doc.page_content
            for i, doc in enumerate(docs)
        ]
    )

# Simulate a knowledge base about party planning
party_ideas = [
    {"text": "A superhero-themed masquerade ball with luxury decor, including gold accents and velvet curtains.", "source": "Party Ideas 1"},
    {"text": "Hire a professional DJ who can play themed music for superheroes like Batman and Wonder Woman.", "source": "Entertainment Ideas"},
    {"text": "For catering, serve dishes named after superheroes, like 'The Hulk's Green Smoothie' and 'Iron Man's Power Steak.'", "source": "Catering Ideas"},
    {"text": "Decorate with iconic superhero logos and projections of Gotham and other superhero cities around the venue.", "source": "Decoration Ideas"},
    {"text": "Interactive experiences with VR where guests can engage in superhero simulations or compete in themed games.", "source": "Entertainment Ideas"}
]

source_docs = [
    Document(page_content=doc["text"], metadata={"source": doc["source"]})
    for doc in party_ideas
]

# Split the documents into smaller chunks for more efficient search
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50,
    add_start_index=True,
    strip_whitespace=True,
    separators=["\n\n", "\n", ".", " ", "", ""],
)
docs_processed = text_splitter.split_documents(source_docs)

# Create the retriever tool
party_planning_retriever = PartyPlanningRetrieverTool(docs_processed)

# Initialize the agent
agent = CodeAgent(tools=[party_planning_retriever], model=InferenceClientModel())

# Example usage
response = agent.run(
    "Find ideas for a luxury superhero-themed party, including entertainment, catering, and decoration options."
)

print(response)

```

This enhanced agent can:

1. First check the documentation for relevant information
1. Combine insights from the knowledge base
1. Maintain conversation context in memory

Enhanced Retrieval Capabilities

When building agentic RAG systems, the agent can employ sophisticated strategies like:

1. Query Reformulation: Instead of using the raw user query, the agent can craft optimized search terms that better match the target documents
1. Query Decomposition: Instead of using the user query directly, if it contains multiple pieces of information to query, it can be decomposed to multiple queries
1. Query Expansion: Somehow similar to Query Reformulation but done multiple times to put the query in multiple wordings to query them all
1. Reranking: Using Cross-Encoders to assign more comprehensive and semantic relevance scores between retrieved documents and search query
1. Multi-Step Retrieval: The agent can perform multiple searches, using initial results to inform subsequent queries
1. Source Integration: Information can be combined from multiple sources like web search and local documentation
1. Result Validation: Retrieved content can be analyzed for relevance and accuracy before being included in responses

Effective agentic RAG systems require careful consideration of several key aspects. The agent should select between available tools based on the query type and context. Memory systems help maintain conversation history and avoid repetitive retrievals. Having fallback strategies ensures the system can still provide value even when primary retrieval methods fail. Additionally, implementing validation steps helps ensure the accuracy and relevance of retrieved information.

Resources

- Agentic RAG: turbocharge your RAG with query reformulation and self-query! (https://huggingface.co/learn/cookbook/agent_rag) - Recipe for developing an Agentic RAG system using smolagents.

Quick Quiz 2

Source file: [units/en/unit2/smolagents/quiz2.mdx](#)

Small Quiz (ungraded)

It's time to test your understanding of the Code Agents, Tool Calling Agents, and Tools sections. This quiz is optional and not graded.

Q1: What is the key difference between creating a tool with the `@tool` decorator versus creating a subclass of `Tool` in `smolagents`?

Which statement best describes the distinction between these two approaches for defining tools?

- Answer choice: Using the `@tool` decorator is mandatory for retrieval-based tools, while subclasses of `Tool` are only for text-generation tasks

Explanation: Both approaches can be used for any type of tool, including retrieval-based or text-generation tools.

},

- Answer choice: The `@tool` decorator is recommended for simple function-based tools, while subclasses of `Tool` offer more flexibility for complex functionality or custom metadata

Explanation: This is correct. The decorator approach is simpler, but subclassing allows more customized behavior.

Correct answer.

},

- Answer choice: `@tool` can only be used in multi-agent systems, while creating a `Tool` subclass is for single-agent scenarios

Explanation: All agents (single or multi) can use either approach to define tools; there is no such restriction.

},

- Answer choice: Decorating a function with `@tool` replaces the need for a docstring, whereas subclasses must not include docstrings

Explanation: Both methods benefit from clear docstrings. The decorator doesn't replace them, and a subclass can still have docstrings.

Q2: How does a CodeAgent handle multi-step tasks using the ReAct (Reason + Act) approach?

Which statement correctly describes how the CodeAgent executes a series of steps to solve a task?

- Answer choice: It passes each step to a different agent in a multi-agent system, then combines results

Explanation: Although multi-agent systems can distribute tasks, CodeAgent itself can handle multiple steps on its own using ReAct.

},

- Answer choice: It stores every action in JSON for easy parsing before executing them all at once

Explanation: This behavior matches ToolCallingAgent's JSON-based approach, not CodeAgent.

},

- Answer choice: It cycles through writing internal thoughts, generating Python code, executing the code, and logging the results until it arrives at a final answer

Explanation: Correct. This describes the ReAct pattern that CodeAgent uses, including iterative reasoning and code execution.

Correct answer.

},

- Answer choice: It relies on a vision module to validate code output before continuing to the next step

Explanation: Vision capabilities are supported in smolagents, but they're not a default requirement for CodeAgent or the ReAct approach.

Q3: Which of the following is a primary advantage of sharing a tool on the Hugging Face Hub?

Select the best reason why a developer might upload and share their custom tool.

- Answer choice: It automatically integrates the tool with a MultiStepAgent for retrieval-augmented generation

Explanation: Sharing a tool doesn't automatically set up retrieval or multi-step logic. It's just making the tool available.

},

- Answer choice: It allows others to discover, reuse, and integrate your tool in their smolagents without extra setup

Explanation: Yes. Sharing on the Hub makes tools accessible for anyone (including yourself) to download and reuse quickly.

Correct answer.

},

- Answer choice: It ensures that only CodeAgents can invoke the tool while ToolCallingAgents cannot

Explanation: Both CodeAgents and ToolCallingAgents can invoke shared tools. There's no restriction by agent type.

},

- Answer choice: It converts your tool into a fully vision-capable function for image processing

Explanation: Tool sharing doesn't alter the tool's functionality or add vision capabilities automatically.

Q4: ToolCallingAgent differs from CodeAgent in how it executes actions. Which statement is correct?

Choose the option that accurately describes how ToolCallingAgent works.

- Answer choice: ToolCallingAgent is only compatible with a multi-agent system, while CodeAgent can run alone

Explanation: Either agent can be used alone or as part of a multi-agent system.

},

- Answer choice: ToolCallingAgent delegates all reasoning to a separate retrieval agent, then returns a final answer

Explanation: ToolCallingAgent still uses a main LLM for reasoning; it doesn't rely solely on retrieval agents.

- },
- Answer choice: ToolCallingAgent outputs JSON instructions specifying tool calls and arguments, which get parsed and executed

Explanation: This is correct. ToolCallingAgent uses the JSON approach to define tool calls.

Correct answer.

- },
- Answer choice: ToolCallingAgent is only meant for single-step tasks and automatically stops after calling one tool

Explanation: ToolCallingAgent can perform multiple steps if needed, just like CodeAgent.

Q5: What is included in the smolagents default toolbox, and why might you use it?

Which statement best captures the purpose and contents of the default toolbox in smolagents?

- Answer choice: It provides a set of commonly-used tools such as DuckDuckGo search, PythonInterpreterTool, and a final answer tool for quick prototyping

Explanation: Correct. The default toolbox contains these ready-made tools for easy integration when building agents.

Correct answer.

- },
- Answer choice: It only supports vision-based tasks like image classification or OCR by default

Explanation: Although smolagents can integrate vision-based features, the default toolbox isn't exclusively vision-oriented.

- },
- Answer choice: It is intended solely for multi-agent systems and is incompatible with a single CodeAgent

Explanation: The default toolbox can be used by any agent type, single or multi-agent setups alike.

- },
- Answer choice: It adds advanced retrieval-based functionality for large-scale question answering from a vector store

Explanation: While you can build retrieval tools, the default toolbox does not automatically provide advanced RAG features.

Congratulations on completing this quiz! If any questions gave you trouble, revisit the Code Agents, Tool Calling Agents, or Tools sections to strengthen your understanding. If you aced it, you're well on your way to building robust smolagents applications!

Multi-Agent Systems

Source file: [units/en/unit2/smolagents/multi_agent_systems.mdx](#)

Multi-Agent Systems

Multi-agent systems enable specialized agents to collaborate on complex tasks, improving modularity, scalability, and robustness. Instead of relying on a single agent, tasks are distributed among agents with distinct capabilities.

In smolagents, different agents can be combined to generate Python code, call external tools, perform web searches, and more. By orchestrating these agents, we can create powerful workflows.

A typical setup might include:

- A Manager Agent for task delegation
- A Code Interpreter Agent for code execution
- A Web Search Agent for information retrieval

The diagram below illustrates a simple multi-agent architecture where a Manager Agent coordinates a Code Interpreter Tool and a Web Search Agent, which in turn utilizes tools like the DuckDuckGoSearchTool and VisitWebpageTool to gather relevant information.

Multi-Agent Systems in Action

A multi-agent system consists of multiple specialized agents working together under the coordination of an Orchestrator Agent. This approach enables complex workflows by distributing tasks among agents with distinct roles.

For example, a Multi-Agent RAG system can integrate:

- A Web Agent for browsing the internet.
- A Retriever Agent for fetching information from knowledge bases.
- An Image Generation Agent for producing visuals.

All of these agents operate under an orchestrator that manages task delegation and interaction.

Solving a complex task with a multi-agent hierarchy

You can follow the code in this notebook that you can run using Google Colab.

The reception is approaching! With your help, Alfred is now nearly finished with the preparations.

But now there's a problem: the Batmobile has disappeared. Alfred needs to find a replacement, and find it quickly.

Fortunately, a few biopics have been done on Bruce Wayne's life, so maybe Alfred could get a car left behind on one of the movie sets, and re-engineer it up to modern standards, which certainly would include a full self-driving option.

But this could be anywhere in the filming locations around the world - which could be numerous.

So Alfred wants your help. Could you build an agent able to solve this task?

Find all Batman filming locations in the world, calculate the time to transfer via boat to there, and represent them on a map, with a color varying by boat transfer time. Also represent some supercar factories with the same boat transfer time.

Let's build this!

This example needs some additional packages, so let's install them first:

```
pip install 'smolagents[litellm]' plotly geopandas shapely kaleido -q
```


We first make a tool to get the cargo plane transfer time.

```
import math
from typing import Optional, Tuple

from smolagents import tool

@tool
def calculate_cargo_travel_time(
    origin_coors: Tuple[float, float],
    destination_coors: Tuple[float, float],
    cruising_speed_kmh: Optional[float] = 750.0, # Average speed for cargo planes
) -> float:
    """
    Calculate the travel time for a cargo plane between two points on Earth using great-circle distance.

    Args:
        origin_coors: Tuple of (latitude, longitude) for the starting point
        destination_coors: Tuple of (latitude, longitude) for the destination
        cruising_speed_kmh: Optional cruising speed in km/h (defaults to 750 km/h for typical cargo plane \
s)

    Returns:
        float: The estimated travel time in hours

    Example:
        >>> # Chicago (41.8781° N, 87.6298° W) to Sydney (33.8688° S, 151.2093° E)
        >>> result = calculate_cargo_travel_time((41.8781, -87.6298), (-33.8688, 151.2093))
        """

    def to_radians(degrees: float) -> float:
        return degrees * (math.pi / 180)

    # Extract coordinates
    lat1, lon1 = map(to_radians, origin_coors)
    lat2, lon2 = map(to_radians, destination_coors)

    # Earth's radius in kilometers
    EARTH_RADIUS_KM = 6371.0

    # Calculate great-circle distance using the haversine formula
    dlon = lon2 - lon1
    dlat = lat2 - lat1

    a = (
        math.sin(dlat / 2) ** 2
        + math.cos(lat1) * math.cos(lat2) * math.sin(dlon / 2) ** 2
    )
    c = 2 * math.asin(math.sqrt(a))
    distance = EARTH_RADIUS_KM * c

    # Add 10% to account for non-direct routes and air traffic controls
    actual_distance = distance * 1.1

    # Calculate flight time
    # Add 1 hour for takeoff and landing procedures
    flight_time = (actual_distance / cruising_speed_kmh) + 1.0

    # Format the results
    return round(flight_time, 2)

print(calculate_cargo_travel_time((41.8781, -87.6298), (-33.8688, 151.2093)))
```

Setting up the agent

For the model provider, we use Together AI, one of the new inference providers on the Hub (<https://huggingface.co/blog/inference-providers>)!

The GoogleSearchTool uses the Serper API (<https://serper.dev>) to search the web, so this requires either having setup env variable SERPAPI_API_KEY and passing provider="serpapi" or having SERPER_API_KEY and passing provider=serper.

If you don't have any Serp API provider setup, you can use DuckDuckGoSearchTool but beware that it has a rate limit.

```
import os
from PIL import Image
from smolagents import CodeAgent, GoogleSearchTool, InferenceClientModel, VisitWebpageTool
```

```
model = InferenceClientModel(model_id="Qwen/Qwen2.5-Coder-32B-Instruct", provider="together")
```

We can start by creating a simple agent as a baseline to give us a simple report.

```
task = """Find all Batman filming locations in the world, calculate the time to transfer via cargo plane \
to here (we're in Gotham, 40.7128° N, 74.0060° W), and return them to me as a pandas dataframe. \
Also give me some supercar factories with the same cargo plane transfer time."""

agent = CodeAgent(
    model=model,
    tools=[GoogleSearchTool("serper"), VisitWebpageTool(), calculate_cargo_travel_time],
    additional_authorized_imports=["pandas"],
    max_steps=20,
)

result = agent.run(task)

result
```

In our case, it generates this output:

	Location	Travel Time to Gotham (hours)
0	Necropolis Cemetery, Glasgow, Scotland, UK	8.60
1	St. George's Hall, Liverpool, England, UK	8.81
2	Two Temple Place, London, England, UK	9.17
3	Wollaton Hall, Nottingham, England, UK	9.00
4	Knebworth House, Knebworth, Hertfordshire, UK	9.15
5	Acton Lane Power Station, Acton Lane, Acton, UK	9.16
6	Queensboro Bridge, New York City, USA	1.01
7	Wall Street, New York City, USA	1.00
8	Mehrangarh Fort, Jodhpur, Rajasthan, India	18.34
9	Turda Gorge, Turda, Romania	11.89
10	Chicago, USA	2.68
11	Hong Kong, China	19.99
12	Cardington Studios, Northamptonshire, UK	9.10
13	Warner Bros. Leavesden Studios, Hertfordshire, UK	9.13
14	Westwood, Los Angeles, CA, USA	6.79
15	Woking, UK (McLaren)	9.13

We could already improve this a bit by throwing in some dedicated planning steps, and adding more prompting.

Planning steps allow the agent to think ahead and plan its next steps, which can be useful for more complex tasks.

```
agent.planning_interval = 4

detailed_report = agent.run(f"""
You're an expert analyst. You make comprehensive reports after visiting many websites.
Don't hesitate to search for many queries at once in a for loop.
For each data point that you find, visit the source url to confirm numbers.

{task}
""")

print(detailed_report)

detailed_report
```

In our case, it generates this output:

	Location	Travel Time (hours)
0	Bridge of Sighs, Glasgow Necropolis, Glasgow, UK	8.6
1	Wishart Street, Glasgow, Scotland, UK	8.6

Thanks to these quick changes, we obtained a much more concise report by simply providing our agent a detailed prompt, and giving it planning capabilities!

The model's context window is quickly filling up. So if we ask our agent to combine the results of detailed search with another, it will be slower and quickly ramp up tokens and costs.

We need to improve the structure of our system.

Splitting the task between two agents

Multi-agent structures allow to separate memories between different sub-tasks, with two great benefits:

- Each agent is more focused on its core task, thus more performant
- Separating memories reduces the count of input tokens at each step, thus reducing latency and cost.

Let's create a team with a dedicated web search agent, managed by another agent.

The manager agent should have plotting capabilities to write its final report: so let us give it access to additional imports, including plotly, and geopandas + shapely for spatial plotting.

```
model = InferenceClientModel(
    "Qwen/Qwen2.5-Coder-32B-Instruct", provider="together", max_tokens=8096
)

web_agent = CodeAgent(
    model=model,
    tools=[
        GoogleSearchTool(provider="serper"),
        VisitWebpageTool(),
        calculate_cargo_travel_time,
    ],
    name="web_agent",
    description="Browses the web to find information",
    verbosity_level=0,
    max_steps=10,
)
```

The manager agent will need to do some mental heavy lifting.

So we give it the stronger model DeepSeek-R1 (<https://huggingface.co/deepseek-ai/DeepSeek-R1>), and add a planning_interval to the mix.

```
from smolagents.utils import encode_image_base64, make_image_url
from smolagents import OpenAIServerModel

def check_reasoning_and_plot(final_answer, agent_memory):
    multimodal_model = OpenAIServerModel("gpt-4o", max_tokens=8096)
    filepath = "saved_map.png"
    assert os.path.exists(filepath), "Make sure to save the plot under saved_map.png!"
    image = Image.open(filepath)
    prompt = (
        f"Here is a user-given task and the agent steps: {agent_memory.get_succinct_steps()}. Now here is \
the plot that was made."
        "Please check that the reasoning process and plot are correct: do they correctly answer the given \
task?"
        "First list reasons why yes/no, then write your final decision: PASS in caps lock if it is satisf \
actory, FAIL if it is not."
        "Don't be harsh: if the plot mostly solves the task, it should pass."
        "To pass, a plot should be made using px.scatter_map and not any other method (scatter_map looks \
nicer)."
    )
    messages = [
        {
            "role": "user",
            "content": [
                {
                    "type": "text",
                    "text": prompt,
                },
                {
                    "type": "image_url",
                    "image_url": {"url": make_image_url(encode_image_base64(image))},
                },
            ],
        },
    ]
    output = multimodal_model(messages).content
    print("Feedback: ", output)
    if "FAIL" in output:
        raise Exception(output)
    return True

manager_agent = CodeAgent(
    model=InferenceClientModel("deepseek-ai/DeepSeek-R1", provider="together", max_tokens=8096),
    tools=[calculate_cargo_travel_time],
    managed_agents=[web_agent],
    additional_authorized_imports=[
        "geopandas",
        "plotly",
        "shapely",
        "json",
        "pandas",
    ],
)
```

```

    "numpy",
],
planning_interval=5,
verbosity_level=2,
final_answer_checks=[check_reasoning_and_plot],
max_steps=15,
)

```

Let us inspect what this team looks like:

```
manager_agent.visualize()
```

This will generate something like this, helping us understand the structure and relationship between agents and tools used:

```

CodeAgent | deepseek-ai/DeepSeek-R1
■■■ Authorized imports: ['geopandas', 'plotly', 'shapely', 'json', 'pandas', 'numpy']
■■■ Tools:
■■■ 
■■■ 
■■■ Name          Description          Arguments          \
■■■ 
■■■ 
■■■ calculate_cargo_travel_time ■ Calculate the travel time for a cargo ■ origin_coords (`array`): Tuple \
e of ■                ■ plane between two points on Earth ■ (latitude, longitude) for the \
■■■                ■ using great-circle distance. ■ starting point          \
■■■                ■                ■ destination_coords (`array`): \
Tuple ■                ■                ■ of (latitude, longitude) for \
the ■                ■                ■ destination              \
■■■                ■                ■ cruising_speed_kmh (`number`) \
: ■                ■                ■ Optional cruising speed in km \
■■■                ■                ■ (defaults to 750 km/h for typ \
/h ■                ■                ■ cargo planes)            \
■■■                ■                \
■■■ final_answer        ■ Provides a final answer to the given ■ answer (`any`): The final ans \
wer to ■                ■ problem.                ■ the problem              \
■■■                \
■■■ 
■■■ Managed agents:
■■■ web_agent | CodeAgent | Qwen/Qwen2.5-Coder-32B-Instruct
■■■ Authorized imports: []
■■■ Description: Browses the web to find information
■■■ Tools:
■■■ 
■■■ 
■■■ Name          Description          Arguments          \
■■■ 
■■■ 
■■■ web_search      ■ Performs a google web search for ■ query (`string`): The sea \
rch ■                ■ your query then returns a string ■ query to perform.        \
■■■                ■ of the top search results. ■ filter_year (`integer`): \
■■■                ■                ■ Optionally restrict resul \
ts to a ■                ■                ■ certain year              \
■■■                \
■■■ visit_webpage   ■ Visits a webpage at the given url ■ url (`string`): The url o \
f the ■                ■ and reads its content as a ■ webpage to visit.        \
■■■                ■                \
■■■                ■ markdown string. Use this to ■                \
■■■                ■ browse webpages. ■                \
■■■                \
■■■ calculate_cargo_travel_time ■ Calculate the travel time for a ■ origin_coords (`array`): \
Tuple of ■                ■ cargo plane between two points on ■ (latitude, longitude) for \
■■■                \

```


However, Why use smolagents discusses alternative solutions supported by smolagents and Hugging Face. If you'd like to explore other options, be sure to check that section.

Empowering agents with visual capabilities is crucial for solving tasks that go beyond text processing. Many real-world challenges, such as web browsing or document understanding, require analyzing rich visual content. Fortunately, smolagents provides built-in support for vision-language models (VLMs), enabling agents to process and interpret images effectively.

In this example, imagine Alfred, the butler at Wayne Manor, is tasked with verifying the identities of the guests attending the party. As you can imagine, Alfred may not be familiar with everyone arriving. To help him, we can use an agent that verifies their identity by searching for visual information about their appearance using a VLM. This will allow Alfred to make informed decisions about who can enter. Let's build this example!

Providing Images at the Start of the Agent's Execution

You can follow the code in this notebook that you can run using Google Colab.

In this approach, images are passed to the agent at the start and stored as `task_images` alongside the task prompt. The agent then processes these images throughout its execution.

Consider the case where Alfred wants to verify the identities of the superheroes attending the party. He already has a dataset of images from previous parties with the names of the guests. Given a new visitor's image, the agent can compare it with the existing dataset and make a decision about letting them in.

In this case, a guest is trying to enter, and Alfred suspects that this visitor might be The Joker impersonating Wonder Woman. Alfred needs to verify their identity to prevent anyone unwanted from entering.

Let's build the example. First, the images are loaded. In this case, we use images from Wikipedia to keep the example minimal, but imagine the possible use-case!

```
from PIL import Image
import requests
from io import BytesIO

image_urls = [
    "https://upload.wikimedia.org/wikipedia/commons/e/e8/The_Joker_at_Wax_Museum_Plus.jpg", # Joker image
    "https://upload.wikimedia.org/wikipedia/en/9/98/Joker_%28DC_Comics_character%29.jpg" # Joker image
]

images = []
for url in image_urls:
    headers = {
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) C \
hrome/114.0.0.0 Safari/537.36"
    }
    response = requests.get(url, headers=headers)
    image = Image.open(BytesIO(response.content)).convert("RGB")
    images.append(image)
```

Now that we have the images, the agent will tell us whether one guest is actually a superhero (Wonder Woman) or a villain (The Joker).

```
from smolagents import CodeAgent, OpenAIServerModel

model = OpenAIServerModel(model_id="gpt-4o")

# Instantiate the agent
agent = CodeAgent(
    tools=[],
    model=model,
    max_steps=20,
    verbosity_level=2
)

response = agent.run(
    """
    Describe the costume and makeup that the comic character in these photos is wearing and return the de \
scription.
    Tell me if the guest is The Joker or Wonder Woman.
    """,
    images=images
)
```

In the case of my run, the output is the following, although it could vary in your case, as we've already discussed:

```
{
  'Costume and Makeup - First Image': (
    'Purple coat and a purple silk-like cravat or tie over a mustard-yellow shirt.',
    'White face paint with exaggerated features, dark eyebrows, blue eye makeup, red lips forming \
a wide smile.'
  ),
  'Costume and Makeup - Second Image': (
    'Dark suit with a flower on the lapel, holding a playing card.',
    'Pale skin, green hair, very red lips with an exaggerated grin.'
  ),
  'Character Identity': 'This character resembles known depictions of The Joker from comic book med \
ia.'
}
```

In this case, the output reveals that the person is impersonating someone else, so we can prevent The Joker from entering the party!

Providing Images with Dynamic Retrieval

You can follow the code in this Python file

The previous approach is valuable and has many potential use cases. However, in situations where the guest is not in the database, we need to explore other ways of identifying them. One possible solution is to dynamically retrieve images and information from external sources, such as browsing the web for details.

In this approach, images are dynamically added to the agent's memory during execution. As we know, agents in smolagents are based on the MultiStepAgent class, which is an abstraction of the ReAct framework. This class operates in a structured cycle where various variables and knowledge are logged at different stages:

1. SystemPromptStep: Stores the system prompt.
1. TaskStep: Logs the user query and any provided input.
1. ActionStep: Captures logs from the agent's actions and results.

This structured approach allows agents to incorporate visual information dynamically and respond adaptively to evolving tasks. Below is the diagram we've already seen, illustrating the dynamic workflow process and how different steps integrate within the agent lifecycle. When browsing, the agent can take screenshots and save them as `observation_images` in the ActionStep.

[Image omitted from text-first adaptation: Dynamic image retrieval]

Now that we understand the need, let's build our complete example. In this case, Alfred wants full control over the guest verification process, so browsing for details becomes a viable solution. To complete this example, we need a new set of tools for the agent. Additionally, we'll use Selenium and Helium, which are browser automation tools. This will allow us to build an agent that explores the web, searching for details about a potential guest and retrieving verification information. Let's install the tools needed:

```
pip install "smolagents[all]" helium selenium python-dotenv
```

We'll need a set of agent tools specifically designed for browsing, such as `search_item_ctrl_f`, `go_back`, and `close_popups`. These tools allow the agent to act like a person navigating the web.

```
@tool
def search_item_ctrl_f(text: str, nth_result: int = 1) -> str:
    """
    Searches for text on the current page via Ctrl + F and jumps to the nth occurrence.
    Args:
        text: The text to search for
        nth_result: Which occurrence to jump to (default: 1)
    """
    elements = driver.find_elements(By.XPATH, f"//*[contains(text(), '{text}')]")
    if nth_result > len(elements):
        raise Exception(f"Match n°{nth_result} not found (only {len(elements)} matches found)")
    result = f"Found {len(elements)} matches for '{text}'."
    elem = elements[nth_result - 1]
    driver.execute_script("arguments[0].scrollIntoView(true);", elem)
    result += f"Focused on element {nth_result} of {len(elements)}"
    return result
```

```
@tool
```

```
def go_back() -> None:
    """Goes back to previous page."""
    driver.back()

@tool
def close_popups() -> str:
    """
    Closes any visible modal or pop-up on the page. Use this to dismiss pop-up windows! This does not work on cookie consent banners.
    """
    webdriver.ActionChains(driver).send_keys(Keys.ESCAPE).perform()
```

We also need functionality for saving screenshots, as this will be an essential part of what our VLM agent uses to complete the task. This functionality captures the screenshot and saves it in `step_log.observations_images = [image.copy()]`, allowing the agent to store and process the images dynamically as it navigates.

```
def save_screenshot(step_log: ActionStep, agent: CodeAgent) -> None:
    sleep(1.0) # Let JavaScript animations happen before taking the screenshot
    driver = helium.get_driver()
    current_step = step_log.step_number
    if driver is not None:
        for step_logs in agent.logs: # Remove previous screenshots from logs for lean processing
            if isinstance(step_log, ActionStep) and step_log.step_number <= current_step - 2:
                step_logs.observations_images = None
        png_bytes = driver.get_screenshot_as_png()
        image = Image.open(BytesIO(png_bytes))
        print(f"Captured a browser screenshot: {image.size} pixels")
        step_log.observations_images = [image.copy()] # Create a copy to ensure it persists, important!

    # Update observations with current URL
    url_info = f"Current url: {driver.current_url}"
    step_log.observations = url_info if step_logs.observations is None else step_log.observations + "\n" + url_info
    return
```

This function is passed to the agent as `step_callback`, as it's triggered at the end of each step during the agent's execution. This allows the agent to dynamically capture and store screenshots throughout its process.

Now, we can generate our vision agent for browsing the web, providing it with the tools we created, along with the `DuckDuckGoSearchTool` to explore the web. This tool will help the agent retrieve necessary information for verifying guests' identities based on visual cues.

```
from smolagents import CodeAgent, OpenAIServerModel, DuckDuckGoSearchTool
model = OpenAIServerModel(model_id="gpt-4o")

agent = CodeAgent(
    tools=[DuckDuckGoSearchTool(), go_back, close_popups, search_item_ctrl_f],
    model=model,
    additional_authorized_imports=["helium"],
    step_callbacks=[save_screenshot],
    max_steps=20,
    verbosity_level=2,
)
```

With that, Alfred is ready to check the guests' identities and make informed decisions about whether to let them into the party:

```
agent.run("""
I am Alfred, the butler of Wayne Manor, responsible for verifying the identity of guests at party. A superhero has arrived at the entrance claiming to be Wonder Woman, but I need to confirm if she is who she says she is.

Please search for images of Wonder Woman and generate a detailed visual description based on those images. Additionally, navigate to Wikipedia to gather key details about her appearance. With this information, I can determine whether to grant her access to the event.
""") + helium_instructions)
```

You can see that we include `helium_instructions` as part of the task. This special prompt is aimed to control the navigation of the agent, ensuring that it follows the correct steps while browsing the web.

Let's see how this works in the video below:

[Embedded media omitted from text-first adaptation: YouTube video player]

This is the final output:

```
Final answer: Wonder Woman is typically depicted wearing a red and gold bustier, blue shorts or skirt with white stars, a golden tiara, silver bracelets, and a golden Lasso of Truth. She is Princess Diana of Themyscira.
```


f Themyscira, known as Diana Prince in the world of men.

With all of that, we've successfully created our identity verifier for the party! Alfred now has the necessary tools to ensure only the right guests make it through the door. Everything is set to have a good time at Wayne Manor!

Further Reading

- We just gave sight to smolagents (<https://huggingface.co/blog/smolagents-can-see>) - Blog describing the vision agent functionality.
- Web Browser Automation with Agents (https://huggingface.co/docs/smolagents/examples/web_browser) - Example for Web browsing using a vision agent.
- Web Browser Vision Agent Example (https://github.com/huggingface/smolagents/blob/main/src/smolagents/vision_web_browser.py) - Example for Web browsing using a vision agent.

Final Quiz

Source file: [units/en/unit2/smolagents/final_quiz.mdx](#)

Exam Time!

Well done on working through the material on smolagents! You've already achieved a lot. Now, it's time to put your knowledge to the test with a quiz.

Instructions

- The quiz consists of code questions.
- You will be given instructions to complete the code snippets.
- Read the instructions carefully and complete the code snippets accordingly.
- For each question, you will be given the result and some feedback.

This quiz is ungraded and uncertified. It's about you understanding the smolagents library and knowing whether you should spend more time on the written material. In the coming units you'll put this knowledge to the test in use cases and projects.

Let's get started!

Quiz

[Embedded media omitted from text-first adaptation: interactive media]

You can also access the quiz here (https://huggingface.co/spaces/agents-course/unit2_smolagents_quiz)

Conclusion

Source file: [units/en/unit2/smolagents/conclusion.mdx](#)

Conclusion

Congratulations on finishing the smolagents module of this second Unit

You've just mastered the fundamentals of smolagents and you've built your own Agent! Now that you have skills in smolagents, you can now start to create Agents that will solve tasks you're interested about.

In the next module, you're going to learn how to build Agents with LlamaIndex.

Finally, we would love to hear what you think of the course and how we can improve it. If you have some feedback then, please fill this form

(<https://docs.google.com/forms/d/e/1FAIpQLSe9VaONn0egIax0uTwI29rIn4tM7H2sYmmybmG5jJNIE5v0xA/viewform?usp=dialog>)

Keep Learning, stay awesome

Unit 2.2 The LlamaIndex framework

Instructional sections below follow the official English course order.

Introduction to LlamaIndex

Source file: `units/en/unit2/llama-index/introduction.mdx`

Introduction to LlamaIndex

Welcome to this module, where you'll learn how to build LLM-powered agents using the LlamaIndex (<https://www.llamaindex.ai/>) toolkit.

LlamaIndex is a complete toolkit for creating LLM-powered agents over your data using indexes and workflows. For this course we'll focus on three main parts that help build agents in LlamaIndex: Components, Agents and Tools and Workflows.

[Image omitted from text-first adaptation: LlamaIndex]

Let's look at these key parts of LlamaIndex and how they help with agents:

- **Components:** Are the basic building blocks you use in LlamaIndex. These include things like prompts, models, and databases. Components often help connect LlamaIndex with other tools and libraries.
- **Tools:** Tools are components that provide specific capabilities like searching, calculating, or accessing external services. They are the building blocks that enable agents to perform tasks.
- **Agents:** Agents are autonomous components that can use tools and make decisions. They coordinate tool usage to accomplish complex goals.
- **Workflows:** Are step-by-step processes that process logic together. Workflows or agentic workflows are a way to structure agentic behaviour without the explicit use of agents.

What Makes LlamaIndex Special?

While LlamaIndex does some things similar to other frameworks like smolagents, it has some key benefits:

- **Clear Workflow System:** Workflows help break down how agents should make decisions step by step using an event-driven and async-first syntax. This helps you clearly compose and organize your logic.
- **Advanced Document Parsing with LlamaParse:** LlamaParse was made specifically for LlamaIndex, so the integration is seamless, although it is a paid feature.
- **Many Ready-to-Use Components:** LlamaIndex has been around for a while, so it works with lots of other frameworks. This means it has many tested and reliable components, like LLMs, retrievers, indexes, and more.
- **LlamaHub:** is a registry of hundreds of these components, agents, and tools that you can use within LlamaIndex.

All of these concepts are required in different scenarios to create useful agents. In the following sections, we will go over each of these concepts in detail. After mastering the concepts, we will use our learnings to create applied use cases with Alfred the agent!

Getting our hands on LlamaIndex is exciting, right? So, what are we waiting for? Let's get started with finding and installing the integrations we need using LlamaHub!

Introduction to LlamaHub

Source file: `units/en/unit2/llama-index/llama-hub.mdx`

Introduction to the LlamaHub

LlamaHub is a registry of hundreds of integrations, agents and tools that you can use within LlamaIndex.

[Image omitted from text-first adaptation: LlamaHub]

We will be using various integrations in this course, so let's first look at the LlamaHub and how it can help us.

Let's see how to find and install the dependencies for the components we need.

Installation

LlamaIndex installation instructions are available as a well-structured overview on LlamaHub (<https://llamahub.ai/>). This might be a bit overwhelming at first, but most of the installation commands generally follow an easy-to-remember format:

```
pip install llama-index-{component-type}-{framework-name}
```

Let's try to install the dependencies for an LLM and embedding component using the Hugging Face inference API integration (<https://llamahub.ai/llms/llama-index-llms-huggingface-api?from=llms>).

```
pip install llama-index-llms-huggingface-api llama-index-embeddings-huggingface
```

Usage

Once installed, we can see the usage patterns. You'll notice that the import paths follow the install command! Underneath, we can see an example of the usage of the Hugging Face inference API for an LLM component.

```
from llama_index.llms.huggingface_api import HuggingFaceInferenceAPI
import os
from dotenv import load_dotenv

# Load the .env file
load_dotenv()

# Retrieve HF_TOKEN from the environment variables
hf_token = os.getenv("HF_TOKEN")

llm = HuggingFaceInferenceAPI(
    model_name="Qwen/Qwen2.5-Coder-32B-Instruct",
    temperature=0.7,
    max_tokens=100,
    token=hf_token,
    provider="auto"
)

response = llm.complete("Hello, how are you?")
print(response)
# I am good, how can I help you today?
```

Wonderful, we now know how to find, install and use the integrations for the components we need. Let's dive deeper into the components and see how we can use them to build our own agents.

What are Components in LlamaIndex?

Source file: [units/en/unit2/llama-index/components.mdx](https://github.com/run-llama/llama-index/blob/main/docs/en/unit2/llama-index/components.mdx)

What are components in LlamaIndex?

Remember Alfred, our helpful butler agent from Unit 1? To assist us effectively, Alfred needs to understand our requests and prepare, find and use relevant information to help complete tasks. This is where LlamaIndex's components come in.

While LlamaIndex has many components, we'll focus specifically on the QueryEngine component. Why? Because it can be used as a Retrieval-Augmented Generation (RAG) tool for an agent.

So, what is RAG? LLMs are trained on enormous bodies of data to learn general knowledge. However, they may not be trained on relevant and up-to-date data. RAG solves this problem by finding and retrieving relevant information from your data and giving that to the LLM.

[Image omitted from text-first adaptation: RAG]

Now, think about how Alfred works:

1. You ask Alfred to help plan a dinner party
1. Alfred needs to check your calendar, dietary preferences, and past successful menus

1. The QueryEngine helps Alfred find this information and use it to plan the dinner party

This makes the QueryEngine a key component for building agentic RAG workflows in LlamaIndex. Just as Alfred needs to search through your household information to be helpful, any agent needs a way to find and understand relevant data. The QueryEngine provides exactly this capability.

Now, let's dive a bit deeper into the components and see how you can combine components to create a RAG pipeline.

Creating a RAG pipeline using components

You can follow the code in this notebook that you can run using Google Colab.

There are five key stages within RAG, which in turn will be a part of most larger applications you build. These are:

1. Loading: this refers to getting your data from where it lives -- whether it's text files, PDFs, another website, a database, or an API -- into your workflow. LlamaHub provides hundreds of integrations to choose from.
1. Indexing: this means creating a data structure that allows for querying the data. For LLMs, this nearly always means creating vector embeddings. Which are numerical representations of the meaning of the data. Indexing can also refer to numerous other metadata strategies to make it easy to accurately find contextually relevant data based on properties.
1. Storing: once your data is indexed you will want to store your index, as well as other metadata, to avoid having to re-index it.
1. Querying: for any given indexing strategy there are many ways you can utilize LLMs and LlamaIndex data structures to query, including sub-queries, multi-step queries and hybrid strategies.
1. Evaluation: a critical step in any flow is checking how effective it is relative to other strategies, or when you make changes. Evaluation provides objective measures of how accurate, faithful and fast your responses to queries are.

Next, let's see how we can reproduce these stages using components.

Loading and embedding documents

As mentioned before, LlamaIndex can work on top of your own data, however, before accessing data, we need to load it. There are three main ways to load data into LlamaIndex:

1. SimpleDirectoryReader: A built-in loader for various file types from a local directory.
1. LlamaParse: LlamaParse, LlamaIndex's official tool for PDF parsing, available as a managed API.
1. LlamaHub: A registry of hundreds of data-loading libraries to ingest data from any source.

Get familiar with LlamaHub loaders and LlamaParse parser for more complex data sources.

The simplest way to load data is with SimpleDirectoryReader. This versatile component can load various file types from a folder and convert them into Document objects that LlamaIndex can work with. Let's see how we can use SimpleDirectoryReader to load data from a folder.

```
from llama_index.core import SimpleDirectoryReader

reader = SimpleDirectoryReader(input_dir="path/to/directory")
documents = reader.load_data()
```

After loading our documents, we need to break them into smaller pieces called Node objects. A Node is just a chunk of text from the original document that's easier for the AI to work with, while it still has references to the original Document object.

The IngestionPipeline helps us create these nodes through two key transformations.

1. SentenceSplitter breaks down documents into manageable chunks by splitting them at natural sentence boundaries.
1. HuggingFaceEmbedding converts each chunk into numerical embeddings - vector representations that capture the semantic meaning in a way AI can process efficiently.

This process helps us organise our documents in a way that's more useful for searching and analysis.

```
from llama_index.core import Document
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
from llama_index.core.node_parser import SentenceSplitter
from llama_index.core.ingestion import IngestionPipeline
```

```
# create the pipeline with transformations
pipeline = IngestionPipeline(
    transformations=[
        SentenceSplitter(chunk_overlap=0),
        HuggingFaceEmbedding(model_name="BAAI/bge-small-en-v1.5"),
    ]
)

nodes = await pipeline.arun(documents=[Document.example()])
```

Storing and indexing documents

After creating our Node objects we need to index them to make them searchable, but before we can do that, we need a place to store our data.

Since we are using an ingestion pipeline, we can directly attach a vector store to the pipeline to populate it. In this case, we will use Chroma to store our documents.

Install ChromaDB

As introduced in the section on the LlamaHub, we can install the ChromaDB vector store with the following command:

```
pip install llama-index-vector-stores-chroma

import chromadb
from llama_index.vector_stores.chroma import ChromaVectorStore

db = chromadb.PersistentClient(path="./alfred_chroma_db")
chroma_collection = db.get_or_create_collection("alfred")
vector_store = ChromaVectorStore(chroma_collection=chroma_collection)

pipeline = IngestionPipeline(
    transformations=[
        SentenceSplitter(chunk_size=25, chunk_overlap=0),
        HuggingFaceEmbedding(model_name="BAAI/bge-small-en-v1.5"),
    ],
    vector_store=vector_store,
)
```

An overview of the different vector stores can be found in the LlamaIndex documentation.

This is where vector embeddings come in - by embedding both the query and nodes in the same vector space, we can find relevant matches. The VectorStoreIndex handles this for us, using the same embedding model we used during ingestion to ensure consistency.

Let's see how to create this index from our vector store and embeddings:

```
from llama_index.core import VectorStoreIndex
from llama_index.embeddings.huggingface import HuggingFaceEmbedding

embed_model = HuggingFaceEmbedding(model_name="BAAI/bge-small-en-v1.5")
index = VectorStoreIndex.from_vector_store(vector_store, embed_model=embed_model)
```

All information is automatically persisted within the ChromaVectorStore object and the passed directory path.

Great! Now that we can save and load our index easily, let's explore how to query it in different ways.

Querying a VectorStoreIndex with prompts and LLMs

Before we can query our index, we need to convert it to a query interface. The most common conversion options are:

- `as_retriever`: For basic document retrieval, returning a list of `NodeWithScore` objects with similarity scores
- `as_query_engine`: For single question-answer interactions, returning a written response
- `as_chat_engine`: For conversational interactions that maintain memory across multiple messages, returning a written response using chat history and indexed context

We'll focus on the query engine since it is more common for agent-like interactions. We also pass in an LLM to the query engine to use for the response.

```
from llama_index.llms.huggingface_api import HuggingFaceInferenceAPI

llm = HuggingFaceInferenceAPI(model_name="Qwen/Qwen2.5-Coder-32B-Instruct")
query_engine = index.as_query_engine(
```

```

    llm=llm,
    response_mode="tree_summarize",
)
query_engine.query("What is the meaning of life?")
# The meaning of life is 42

```

Response Processing

Under the hood, the query engine doesn't only use the LLM to answer the question but also uses a ResponseSynthesizer as a strategy to process the response. Once again, this is fully customisable but there are three main strategies that work well out of the box:

- refine: create and refine an answer by sequentially going through each retrieved text chunk. This makes a separate LLM call per Node/retrieved chunk.
- compact (default): similar to refining but concatenating the chunks beforehand, resulting in fewer LLM calls.
- tree_summarize: create a detailed answer by going through each retrieved text chunk and creating a tree structure of the answer.

Take fine-grained control of your query workflows with the low-level composition API. This API lets you customize and fine-tune every step of the query process to match your exact needs, which also pairs great with Workflows

The language model won't always perform in predictable ways, so we can't be sure that the answer we get is always correct. We can deal with this by evaluating the quality of the answer.

Evaluation and observability

LlamaIndex provides built-in evaluation tools to assess response quality. These evaluators leverage LLMs to analyze responses across different dimensions. Let's look at the three main evaluators available:

- FaithfulnessEvaluator: Evaluates the faithfulness of the answer by checking if the answer is supported by the context.
- AnswerRelevancyEvaluator: Evaluate the relevance of the answer by checking if the answer is relevant to the question.
- CorrectnessEvaluator: Evaluate the correctness of the answer by checking if the answer is correct.

Want to learn more about agent observability and evaluation? Continue your journey with the Bonus Unit 2.

```

from llama_index.core.evaluation import FaithfulnessEvaluator

query_engine = # from the previous section
llm = # from the previous section

# query index
evaluator = FaithfulnessEvaluator(llm=llm)
response = query_engine.query(
    "What battles took place in New York City in the American Revolution?"
)
eval_result = evaluator.evaluate_response(response=response)
eval_result.passing

```

Even without direct evaluation, we can gain insights into how our system is performing through observability. This is especially useful when we are building more complex workflows and want to understand how each component is performing.

Install LlamaTrace

As introduced in the section on the LlamaHub, we can install the LlamaTrace callback from Arize Phoenix with the following command:

```
pip install -U llama-index-callbacks-arize-phoenix
```

Additionally, we need to set the PHOENIX_API_KEY environment variable to our LlamaTrace API key. We can get this by:

- Creating an account at LlamaTrace (<https://llamatrace.com/login>)
- Generating an API key in your account settings
- Using the API key in the code below to enable tracing

```

import llama_index
import os

PHOENIX_API_KEY = "<PHOENIX_API_KEY>"

```

```
os.environ["OTEL_EXPORTER_OTLP_HEADERS"] = f"api_key={PHOENIX_API_KEY}"
llama_index.core.set_global_handler(
    "arize_phoenix",
    endpoint="https://llamatrace.com/v1/traces"
)
```

Want to learn more about components and how to use them? Continue your journey with the [Components Guides](#) or the [Guide on RAG](#).

We have seen how to use components to create a QueryEngine. Now, let's see how we can use the QueryEngine as a tool for an agent!

Using Tools in LlamaIndex

Source file: [units/en/unit2/llama-index/tools.mdx](#)

Using Tools in LlamaIndex

Defining a clear set of Tools is crucial to performance. As we discussed in unit 1 ([../unit1/tools](#)), clear tool interfaces are easier for LLMs to use. Much like a software API interface for human engineers, they can get more out of the tool if it's easy to understand how it works.

There are four main types of tools in LlamaIndex:

[Image omitted from text-first adaptation: Tools]

1. **FunctionTool**: Convert any Python function into a tool that an agent can use. It automatically figures out how the function works.
1. **QueryEngineTool**: A tool that lets agents use query engines. Since agents are built on query engines, they can also use other agents as tools.
1. **Toolspecs**: Sets of tools created by the community, which often include tools for specific services like Gmail.
1. **Utility Tools**: Special tools that help handle large amounts of data from other tools.

We will go over each of them in more detail below.

Creating a FunctionTool

You can follow the code in this notebook that you can run using [Google Colab](#).

A FunctionTool provides a simple way to wrap any Python function and make it available to an agent. You can pass either a synchronous or asynchronous function to the tool, along with optional name and description parameters. The name and description are particularly important as they help the agent understand when and how to use the tool effectively. Let's look at how to create a FunctionTool below and then call it.

```
from llama_index.core.tools import FunctionTool

def get_weather(location: str) -> str:
    """Useful for getting the weather for a given location."""
    print(f"Getting weather for {location}")
    return f"The weather in {location} is sunny"

tool = FunctionTool.from_defaults(
    get_weather,
    name="my_weather_tool",
    description="Useful for getting the weather for a given location.",
)
tool.call("New York")
```

When using an agent or LLM with function calling, the tool selected (and the arguments written for that tool) rely strongly on the tool name and description of the purpose and arguments of the tool. Learn more about function calling in the [Function Calling Guide](#).

Creating a QueryEngineTool

The QueryEngine we defined in the previous unit can be easily transformed into a tool using the QueryEngineTool class. Let's see how to create a QueryEngineTool from a QueryEngine in the example below.


```

from llama_index.core import VectorStoreIndex
from llama_index.core.tools import QueryEngineTool
from llama_index.llms.huggingface_api import HuggingFaceInferenceAPI
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
from llama_index.vector_stores.chroma import ChromaVectorStore

embed_model = HuggingFaceEmbedding("BAAI/bge-small-en-v1.5")

db = chromadb.PersistentClient(path="./alfred_chroma_db")
chroma_collection = db.get_or_create_collection("alfred")
vector_store = ChromaVectorStore(chroma_collection=chroma_collection)

index = VectorStoreIndex.from_vector_store(vector_store, embed_model=embed_model)

llm = HuggingFaceInferenceAPI(model_name="Qwen/Qwen2.5-Coder-32B-Instruct")
query_engine = index.as_query_engine(llm=llm)
tool = QueryEngineTool.from_defaults(query_engine, name="some useful name", description="some useful desc \
ription")

```

Creating ToolSpecs

Think of ToolSpecs as collections of tools that work together harmoniously - like a well-organized professional toolkit. Just as a mechanic's toolkit contains complementary tools that work together for vehicle repairs, a ToolSpec combines related tools for specific purposes. For example, an accounting agent's ToolSpec might elegantly integrate spreadsheet capabilities, email functionality, and calculation tools to handle financial tasks with precision and efficiency.

Install the Google ToolSpec As introduced in the section on the LlamaHub, we can install the Google toolspec with the following command:

```
pip install llama-index-tools-google
```

And now we can load the toolspec and convert it to a list of tools.

```

from llama_index.tools.google import GmailToolSpec

tool_spec = GmailToolSpec()
tool_spec_list = tool_spec.to_tool_list()

```

To get a more detailed view of the tools, we can take a look at the metadata of each tool.

```
[(tool.metadata.name, tool.metadata.description) for tool in tool_spec_list]
```

Model Context Protocol (MCP) in LlamaIndex

LlamaIndex also allows using MCP tools through a ToolSpec on the LlamaHub (<https://llamahub.ai//tools/llama-index-tools-mcp?from=>). You can simply run an MCP server and start using it through the following implementation.

If you want to dive deeper about MCP, you can check our free MCP Course (<https://huggingface.co/learn/mcp-course/>).

Install the MCP ToolSpec As introduced in the section on the LlamaHub, we can install the MCP toolspec with the following command:

```

pip install llama-index-tools-mcp

from llama_index.tools.mcp import BasicMCPClient, McpToolSpec

# We consider there is a mcp server running on 127.0.0.1:8000, or you can use the mcp client to connect to your own mcp server.
mcp_client = BasicMCPClient("http://127.0.0.1:8000/sse")
mcp_tool = McpToolSpec(client=mcp_client)

# get the agent
agent = await get_agent(mcp_tool)

# create the agent context
agent_context = Context(agent)

```

Utility Tools

Oftentimes, directly querying an API can return an excessive amount of data, some of which may be irrelevant, overflow the context window of the LLM, or unnecessarily increase the number of tokens that you are using. Let's walk through our two main utility tools below.

1. **OnDemandToolLoader**: This tool turns any existing LlamaIndex data loader (BaseReader class) into a tool that an agent can use. The tool can be called with all the parameters needed to trigger `load_data` from the data loader, along with a natural language query string. During execution, we first load data from the data loader, index it (for instance with a vector store), and then query it 'on-demand'. All three of these steps happen in a single tool call.
1. **LoadAndSearchToolSpec**: The `LoadAndSearchToolSpec` takes in any existing Tool as input. As a tool spec, it implements `to_tool_list`, and when that function is called, two tools are returned: a loading tool and then a search tool. The load Tool execution would call the underlying Tool, and then index the output (by default with a vector index). The search Tool execution would take in a query string as input and call the underlying index.

You can find toolspecs and utility tools on the LlamaHub

Now that we understand the basics of agents and tools in LlamaIndex, let's see how we can use LlamaIndex to create configurable and manageable workflows!

Quick Quiz 1

Source file: `units/en/unit2/llama-index/quiz1.mdx`

Small Quiz (ungraded)

So far we've discussed the key components and tools used in LlamaIndex. It's time to make a short quiz, since testing yourself is the best way to learn and to avoid the illusion of competence

(<https://www.coursera.org/lecture/learning-how-to-learn/illusions-of-competence-BuFzf>). This will help you find where you need to reinforce your knowledge.

This is an optional quiz and it's not graded.

Q1: What is a QueryEngine?

Which of the following best describes a QueryEngine component?

- Answer choice: A system that only processes static text without any retrieval capabilities.

Explanation: A QueryEngine must be able to retrieve and process relevant information.

},

- Answer choice: A component that finds and retrieves relevant information as part of the RAG process.

Explanation: This captures the core purpose of a QueryEngine component.

Correct answer.

},

- Answer choice: A tool that only stores vector embeddings without search functionality.

Explanation: A QueryEngine does more than just store embeddings - it actively searches and retrieves information.

},

- Answer choice: A component that only evaluates response quality.

Explanation: Evaluation is separate from the QueryEngine's main retrieval purpose.

Q2: What is the Purpose of FunctionTools?

Why are FunctionTools important for an Agent?

- Answer choice: To handle large amounts of data storage.

Explanation: FunctionTools are not primarily for data storage.

},

- Answer choice: To convert Python functions into tools that an agent can use.

Explanation: FunctionTools wrap Python functions to make them accessible to agents.

Correct answer.

},

- Answer choice: To allow agents to create random functions definitions.

Explanation: FunctionTools serve the specific purpose of making functions available to agents.

},

- Answer choice: To only process text data.

Explanation: FunctionTools can work with various types of functions, not just text processing.

Q3: What are Toolspecs in LlamaIndex?

What is the main purpose of Toolspecs?

- Answer choice: They are redundant components that don't add functionality.

Explanation: Toolspecs serve an important purpose in the LlamaIndex ecosystem.

},

- Answer choice: They are sets of community-created tools that extend agent capabilities.

Explanation: Toolspecs allow the community to share and reuse tools.

Correct answer.

},

- Answer choice: They are used solely for memory management.

Explanation: Toolspecs are about providing tools, not managing memory.

},

- Answer choice: They only work with text processing.

Explanation: Toolspecs can include various types of tools, not just text processing.

Q4: What is Required to create a tool?

What information must be included when creating a tool?

- Answer choice: A function, a name, and description must be defined.

Explanation: While these all make up a tool, the name and description can be parsed from the function and docstring.

},

- Answer choice: Only the name is required.

Explanation: A function and description/docstring is also required for proper tool documentation.

},

- Answer choice: Only the description is required.

Explanation: A function is required so that we have code to run when an agent selects a tool

},

- Answer choice: Only the function is required.

Explanation: The name and description default to the name and docstring from the provided function

Correct answer.

Congrats on finishing this Quiz , if you missed some elements, take time to read again the chapter to reinforce your knowledge. If you pass it, you're ready to dive deeper into building with these components!

Using Agents in LlamaIndex

Source file: [units/en/unit2/llama-index/agents.mdx](#)

Using Agents in LlamaIndex

Remember Alfred, our helpful butler agent from earlier? Well, he's about to get an upgrade! Now that we understand the tools available in LlamaIndex, we can give Alfred new capabilities to serve us better.

But before we continue, let's remind ourselves what makes an agent like Alfred tick. Back in Unit 1, we learned that:

An Agent is a system that leverages an AI model to interact with its environment to achieve a user-defined objective. It combines reasoning, planning, and action execution (often via external tools) to fulfil tasks.

LlamaIndex supports three main types of reasoning agents:

[Image omitted from text-first adaptation: Agents]

1. Function Calling Agents - These work with AI models that can call specific functions.
1. ReAct Agents - These can work with any AI that does chat or text endpoint and deal with complex reasoning tasks.
1. Advanced Custom Agents - These use more complex methods to deal with more complex tasks and workflows.

Find more information on advanced agents on BaseWorkflowAgent

Initialising Agents

You can follow the code in this notebook that you can run using Google Colab.

To create an agent, we start by providing it with a set of functions/tools that define its capabilities. Let's look at how to create an agent with some basic tools. As of this writing, the agent will automatically use the function calling API (if available), or a standard ReAct agent loop.

LLMs that support a tools/functions API are relatively new, but they provide a powerful way to call tools by avoiding specific prompting and allowing the LLM to create tool calls based on provided schemas.

ReAct agents are also good at complex reasoning tasks and can work with any LLM that has chat or text completion capabilities. They are more verbose, and show the reasoning behind certain actions that they take.

```
from llama_index.llms.huggingface_api import HuggingFaceInferenceAPI
from llama_index.core.agent.workflow import AgentWorkflow
from llama_index.core.tools import FunctionTool

# define sample Tool -- type annotations, function names, and docstrings, are all included in parsed schema!
def multiply(a: int, b: int) -> int:
    """Multiplies two integers and returns the resulting integer"""
    return a * b

# initialize llm
llm = HuggingFaceInferenceAPI(model_name="Qwen/Qwen2.5-Coder-32B-Instruct")

# initialize agent
agent = AgentWorkflow.from_tools_or_functions(
    [FunctionTool.from_defaults(multiply)],
    llm=llm
)
```

Agents are stateless by default, however, they can remember past interactions using a Context object. This might be useful if you want to use an agent that needs to remember previous interactions, like a chatbot that maintains context across multiple messages or a task manager that needs to track progress over time.

```
# stateless
response = await agent.run("What is 2 times 2?")

# remembering state
from llama_index.core.workflow import Context

ctx = Context(agent)
```

```
response = await agent.run("My name is Bob.", ctx=ctx)
response = await agent.run("What was my name again?", ctx=ctx)
```

You'll notice that agents in LlamaIndex are async because they use Python's await operator. If you are new to async code in Python, or need a refresher, they have an excellent async guide (https://docs.llamaindex.ai/en/stable/getting_started/async_python/).

Now we've gotten the basics, let's take a look at how we can use more complex tools in our agents.

Creating RAG Agents with QueryEngineTools

Agentic RAG is a powerful way to use agents to answer questions about your data. We can pass various tools to Alfred to help him answer questions. However, instead of answering the question on top of documents automatically, Alfred can decide to use any other tool or flow to answer the question.

[Image omitted from text-first adaptation: Agentic RAG]

It is easy to wrap QueryEngine as a tool for an agent. When doing so, we need to define a name and description. The LLM will use this information to correctly use the tool. Let's see how to load in a QueryEngineTool using the QueryEngine we created in the component section (components).

```
from llama_index.core.tools import QueryEngineTool

query_engine = index.as_query_engine(llm=llm, similarity_top_k=3) # as shown in the Components in LlamaIn \
dex section

query_engine_tool = QueryEngineTool.from_defaults(
    query_engine=query_engine,
    name="name",
    description="a specific description",
    return_direct=False,
)

query_engine_agent = AgentWorkflow.from_tools_or_functions(
    [query_engine_tool],
    llm=llm,
    system_prompt="You are a helpful assistant that has access to a database containing persona descripti \
ons. "
)
```

Creating Multi-agent systems

The AgentWorkflow class also directly supports multi-agent systems. By giving each agent a name and description, the system maintains a single active speaker, with each agent having the ability to hand off to another agent.

By narrowing the scope of each agent, we can help increase their general accuracy when responding to user messages.

Agents in LlamaIndex can also directly be used as tools for other agents, for more complex and custom scenarios.

```
from llama_index.core.agent.workflow import (
    AgentWorkflow,
    FunctionAgent,
    ReActAgent,
)

# Define some tools
def add(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b

def subtract(a: int, b: int) -> int:
    """Subtract two numbers."""
    return a - b

# Create agent configs
# NOTE: we can use FunctionAgent or ReActAgent here.
# FunctionAgent works for LLMs with a function calling API.
# ReActAgent works for any LLM.
calculator_agent = ReActAgent(
    name="calculator",
    description="Performs basic arithmetic operations",
    system_prompt="You are a calculator assistant. Use your tools for any math operation.",
    tools=[add, subtract],
    llm=llm,
```

```

)

query_agent = ReActAgent(
    name="info_lookup",
    description="Looks up information about XYZ",
    system_prompt="Use your tool to query a RAG system to answer information about XYZ",
    tools=[query_engine_tool],
    llm=llm
)

# Create and run the workflow
agent = AgentWorkflow(
    agents=[calculator_agent, query_agent], root_agent="calculator"
)

# Run the system
response = await agent.run(user_msg="Can you add 5 and 3?")

```

Haven't learned enough yet? There is a lot more to discover about agents and tools in LlamaIndex within the AgentWorkflow Basic Introduction or the Agent Learning Guide, where you can read more about streaming, context serialization, and human-in-the-loop!

Now that we understand the basics of agents and tools in LlamaIndex, let's see how we can use LlamaIndex to create configurable and manageable workflows!

Creating Agentic Workflows in LlamaIndex

Source file: [units/en/unit2/llama-index/workflows.mdx](#)

Creating agentic workflows in LlamaIndex

A workflow in LlamaIndex provides a structured way to organize your code into sequential and manageable steps.

Such a workflow is created by defining Steps which are triggered by Events, and themselves emit Events to trigger further steps. Let's take a look at Alfred showing a LlamaIndex workflow for a RAG task.

[Image omitted from text-first adaptation: Workflow Schematic]

Workflows offer several key benefits:

- Clear organization of code into discrete steps
- Event-driven architecture for flexible control flow
- Type-safe communication between steps
- Built-in state management
- Support for both simple and complex agent interactions

As you might have guessed, workflows strike a great balance between the autonomy of agents while maintaining control over the overall workflow.

So, let's learn how to create a workflow ourselves!

Creating Workflows

You can follow the code in this notebook that you can run using Google Colab.

Basic Workflow Creation

Install the Workflow package As introduced in the section on the LlamaHub, we can install the Workflow package with the following command:

```
pip install llama-index-utils-workflow
```

We can create a single-step workflow by defining a class that inherits from Workflow and decorating your functions with `@step`. We will also need to add `StartEvent` and `StopEvent`, which are special events that are used to indicate the start and end of the workflow.

```
from llama_index.core.workflow import StartEvent, StopEvent, Workflow, step

class MyWorkflow(Workflow):
    @step
    async def my_step(self, ev: StartEvent) -> StopEvent:
        # do something here
        return StopEvent(result="Hello, world!")

w = MyWorkflow(timeout=10, verbose=False)
result = await w.run()
```

As you can see, we can now run the workflow by calling `w.run()`.

Connecting Multiple Steps

To connect multiple steps, we create custom events that carry data between steps. To do so, we need to add an Event that is passed between the steps and transfers the output of the first step to the second step.

```
from llama_index.core.workflow import Event

class ProcessingEvent(Event):
    intermediate_result: str

class MultiStepWorkflow(Workflow):
    @step
    async def step_one(self, ev: StartEvent) -> ProcessingEvent:
        # Process initial data
        return ProcessingEvent(intermediate_result="Step 1 complete")

    @step
    async def step_two(self, ev: ProcessingEvent) -> StopEvent:
        # Use the intermediate result
        final_result = f"Finished processing: {ev.intermediate_result}"
        return StopEvent(result=final_result)

w = MultiStepWorkflow(timeout=10, verbose=False)
result = await w.run()
result
```

The type hinting is important here, as it ensures that the workflow is executed correctly. Let's complicate things a bit more!

Loops and Branches

The type hinting is the most powerful part of workflows because it allows us to create branches, loops, and joins to facilitate more complex workflows.

Let's show an example of creating a loop by using the union operator `|`. In the example below, we see that the `LoopEvent` is taken as input for the step and can also be returned as output.

```
from llama_index.core.workflow import Event
import random

class ProcessingEvent(Event):
    intermediate_result: str

class LoopEvent(Event):
    loop_output: str

class MultiStepWorkflow(Workflow):
    @step
    async def step_one(self, ev: StartEvent | LoopEvent) -> ProcessingEvent | LoopEvent:
        if random.randint(0, 1) == 0:
            print("Bad thing happened")
            return LoopEvent(loop_output="Back to step one.")
        else:
            print("Good thing happened")
            return ProcessingEvent(intermediate_result="First step complete.")

    @step
    async def step_two(self, ev: ProcessingEvent) -> StopEvent:
        # Use the intermediate result
        final_result = f"Finished processing: {ev.intermediate_result}"
        return StopEvent(result=final_result)

w = MultiStepWorkflow(verbose=False)
```

```
result = await w.run()
result
```

Drawing Workflows

We can also draw workflows. Let's use the `draw_all_possible_flows` function to draw the workflow. This stores the workflow in an HTML file.

```
from llama_index.utils.workflow import draw_all_possible_flows

w = ... # as defined in the previous section
draw_all_possible_flows(w, "flow.html")
```

[Image omitted from text-first adaptation: workflow drawing]

There is one last cool trick that we will cover in the course, which is the ability to add state to the workflow.

State Management

State management is useful when you want to keep track of the state of the workflow, so that every step has access to the same state. We can do this by using the Context type hint on top of a parameter in the step function.

```
from llama_index.core.workflow import Context, StartEvent, StopEvent

@step
async def query(self, ctx: Context, ev: StartEvent) -> StopEvent:
    # store query in the context
    await ctx.store.set("query", "What is the capital of France?")

    # do something with context and event
    val = ...

    # retrieve query from the context
    query = await ctx.store.get("query")

    return StopEvent(result=val)
```

Great! Now you know how to create basic workflows in LlamaIndex!

There are some more complex nuances to workflows, which you can learn about in the LlamaIndex documentation.

However, there is another way to create workflows, which relies on the `AgentWorkflow` class. Let's take a look at how we can use this to create a multi-agent workflow.

Automating workflows with Multi-Agent Workflows

Instead of manual workflow creation, we can use the `AgentWorkflow` class to create a multi-agent workflow. The `AgentWorkflow` uses `Workflow Agents` to allow you to create a system of one or more agents that can collaborate and hand off tasks to each other based on their specialized capabilities. This enables building complex agent systems where different agents handle different aspects of a task. Instead of importing classes from `llama_index.core.agent`, we will import the agent classes from `llama_index.core.agent.workflow`. One agent must be designated as the root agent in the `AgentWorkflow` constructor. When a user message comes in, it is first routed to the root agent.

Each agent can then:

- Handle the request directly using their tools
- Handoff to another agent better suited for the task
- Return a response to the user

Let's see how to create a multi-agent workflow.

```
from llama_index.core.agent.workflow import AgentWorkflow, ReActAgent
from llama_index.llms.huggingface_api import HuggingFaceInferenceAPI

# Define some tools
def add(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b

def multiply(a: int, b: int) -> int:
```



```

    """Multiply two numbers."""
    return a * b

llm = HuggingFaceInferenceAPI(model_name="Qwen/Qwen2.5-Coder-32B-Instruct")

# we can pass functions directly without FunctionTool -- the fn/docstring are parsed for the name/description
multiply_agent = ReActAgent(
    name="multiply_agent",
    description="Is able to multiply two integers",
    system_prompt="A helpful assistant that can use a tool to multiply numbers.",
    tools=[multiply],
    llm=llm,
)

addition_agent = ReActAgent(
    name="add_agent",
    description="Is able to add two integers",
    system_prompt="A helpful assistant that can use a tool to add numbers.",
    tools=[add],
    llm=llm,
)

# Create the workflow
workflow = AgentWorkflow(
    agents=[multiply_agent, addition_agent],
    root_agent="multiply_agent",
)

# Run the system
response = await workflow.run(user_msg="Can you add 5 and 3?")

```

Agent tools can also modify the workflow state we mentioned earlier. Before starting the workflow, we can provide an initial state dict that will be available to all agents. The state is stored in the state key of the workflow context. It will be injected into the state_prompt which augments each new user message.

Let's inject a counter to count function calls by modifying the previous example:

```

from llama_index.core.workflow import Context

# Define some tools
async def add(ctx: Context, a: int, b: int) -> int:
    """Add two numbers."""
    # update our count
    cur_state = await ctx.store.get("state")
    cur_state["num_fn_calls"] += 1
    await ctx.store.set("state", cur_state)

    return a + b

async def multiply(ctx: Context, a: int, b: int) -> int:
    """Multiply two numbers."""
    # update our count
    cur_state = await ctx.store.get("state")
    cur_state["num_fn_calls"] += 1
    await ctx.store.set("state", cur_state)

    return a * b

...

workflow = AgentWorkflow(
    agents=[multiply_agent, addition_agent],
    root_agent="multiply_agent",
    initial_state={"num_fn_calls": 0},
    state_prompt="Current state: {state}. User message: {msg}",
)

# run the workflow with context
ctx = Context(workflow)
response = await workflow.run(user_msg="Can you add 5 and 3?", ctx=ctx)

# pull out and inspect the state
state = await ctx.store.get("state")
print(state["num_fn_calls"])

```

Congratulations! You have now mastered the basics of Agents in LlamaIndex!

Let's continue with one final quiz to solidify your knowledge!

Quick Quiz 2

Source file: [units/en/unit2/llama-index/quiz2.mdx](#)

Quick Self-Check (ungraded)

What?! Another Quiz? We know, we know, ... But this short, ungraded quiz is here to help you reinforce key concepts you've just learned.

This quiz covers agent workflows and interactions - essential components for building effective AI agents.

Q1: What is the purpose of AgentWorkflow in LlamaIndex?

- Answer choice: To run one or more agents with tools

Explanation: Yes, the AgentWorkflow is the main way to quickly create a system with one or more agents.

Correct answer.

},

- Answer choice: To create a single agent that can query your data without memory

Explanation: No, the AgentWorkflow is more capable than that, the QueryEngine is for simple queries over your data.

},

- Answer choice: To automatically build tools for agents

Explanation: The AgentWorkflow does not build tools, that is the job of the developer.

},

- Answer choice: To manage agent memory and state

Explanation: Managing memory and state is not the primary purpose of AgentWorkflow.

Q2: What object is used for keeping track of the state of the workflow?

- Answer choice: State

Explanation: State is not the correct object for workflow state management.

},

- Answer choice: Context

Explanation: Context is the correct object used for keeping track of workflow state.

Correct answer.

},

- Answer choice: WorkflowState

Explanation: WorkflowState is not the correct object.

},

- Answer choice: Management

Explanation: Management is not a valid object for workflow state.

Q3: Which method should be used if you want an agent to remember previous interactions?

- Answer choice: run(query_str)

Explanation: .run(query_str) does not maintain conversation history.

},

- Answer choice: chat(query_str, ctx=ctx)

Explanation: chat() is not a valid method on workflows.

},

- Answer choice: interact(query_str)

Explanation: interact() is not a valid method for agent interactions.

},

- Answer choice: run(query_str, ctx=ctx)

Explanation: By passing in and maintaining the context, we can maintain state!

Correct answer.

Q4: What is a key feature of Agentic RAG?

- Answer choice: It can only use document-based tools, to answer questions in a RAG workflow

Explanation: Agentic RAG can use different tools, including document-based tools.

},

- Answer choice: It automatically answers questions without tools, like a chatbot

Explanation: Agentic RAG does use tools to answer questions.

},

- Answer choice: It can decide to use any tool to answer questions, including RAG tools

Explanation: Agentic RAG has the flexibility to use different tools to answer questions.

Correct answer.

},

- Answer choice: It only works with Function Calling Agents

Explanation: Agentic RAG is not limited to Function Calling Agents.

Got it? Great! Now let's do a brief recap of the unit!

Conclusion

Source file: [units/en/unit2/llama-index/conclusion.mdx](#)

Conclusion

Congratulations on finishing the llama-index module of this second Unit

You've just mastered the fundamentals of llama-index and you've seen how to build your own agentic workflows! Now that you have skills in llama-index, you can start to create search engines that will solve tasks you're interested in.

In the next module of the unit, you're going to learn how to build Agents with LangGraph.

Finally, we would love to hear what you think of the course and how we can improve it. If you have some feedback then, please fill this form

(<https://docs.google.com/forms/d/e/1FAIpQLSe9VaONn0eglax0uTwI29rIn4tM7H2sYmmybmG5jJNIE5v0xA/viewform?usp=dialog>)

Keep Learning, and stay awesome

Unit 2.3 The LangGraph framework

Instructional sections below follow the official English course order.

Introduction to LangGraph

Source file: [units/en/unit2/langgraph/introduction.mdx](https://github.com/langchain-ai/langgraph/blob/main/docs/en/introduction.md)

Introduction to LangGraph

[Image omitted from text-first adaptation: Unit 2.3 Thumbnail]

Welcome to this next part of our journey, where you'll learn how to build applications using the LangGraph (<https://github.com/langchain-ai/langgraph>) framework designed to help you structure and orchestrate complex LLM workflows.

LangGraph is a framework that allows you to build production-ready applications by giving you control tools over the flow of your agent.

Module Overview

In this unit, you'll discover:

1 What is LangGraph, and when to use it? (./when_to_use_langgraph)

2 Building Blocks of LangGraph (./building_blocks)

3 Alfred, the mail sorting butler (./first_graph)

4 Alfred, the document Analyst agent (./document_analysis_agent)

5 Quiz (./quizz1)

The examples in this section require access to a powerful LLM/VLM model. We ran them using the GPT-4o API because it has the best compatibility with LangGraph.

By the end of this unit, you'll be equipped to build robust, organized and production ready applications !

That being said, this section is an introduction to LangGraph and more advanced topics can be discovered in the free LangChain academy course : Introduction to LangGraph (<https://academy.langchain.com/courses/intro-to-langgraph>)

Let's get started!

Resources

- LangGraph Agents (<https://langchain-ai.github.io/langgraph/>) - Examples of LangGraph agent
- LangChain academy (<https://academy.langchain.com/courses/intro-to-langgraph>) - Full course on LangGraph from LangChain

What is LangGraph?

Source file: [units/en/unit2/langgraph/when_to_use_langgraph.mdx](https://github.com/langchain-ai/langgraph/blob/main/docs/en/when_to_use_langgraph.md)

What is LangGraph?

LangGraph is a framework developed by LangChain (<https://www.langchain.com/>) to manage the control flow of applications that integrate an LLM.

Is LangGraph different from LangChain?

LangChain provides a standard interface to interact with models and other components, useful for retrieval, LLM calls and tools calls. The classes from LangChain might be used in LangGraph, but do not HAVE to be used.

The packages are different and can be used in isolation, but, in the end, all resources you will find online use both packages hand in hand.

When should I use LangGraph?

Control vs freedom

When designing AI applications, you face a fundamental trade-off between control and freedom:

- Freedom gives your LLM more room to be creative and tackle unexpected problems.
- Control allows you to ensure predictable behavior and maintain guardrails.

Code Agents, like the ones you can encounter in smolagents, are very free. They can call multiple tools in a single action step, create their own tools, etc. However, this behavior can make them less predictable and less controllable than a regular Agent working with JSON!

LangGraph is on the other end of the spectrum, it shines when you need "Control" on the execution of your agent.

LangGraph is particularly valuable when you need Control over your applications. It gives you the tools to build an application that follows a predictable process while still leveraging the power of LLMs.

Put simply, if your application involves a series of steps that need to be orchestrated in a specific way, with decisions being made at each junction point, LangGraph provides the structure you need.

As an example, let's say we want to build an LLM assistant that can answer some questions over some documents.

Since LLMs understand text the best, before being able to answer the question, you will need to convert other complex modalities (charts, tables) into text. However, that choice depends on the type of document you have!

This is a branching that I chose to represent as follow :

[Image omitted from text-first adaptation: Control flow]

Tip: The left part is not an agent, as here no tool call is involved. but the right part will need to write some code to query the xls (convert to pandas and manipulate it).

While this branching is deterministic, you can also design branching that are conditioned on the output of an LLM making them undeterministic.

The key scenarios where LangGraph excels include:

- Multi-step reasoning processes that need explicit control on the flow
- Applications requiring persistence of state between steps
- Systems that combine deterministic logic with AI capabilities
- Workflows that need human-in-the-loop interventions
- Complex agent architectures with multiple components working together

In essence, whenever possible, as a human, design a flow of actions based on the output of each action, and decide what to execute next accordingly. In this case, LangGraph is the correct framework for you!

LangGraph is, in my opinion, the most production-ready agent framework on the market.

How does LangGraph work?

At its core, LangGraph uses a directed graph structure to define the flow of your application:

- Nodes represent individual processing steps (like calling an LLM, using a tool, or making a decision).
- Edges define the possible transitions between steps.
- State is user defined and maintained and passed between nodes during execution. When deciding which node to target next, this is the current state that we look at.

We will explore those fundamental blocks more in the next chapter!

How is it different from regular python? Why do I need LangGraph?

You might wonder: "I could just write regular Python code with if-else statements to handle all these flows, right?"

While technically true, LangGraph offers some advantages over vanilla Python for building complex systems. You could build the same application without LangGraph, but it builds easier tools and abstractions for you.

It includes states, visualization, logging (traces), built-in human-in-the-loop, and more.

Building Blocks of LangGraph

Source file: [units/en/unit2/langgraph/building_blocks.mdx](#)

Building Blocks of LangGraph

To build applications with LangGraph, you need to understand its core components. Let's explore the fundamental building blocks that make up a LangGraph application.

[Image omitted from text-first adaptation: Building Blocks]

An application in LangGraph starts from an entrypoint, and depending on the execution, the flow may go to one function or another until it reaches the END.

[Image omitted from text-first adaptation: Application]

1. State

State is the central concept in LangGraph. It represents all the information that flows through your application.

```
from typing_extensions import TypedDict
```

```
class State(TypedDict):
    graph_state: str
```

The state is User defined, hence the fields should carefully be crafted to contain all data needed for decision-making process!

Tip: Think carefully about what information your application needs to track between steps.

2. Nodes

Nodes are python functions. Each node:

- Takes the state as input
- Performs some operation
- Returns updates to the state

```
def node_1(state):
    print("---Node 1---")
    return {"graph_state": state['graph_state'] + " I am"}

def node_2(state):
    print("---Node 2---")
    return {"graph_state": state['graph_state'] + " happy!"}

def node_3(state):
    print("---Node 3---")
    return {"graph_state": state['graph_state'] + " sad!"}
```

For example, Nodes can contain:

- LLM calls: Generate text or make decisions
- Tool calls: Interact with external systems
- Conditional logic: Determine next steps

- Human intervention: Get input from users

Info: Some nodes necessary for the whole workflow like START and END exist from LangGraph directly.

3. Edges

Edges connect nodes and define the possible paths through your graph:

```
import random
from typing import Literal

def decide_mood(state) -> Literal["node_2", "node_3"]:

    # Often, we will use state to decide on the next node to visit
    user_input = state['graph_state']

    # Here, let's just do a 50 / 50 split between nodes 2, 3
    if random.random() < 0.5:

        # 50% of the time, we return Node 2
        return "node_2"

    # 50% of the time, we return Node 3
    return "node_3"
```

Edges can be:

- Direct: Always go from node A to node B
- Conditional: Choose the next node based on the current state

4. StateGraph

The StateGraph is the container that holds your entire agent workflow:

```
from IPython.display import Image, display
from langgraph.graph import StateGraph, START, END

# Build graph
builder = StateGraph(State)
builder.add_node("node_1", node_1)
builder.add_node("node_2", node_2)
builder.add_node("node_3", node_3)

# Logic
builder.add_edge(START, "node_1")
builder.add_conditional_edges("node_1", decide_mood)
builder.add_edge("node_2", END)
builder.add_edge("node_3", END)

# Add
graph = builder.compile()
```

Which can then be visualized!

```
# View
display(Image(graph.get_graph().draw_mermaid_png()))
```

[Image omitted from text-first adaptation: Graph Visualization]

But most importantly, invoked:

```
graph.invoke({"graph_state" : "Hi, this is Lance."})
```

output :

```
---Node 1---
---Node 3---
{'graph_state': 'Hi, this is Lance. I am sad!'}
```

What's Next?

In the next section, we'll put these concepts into practice by building our first graph. This graph lets Alfred take in your e-mails, classify them, and craft a preliminary answer if they are genuine.

Building Your First LangGraph

Source file: [units/en/unit2/langgraph/first_graph.mdx](#)

Building Your First LangGraph

Now that we understand the building blocks, let's put them into practice by building our first functional graph. We'll implement Alfred's email processing system, where he needs to:

1. Read incoming emails
1. Classify them as spam or legitimate
1. Draft a preliminary response for legitimate emails
1. Send information to Mr. Wayne when legitimate (printing only)

This example demonstrates how to structure a workflow with LangGraph that involves LLM-based decision-making. While this can't be considered an Agent as no tool is involved, this section focuses more on learning the LangGraph framework than Agents.

You can follow the code in this notebook that you can run using Google Colab.

Our Workflow

Here's the workflow we'll build:

[Image omitted from text-first adaptation: First LangGraph]

Setting Up Our Environment

First, let's install the required packages:

```
%pip install langgraph langchain_openai
```

Next, let's import the necessary modules:

```
import os
from typing import TypedDict, List, Dict, Any, Optional
from langgraph.graph import StateGraph, START, END
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage
```

Step 1: Define Our State

Let's define what information Alfred needs to track during the email processing workflow:

```
class EmailState(TypedDict):
    # The email being processed
    email: Dict[str, Any] # Contains subject, sender, body, etc.

    # Category of the email (inquiry, complaint, etc.)
    email_category: Optional[str]

    # Reason why the email was marked as spam
    spam_reason: Optional[str]

    # Analysis and decisions
    is_spam: Optional[bool]

    # Response generation
    email_draft: Optional[str]

    # Processing metadata
    messages: List[Dict[str, Any]] # Track conversation with LLM for analysis
```

Tip: Make your state comprehensive enough to track all the important information, but avoid bloating it with unnecessary details.

Step 2: Define Our Nodes

Now, let's create the processing functions that will form our nodes:

```
# Initialize our LLM
```



```

model = ChatOpenAI(temperature=0)

def read_email(state: EmailState):
    """Alfred reads and logs the incoming email"""
    email = state["email"]

    # Here we might do some initial preprocessing
    print(f"Alfred is processing an email from {email['sender']} with subject: {email['subject']}")

    # No state changes needed here
    return {}

def classify_email(state: EmailState):
    """Alfred uses an LLM to determine if the email is spam or legitimate"""
    email = state["email"]

    # Prepare our prompt for the LLM
    prompt = f"""
    As Alfred the butler, analyze this email and determine if it is spam or legitimate.

    Email:
    From: {email['sender']}
    Subject: {email['subject']}
    Body: {email['body']}

    First, determine if this email is spam. If it is spam, explain why.
    If it is legitimate, categorize it (inquiry, complaint, thank you, etc.).
    """

    # Call the LLM
    messages = [HumanMessage(content=prompt)]
    response = model.invoke(messages)

    # Simple logic to parse the response (in a real app, you'd want more robust parsing)
    response_text = response.content.lower()
    is_spam = "spam" in response_text and "not spam" not in response_text

    # Extract a reason if it's spam
    spam_reason = None
    if is_spam and "reason:" in response_text:
        spam_reason = response_text.split("reason:")[1].strip()

    # Determine category if legitimate
    email_category = None
    if not is_spam:
        categories = ["inquiry", "complaint", "thank you", "request", "information"]
        for category in categories:
            if category in response_text:
                email_category = category
                break

    # Update messages for tracking
    new_messages = state.get("messages", []) + [
        {"role": "user", "content": prompt},
        {"role": "assistant", "content": response.content}
    ]

    # Return state updates
    return {
        "is_spam": is_spam,
        "spam_reason": spam_reason,
        "email_category": email_category,
        "messages": new_messages
    }

def handle_spam(state: EmailState):
    """Alfred discards spam email with a note"""
    print(f"Alfred has marked the email as spam. Reason: {state['spam_reason']}")
    print("The email has been moved to the spam folder.")

    # We're done processing this email
    return {}

def draft_response(state: EmailState):
    """Alfred drafts a preliminary response for legitimate emails"""
    email = state["email"]
    category = state["email_category"] or "general"

    # Prepare our prompt for the LLM
    prompt = f"""
    As Alfred the butler, draft a polite preliminary response to this email.
  
```

```

Email:
From: {email['sender']}
Subject: {email['subject']}
Body: {email['body']}

This email has been categorized as: {category}

Draft a brief, professional response that Mr. Hugg can review and personalize before sending.
"""

# Call the LLM
messages = [HumanMessage(content=prompt)]
response = model.invoke(messages)

# Update messages for tracking
new_messages = state.get("messages", []) + [
    {"role": "user", "content": prompt},
    {"role": "assistant", "content": response.content}
]

# Return state updates
return {
    "email_draft": response.content,
    "messages": new_messages
}

def notify_mr_hugg(state: EmailState):
    """Alfred notifies Mr. Hugg about the email and presents the draft response"""
    email = state["email"]

    print("\n" + "="*50)
    print(f"Sir, you've received an email from {email['sender']}.")
    print(f"Subject: {email['subject']}")
    print(f"Category: {state['email_category']}")
    print("\nI've prepared a draft response for your review:")
    print("-"*50)
    print(state["email_draft"])
    print("="*50 + "\n")

    # We're done processing this email
    return {}

```

Step 3: Define Our Routing Logic

We need a function to determine which path to take after classification:

```

def route_email(state: EmailState) -> str:
    """Determine the next step based on spam classification"""
    if state["is_spam"]:
        return "spam"
    else:
        return "legitimate"

```

Note: This routing function is called by LangGraph to determine which edge to follow after the classification node. The return value must match one of the keys in our conditional edges mapping.

Step 4: Create the StateGraph and Define Edges

Now we connect everything together:

```

# Create the graph
email_graph = StateGraph(EmailState)

# Add nodes
email_graph.add_node("read_email", read_email)
email_graph.add_node("classify_email", classify_email)
email_graph.add_node("handle_spam", handle_spam)
email_graph.add_node("draft_response", draft_response)
email_graph.add_node("notify_mr_hugg", notify_mr_hugg)

# Start the edges
email_graph.add_edge(START, "read_email")
# Add edges - defining the flow
email_graph.add_edge("read_email", "classify_email")

# Add conditional branching from classify_email
email_graph.add_conditional_edges(
    "classify_email",
    route_email,

```

```

    {
        "spam": "handle_spam",
        "legitimate": "draft_response"
    }
)

# Add the final edges
email_graph.add_edge("handle_spam", END)
email_graph.add_edge("draft_response", "notify_mr_hugg")
email_graph.add_edge("notify_mr_hugg", END)

# Compile the graph
compiled_graph = email_graph.compile()

```

Notice how we use the special END node provided by LangGraph. This indicates terminal states where the workflow completes.

Step 5: Run the Application

Let's test our graph with a legitimate email and a spam email:

```

# Example legitimate email
legitimate_email = {
    "sender": "john.smith@example.com",
    "subject": "Question about your services",
    "body": "Dear Mr. Hugg, I was referred to you by a colleague and I'm interested in learning more about \
t your consulting services. Could we schedule a call next week? Best regards, John Smith"
}

# Example spam email
spam_email = {
    "sender": "winner@lottery-intl.com",
    "subject": "YOU HAVE WON $5,000,000!!!",
    "body": "CONGRATULATIONS! You have been selected as the winner of our international lottery! To claim \
your $5,000,000 prize, please send us your bank details and a processing fee of $100."
}

# Process the legitimate email
print("\nProcessing legitimate email...")
legitimate_result = compiled_graph.invoke({
    "email": legitimate_email,
    "is_spam": None,
    "spam_reason": None,
    "email_category": None,
    "email_draft": None,
    "messages": []
})

# Process the spam email
print("\nProcessing spam email...")
spam_result = compiled_graph.invoke({
    "email": spam_email,
    "is_spam": None,
    "spam_reason": None,
    "email_category": None,
    "email_draft": None,
    "messages": []
})

```

Step 6: Inspecting Our Mail Sorting Agent with Langfuse

As Alfred fine-tunes the Mail Sorting Agent, he's growing weary of debugging its runs. Agents, by nature, are unpredictable and difficult to inspect. But since he aims to build the ultimate Spam Detection Agent and deploy it in production, he needs robust traceability for future monitoring and analysis.

To do this, Alfred can use an observability tool such as Langfuse (<https://langfuse.com/>) to trace and monitor the agent.

First, we pip install Langfuse:

```
%pip install -q langfuse
```

Second, we pip install Langchain (LangChain is required because we use LangFuse):

```
%pip install langchain
```

Next, we add the Langfuse API keys and host address as environment variables. You can get your Langfuse credentials by signing up for Langfuse Cloud (<https://cloud.langfuse.com>) or self-host Langfuse (<https://langfuse.com/self-hosting>).

```
import os
```

```
# Get keys for your project from the project settings page: https://cloud.langfuse.com
os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
os.environ["LANGFUSE_HOST"] = "https://cloud.langfuse.com" # EU region
# os.environ["LANGFUSE_HOST"] = "https://us.cloud.langfuse.com" # US region
```

Then, we configure the Langfuse callback_handler

(<https://langfuse.com/docs/integrations/langchain/tracing#add-langfuse-to-your-langchain-application>) and instrument the agent by adding the langfuse_callback to the invocation of the graph: config={"callbacks": [langfuse_handler]}.

```
from langfuse.langchain import CallbackHandler

# Initialize Langfuse CallbackHandler for LangGraph/Langchain (tracing)
langfuse_handler = CallbackHandler()

# Process legitimate email
legitimate_result = compiled_graph.invoke(
    input={"email": legitimate_email, "is_spam": None, "spam_reason": None, "email_category": None, "draft_response": None, "messages": []},
    config={"callbacks": [langfuse_handler]}
)
```

Alfred is now connected ! The runs from LangGraph are being logged in Langfuse, giving him full visibility into the agent's behavior. With this setup, he's ready to revisit previous runs and refine his Mail Sorting Agent even further.

[Image omitted from text-first adaptation: Example trace in Langfuse]

Public link to the trace with the legit email (<https://cloud.langfuse.com/project/cloramnkj0002jz088vzn1ja4/traces/f5d6d72e-20af-4357-b232-af44c3728a7b?timestamp=2025-03-17T10%3A13%3A28.413Z&observation=6997ba69-043f-4f77-9445-700a033afba1>)

Visualizing Our Graph

LangGraph allows us to visualize our workflow to better understand and debug its structure:

```
compiled_graph.get_graph().draw_mermaid_png()
```

[Image omitted from text-first adaptation: Mail LangGraph]

This produces a visual representation showing how our nodes are connected and the conditional paths that can be taken.

What We've Built

We've created a complete email processing workflow that:

1. Takes an incoming email
1. Uses an LLM to classify it as spam or legitimate
1. Handles spam by discarding it
1. For legitimate emails, drafts a response and notifies Mr. Hugg

This demonstrates the power of LangGraph to orchestrate complex workflows with LLMs while maintaining a clear, structured flow.

Key Takeaways

- State Management: We defined comprehensive state to track all aspects of email processing
- Node Implementation: We created functional nodes that interact with an LLM
- Conditional Routing: We implemented branching logic based on email classification
- Terminal States: We used the END node to mark completion points in our workflow

What's Next?

In the next section, we'll explore more advanced features of LangGraph, including handling human interaction in the workflow and implementing more complex branching logic based on multiple conditions.

Document Analysis Graph

Source file: `units/en/unit2/langgraph/document_analysis_agent.mdx`

Document Analysis Graph

Alfred at your service. As Mr. Wayne's trusted butler, I've taken the liberty of documenting how I assist Mr Wayne with his various documentary needs. While he's out attending to his... nighttime activities, I ensure all his paperwork, training schedules, and nutritional plans are properly analyzed and organized.

Before leaving, he left a note with his week's training program. I then took the responsibility to come up with a menu for tomorrow's meals.

For future such events, let's create a document analysis system using LangGraph to serve Mr. Wayne's needs. This system can:

1. Process images document
1. Extract text using vision models (Vision Language Model)
1. Perform calculations when needed (to demonstrate normal tools)
1. Analyze content and provide concise summaries
1. Execute specific instructions related to documents

The Butler's Workflow

The workflow we'll build follows this structured schema:

[Image omitted from text-first adaptation: Butler's Document Analysis Workflow]

You can follow the code in this notebook that you can run using Google Colab.

Setting Up the environment

```
%pip install langgraph langchain_openai langchain_core
```

and imports :

```
import base64
from typing import List, TypedDict, Annotated, Optional
from langchain_openai import ChatOpenAI
from langchain_core.messages import AnyMessage, SystemMessage, HumanMessage
from langgraph.graph.message import add_messages
from langgraph.graph import START, StateGraph
from langgraph.prebuilt import ToolNode, tools_condition
from IPython.display import Image, display
```

Defining Agent's State

This state is a little more complex than the previous ones we have seen. AnyMessage is a class from Langchain that defines messages, and add_messages is an operator that adds the latest message rather than overwriting it with the latest state.

This is a new concept in LangGraph, where you can add operators in your state to define the way they should interact together.

```
class AgentState(TypedDict):
    # The document provided
    input_file: Optional[str] # Contains file path (PDF/PNG)
    messages: Annotated[list[AnyMessage], add_messages]
```

Preparing Tools

```
vision_llm = ChatOpenAI(model="gpt-4o")

def extract_text(img_path: str) -> str:
    """
    Extract text from an image file using a multimodal model.

    Master Wayne often leaves notes with his training regimen or meal plans.
    This allows me to properly analyze the contents.
    """
    all_text = ""
    try:
        # Read image and encode as base64
        with open(img_path, "rb") as image_file:
            image_bytes = image_file.read()

        image_base64 = base64.b64encode(image_bytes).decode("utf-8")

        # Prepare the prompt including the base64 image data
        message = [
            HumanMessage(
                content=[
                    {
                        "type": "text",
                        "text": (
                            "Extract all the text from this image. "
                            "Return only the extracted text, no explanations."
                        ),
                    },
                    {
                        "type": "image_url",
                        "image_url": {
                            "url": f"data:image/png;base64,{image_base64}"
                        },
                    },
                ]
            )
        ]

        # Call the vision-capable model
        response = vision_llm.invoke(message)

        # Append extracted text
        all_text += response.content + "\n\n"

        return all_text.strip()
    except Exception as e:
        # A butler should handle errors gracefully
        error_msg = f"Error extracting text: {str(e)}"
        print(error_msg)
        return ""

def divide(a: int, b: int) -> float:
    """Divide a and b - for Master Wayne's occasional calculations."""
    return a / b

# Equip the butler with tools
tools = [
    divide,
    extract_text
]

llm = ChatOpenAI(model="gpt-4o")
llm_with_tools = llm.bind_tools(tools, parallel_tool_calls=False)
```

The nodes

```
def assistant(state: AgentState):
    # System message
    textual_description_of_tool=""
    extract_text(img_path: str) -> str:
        Extract text from an image file using a multimodal model.

    Args:
        img_path: A local image file path (strings).

    Returns:
        A single string containing the concatenated text extracted from each image.
divide(a: int, b: int) -> float:
    Divide a and b
"""
    image=state["input_file"]
    sys_msg = SystemMessage(content=f"You are a helpful butler named Alfred that serves Mr. Wayne and Bat \
man. You can analyse documents and run computations with provided tools:\n{textual_description_of_too \
l} \n You have access to some optional images. Currently the loaded image is: {image}")

    return {
        "messages": [llm_with_tools.invoke([sys_msg] + state["messages"])],
        "input_file": state["input_file"]
    }
```

The ReAct Pattern: How I Assist Mr. Wayne

Allow me to explain the approach in this agent. The agent follows what's known as the ReAct pattern (Reason-Act-Observe)

1. Reason about his documents and requests
1. Act by using appropriate tools
1. Observe the results
1. Repeat as necessary until I've fully addressed his needs

This is a simple implementation of an agent using LangGraph.

```
# The graph
builder = StateGraph(AgentState)

# Define nodes: these do the work
builder.add_node("assistant", assistant)
builder.add_node("tools", ToolNode(tools))

# Define edges: these determine how the control flow moves
builder.add_edge(START, "assistant")
builder.add_conditional_edges(
    "assistant",
    # If the latest message requires a tool, route to tools
    # Otherwise, provide a direct response
    tools_condition,
)
builder.add_edge("tools", "assistant")
react_graph = builder.compile()

# Show the butler's thought process
display(Image(react_graph.get_graph(xray=True).draw_mermaid_png()))
```

We define a tools node with our list of tools. The assistant node is just our model with bound tools. We create a graph with assistant and tools nodes.

We add a tools_condition edge, which routes to End or to tools based on whether the assistant calls a tool.

Now, we add one new step:

We connect the tools node back to the assistant, forming a loop.

- After the assistant node executes, tools_condition checks if the model's output is a tool call.
- If it is a tool call, the flow is directed to the tools node.
- The tools node connects back to assistant.

- This loop continues as long as the model decides to call tools.
- If the model response is not a tool call, the flow is directed to END, terminating the process.

[Image omitted from text-first adaptation: ReAct Pattern]

The Butler in Action

Example 1: Simple Calculations

Here is an example to show a simple use case of an agent using a tool in LangGraph.

```
messages = [HumanMessage(content="Divide 6790 by 5")]
messages = react_graph.invoke({"messages": messages, "input_file": None})

# Show the messages
for m in messages['messages']:
    m.pretty_print()
```

The conversation would proceed:

```
Human: Divide 6790 by 5

AI Tool Call: divide(a=6790, b=5)

Tool Response: 1358.0

Alfred: The result of dividing 6790 by 5 is 1358.0.
```

Example 2: Analyzing Master Wayne's Training Documents

When Master Wayne leaves his training and meal notes:

```
messages = [HumanMessage(content="According to the note provided by Mr. Wayne in the provided images. What's the list of items I should buy for the dinner menu?")]
messages = react_graph.invoke({"messages": messages, "input_file": "Batman_training_and_meals.png"})
```

The interaction would proceed:

```
Human: According to the note provided by Mr. Wayne in the provided images. What's the list of items I should buy for the dinner menu?

AI Tool Call: extract_text(img_path="Batman_training_and_meals.png")

Tool Response: [Extracted text with training schedule and menu details]

Alfred: For the dinner menu, you should buy the following items:

1. Grass-fed local sirloin steak
2. Organic spinach
3. Piquillo peppers
4. Potatoes (for oven-baked golden herb potato)
5. Fish oil (2 grams)

Ensure the steak is grass-fed and the spinach and peppers are organic for the best quality meal.
```

Key Takeaways

Should you wish to create your own document analysis butler, here are key considerations:

1. Define clear tools for specific document-related tasks
1. Create a robust state tracker to maintain context between tool calls
1. Consider error handling for tool failures
1. Maintain contextual awareness of previous interactions (ensured by the operator add_messages)

With these principles, you too can provide exemplary document analysis service worthy of Wayne Manor.

I trust this explanation has been satisfactory. Now, if you'll excuse me, Master Wayne's cape requires pressing before tonight's activities.

Quick Quiz 1

Source file: [units/en/unit2/langgraph/quiz1.mdx](#)

Test Your Understanding of LangGraph

Let's test your understanding of LangGraph with a quick quiz! This will help reinforce the key concepts we've covered so far.

This is an optional quiz and it's not graded.

Q1: What is the primary purpose of LangGraph?

Which statement best describes what LangGraph is designed for?

- Answer choice: A framework to build control flows for applications containing LLMs

Explanation: LangGraph is specifically designed to help build and manage the control flow of applications that use LLMs.

Correct answer.

},

- Answer choice: A library that provides interfaces to interact with different LLM models

Explanation: This better describes LangChain's role, which provides standard interfaces for model interaction. LangGraph focuses on control flow.

},

- Answer choice: An Agent library for tool calling

Explanation: While LangGraph works with agents, the main purpose of LangGraph is 'Orchestration'.

Q2: In the context of the "Control vs Freedom" trade-off, where does LangGraph stand?

Which statement best characterizes LangGraph's approach to agent design?

- Answer choice: LangGraph maximizes freedom, allowing LLMs to make all decisions independently

Explanation: LangGraph actually focuses more on control than freedom, providing structure for LLM workflows.

},

- Answer choice: LangGraph provides strong control over execution flow while still leveraging LLM capabilities for decision making

Explanation: LangGraph shines when you need control over your agent's execution, providing predictable behavior through structured workflows.

Correct answer.

},

Q3: What role does State play in LangGraph?

Choose the most accurate description of State in LangGraph.

- Answer choice: State is the latest generation from the LLM

Explanation: State is a user-defined class in LangGraph, not LLM generated. It's fields are user defined, the values can be LLM filled

},

- Answer choice: State is only used to track errors during execution

Explanation: State has a much broader purpose than just error tracking. But that's still usefull.

},

- Answer choice: State represents the information that flows through your agent application

Explanation: State is central to LangGraph and contains all the information needed for decision-making between steps. You provide the fields than you need to compute and the nodes can alter the values to decide on a branching.

Correct answer.

},

- Answer choice: State is only relevant when working with external APIs

Explanation: State is fundamental to all LangGraph applications, not just those working with external APIs.

Q4: What is a Conditional Edge in LangGraph?

Select the most accurate description.

- Answer choice: An edge that determines which node to execute next based on evaluating a condition

Explanation: Conditional edges allow your graph to make dynamic routing decisions based on the current state, creating branching logic in your workflow.

Correct answer.

},

- Answer choice: An edge that is only followed when a specific condition occurs

Explanation: Conditional edges control the flow of the application on it's outputs, not on the input.

},

- Answer choice: An edge that requires user confirmation before proceeding

Explanation: Conditional edges are based on programmatic conditions, not user interaction requirements.

Q5: How does LangGraph help address the hallucination problem in LLMs?

Choose the best answer.

- Answer choice: LangGraph eliminates hallucinations entirely by limiting LLM responses

Explanation: No framework can completely eliminate hallucinations from LLMs, LangGraph is no exception.

},

- Answer choice: LangGraph provides structured workflows that can validate and verify LLM outputs

Explanation: By creating structured workflows with validation steps, verification nodes, and error handling paths, LangGraph helps reduce the impact of hallucinations.

Correct answer.

},

- Answer choice: LangGraph has no effect on hallucinations

Explanation: LangGraph's structured approach to workflows can help significantly in mitigating hallucinations at the cost of speed.

Congratulations on completing the quiz! If you missed any questions, consider reviewing the previous sections to strengthen your understanding. Next, we'll explore more advanced features of LangGraph and see how to build more complex agent workflows.

Conclusion

Source file: [units/en/unit2/langgraph/conclusion.mdx](https://github.com/langchain-ai/langgraph/blob/main/docs/en/unit2/langgraph/conclusion.mdx)

Conclusion

Congratulations on finishing the LangGraph module of this second Unit!

You've now mastered the fundamentals of building structured workflows with LangGraph which you will be able to send to production.

This module is just the beginning of your journey with LangGraph. For more advanced topics, we recommend:

- Exploring the official LangGraph documentation (<https://github.com/langchain-ai/langgraph>)
- Taking the comprehensive Introduction to LangGraph (<https://academy.langchain.com/courses/intro-to-langgraph>) course from LangChain Academy
- Build something yourself !

In the next Unit, you'll now explore real use cases. It's time to leave theory to get into real action !

We would greatly appreciate your thoughts on the course and suggestions for improvement. If you have feedback, please fill this form

(<https://docs.google.com/forms/d/e/1FAIpQLSe9VaONn0eglax0uTwi29rln4tM7H2sYmmybmG5jJNIE5v0xA/viewform?usp=dialog>)

Keep Learning, Stay Awesome!

Good Sir/Madam!

-Alfred-

Unit 3. Use Case for Agentic RAG

Instructional sections below follow the official English course order.

Introduction to Use Case for Agentic RAG

Source file: [units/en/unit3/agentic-rag/introduction.mdx](#)

Introduction to Use Case for Agentic RAG

[Image omitted from text-first adaptation: Agentic RAG banner]

In this unit, we will help Alfred, our friendly agent who is hosting the gala, by using Agentic RAG to create a tool that can be used to answer questions about the guests at the gala.

This is a 'real-world' use case for Agentic RAG, that you could use in your own projects or workplaces. If you want to get more out of this project, why not try it out on your own use case and share in Discord?

You can choose any of the frameworks discussed in the course for this use case. We provide code samples for each in separate tabs.

A Gala to Remember

Now, it's time to get our hands dirty with an actual use case. Let's set the stage!

You decided to host the most extravagant and opulent party of the century. This means lavish feasts, enchanting dancers, renowned DJs, exquisite drinks, a breathtaking fireworks display, and much more.

Alfred, your friendly neighbourhood agent, is getting ready to watch over all of your needs for this party, and Alfred is going to manage everything himself. To do so, he needs to have access to all of the information about the party, including the menu, the guests, the schedule, weather forecasts, and much more!

Not only that, but he also needs to make sure that the party is going to be a success, so he needs to be able to answer any questions about the party during the party, whilst handling unexpected situations that may arise.

He can't do this alone, so we need to make sure that Alfred has access to all of the information and tools he needs.

First, let's give him a list of hard requirements for the gala.

The Gala Requirements

A properly educated person in the age of the Renaissance needs to have three main traits. He or she needed to be profound in the knowledge of sports, culture, and science. So, we need to make sure we can impress our guests with our knowledge and provide them with a truly unforgettable gala. However, to avoid any conflicts, there are some topics, like politics and religion, that are to be avoided at a gala. It needs to be a fun party without conflicts related to beliefs and ideals.

According to etiquette, a good host should be aware of guests' backgrounds, including their interests and endeavours. A good host also gossips and shares stories about the guests with one another.

Lastly, we need to make sure that we've got some general knowledge about the weather to ensure we can continuously find a real-time update to ensure perfect timing to launch the fireworks and end the gala with a bang!

As you can see, Alfred needs a lot of information to host the gala. Luckily, we can help and prepare Alfred by giving him some Retrieval Augmented Generation (RAG) training!

Let's start by creating the tools that Alfred needs to be able to host the gala!

Agentic Retrieval Augmented Generation (RAG)

Source file: [units/en/unit3/agentic-rag/agentic-rag.mdx](#)

Agentic Retrieval Augmented Generation (RAG)

In this unit, we'll be taking a look at how we can use Agentic RAG to help Alfred prepare for the amazing gala.

We know we've already discussed Retrieval Augmented Generation (RAG) and agentic RAG in the previous unit, so feel free to skip ahead if you're already familiar with the concepts.

LLMs are trained on enormous bodies of data to learn general knowledge. However, the world knowledge model of LLMs may not always be relevant and up-to-date information. RAG solves this problem by finding and retrieving relevant information from your data and forwarding that to the LLM.

[Image omitted from text-first adaptation: RAG]

Now, think about how Alfred works:

1. We've asked Alfred to help plan a gala
1. Alfred needs to find the latest news and weather information
1. Alfred needs to structure and search the guest information

Just as Alfred needs to search through your household information to be helpful, any agent needs a way to find and understand relevant data. Agentic RAG is a powerful way to use agents to answer questions about your data. We can pass various tools to Alfred to help him answer questions. However, instead of answering the question on top of documents automatically, Alfred can decide to use any other tool or flow to answer the question.

[Image omitted from text-first adaptation: Agentic RAG]

Let's start building our agentic RAG workflow!

First, we'll create a RAG tool to retrieve up-to-date details about the invitees. Next, we'll develop tools for web search, weather updates, and Hugging Face Hub model download statistics. Finally, we'll integrate everything to bring our agentic RAG agent to life!

Creating a RAG Tool for Guest Stories

Source file: <units/en/unit3/agentic-rag/invitees.mdx>

Creating a RAG Tool for Guest Stories

Alfred, your trusted agent, is preparing for the most extravagant gala of the century. To ensure the event runs smoothly, Alfred needs quick access to up-to-date information about each guest. Let's help Alfred by creating a custom Retrieval-Augmented Generation (RAG) tool, powered by our custom dataset.

Why RAG for a Gala?

Imagine Alfred mingling among the guests, needing to recall specific details about each person at a moment's notice. A traditional LLM might struggle with this task because:

1. The guest list is specific to your event and not in the model's training data
1. Guest information may change or be updated frequently
1. Alfred needs to retrieve precise details like email addresses

This is where Retrieval Augmented Generation (RAG) shines! By combining a retrieval system with an LLM, Alfred can access accurate, up-to-date information about your guests on demand.

You can choose any of the frameworks covered in the course for this use case. Select your preferred option from the code tabs.

Setting up our application

In this unit, we'll develop our agent within a HF Space, as a structured Python project. This approach helps us maintain clean, modular code by organizing different functionalities into separate files. Also, this makes for a more realistic use case where you would deploy the application for public use.

Project Structure

- tools.py - Provides auxiliary tools for the agent.
- retriever.py - Implements retrieval functions to support knowledge access.
- app.py - Integrates all components into a fully functional agent, which we'll finalize in the last part of this unit.

For a hands-on reference, check out this HF Space (https://huggingface.co/spaces/agents-course/Unit_3_Agentic_RAG), where the Agentic RAG developed in this unit is live. Feel free to clone it and experiment!

You can directly test the agent below:

[Embedded media omitted from text-first adaptation: interactive media]

Dataset Overview

Our dataset `agents-course/unit3-invitees` (<https://huggingface.co/datasets/agents-course/unit3-invitees/>) contains the following fields for each guest:

- Name: Guest's full name
- Relation: How the guest is related to the host
- Description: A brief biography or interesting facts about the guest
- Email Address: Contact information for sending invitations or follow-ups

Below is a preview of the dataset:

[Embedded media omitted from text-first adaptation: interactive media]

In a real-world scenario, this dataset could be expanded to include dietary preferences, gift interests, conversation topics to avoid, and other helpful details for a host.

Building the Guestbook Tool

We'll create a custom tool that Alfred can use to quickly retrieve guest information during the gala. Let's break this down into three manageable steps:

1. Load and prepare the dataset
1. Create the Retriever Tool
1. Integrate the Tool with Alfred

Let's start with loading and preparing the dataset!

Step 1: Load and Prepare the Dataset

First, we need to transform our raw guest data into a format that's optimized for retrieval.

We will use the Hugging Face datasets library to load the dataset and convert it into a list of Document objects from the `langchain.docstore.document` module.

```
import datasets
from langchain_core.documents import Document

# Load the dataset
guest_dataset = datasets.load_dataset("agents-course/unit3-invitees", split="train")

# Convert dataset entries into Document objects
docs = [
    Document(
```

```

        page_content="\n".join([
            f"Name: {guest['name']}",
            f"Relation: {guest['relation']}",
            f"Description: {guest['description']}",
            f"Email: {guest['email']}"
        ]),
        metadata={"name": guest["name"]}
    )
    for guest in guest_dataset
]

```

We will use the Hugging Face datasets library to load the dataset and convert it into a list of Document objects from the llama_index.core.schema module.

```

import datasets
from llama_index.core.schema import Document

# Load the dataset
guest_dataset = datasets.load_dataset("agents-course/unit3-invitees", split="train")

# Convert dataset entries into Document objects
docs = [
    Document(
        text="\n".join([
            f"Name: {guest_dataset['name'][i]}",
            f"Relation: {guest_dataset['relation'][i]}",
            f"Description: {guest_dataset['description'][i]}",
            f"Email: {guest_dataset['email'][i]}"
        ]),
        metadata={"name": guest_dataset['name'][i]}
    )
    for i in range(len(guest_dataset))
]

```

We will use the Hugging Face datasets library to load the dataset and convert it into a list of Document objects from the langchain.docstore.document module.

```

import datasets
from langchain_core.documents import Document

# Load the dataset
guest_dataset = datasets.load_dataset("agents-course/unit3-invitees", split="train")

# Convert dataset entries into Document objects
docs = [
    Document(
        page_content="\n".join([
            f"Name: {guest['name']}",
            f"Relation: {guest['relation']}",
            f"Description: {guest['description']}",
            f"Email: {guest['email']}"
        ]),
        metadata={"name": guest["name"]}
    )
    for guest in guest_dataset
]

```

In the code above, we:

- Load the dataset
- Convert each guest entry into a Document object with formatted content
- Store the Document objects in a list

This means we've got all of our data nicely available so we can get started with configuring our retrieval.

Step 2: Create the Retriever Tool

Now, let's create a custom tool that Alfred can use to search through our guest information.

We will use the BM25Retriever from the langchain_community.retrievers module to create a retriever tool.

The BM25Retriever is a great starting point for retrieval, but for more advanced semantic search, you might consider using embedding-based retrievers like those from sentence-transformers.

```

from smolagents import Tool

```

```

from langchain_community.retrievers import BM25Retriever

class GuestInfoRetrieverTool(Tool):
    name = "guest_info_retriever"
    description = "Retrieves detailed information about gala guests based on their name or relation."
    inputs = {
        "query": {
            "type": "string",
            "description": "The name or relation of the guest you want information about."
        }
    }
    output_type = "string"

    def __init__(self, docs):
        self.is_initialized = False
        self.retriever = BM25Retriever.from_documents(docs)

    def forward(self, query: str):
        results = self.retriever.invoke(query)
        if results:
            return "\n\n".join([doc.page_content for doc in results[:3]])
        else:
            return "No matching guest information found."

# Initialize the tool
guest_info_tool = GuestInfoRetrieverTool(docs)

```

Let's understand this tool step-by-step:

- The name and description help the agent understand when and how to use this tool
- The inputs define what parameters the tool expects (in this case, a search query)
- We're using a BM25Retriever, which is a powerful text retrieval algorithm that doesn't require embeddings
- The forward method processes the query and returns the most relevant guest information

We will use the BM25Retriever from the llama_index.retrievers.bm25 module to create a retriever tool.

The BM25Retriever is a great starting point for retrieval, but for more advanced semantic search, you might consider using embedding-based retrievers like those from sentence-transformers.

```

from llama_index.core.tools import FunctionTool
from llama_index.retrievers.bm25 import BM25Retriever

bm25_retriever = BM25Retriever.from_defaults(nodes=docs)

def get_guest_info_retriever(query: str) -> str:
    """Retrieves detailed information about gala guests based on their name or relation."""
    results = bm25_retriever.retrieve(query)
    if results:
        return "\n\n".join([doc.text for doc in results[:3]])
    else:
        return "No matching guest information found."

# Initialize the tool
guest_info_tool = FunctionTool.from_defaults(get_guest_info_retriever)

```

Let's understand this tool step-by-step.

- The docstring helps the agent understand when and how to use this tool
- The type decorators define what parameters the tool expects (in this case, a search query)
- We're using a BM25Retriever, which is a powerful text retrieval algorithm that doesn't require embeddings
- The method processes the query and returns the most relevant guest information

We will use the BM25Retriever from the langchain_community.retrievers module to create a retriever tool.

The BM25Retriever is a great starting point for retrieval, but for more advanced semantic search, you might consider using embedding-based retrievers like those from sentence-transformers.

```

from langchain_community.retrievers import BM25Retriever
from langchain_core.tools import Tool

bm25_retriever = BM25Retriever.from_documents(docs)

```



```
def extract_text(query: str) -> str:
    """Retrieves detailed information about gala guests based on their name or relation."""
    results = bm25_retriever.invoke(query)
    if results:
        return "\n\n".join([doc.page_content for doc in results[:3]])
    else:
        return "No matching guest information found."

guest_info_tool = Tool(
    name="guest_info_retriever",
    func=extract_text,
    description="Retrieves detailed information about gala guests based on their name or relation."
)
```

Let's understand this tool step-by-step.

- The name and description help the agent understand when and how to use this tool
- The type decorators define what parameters the tool expects (in this case, a search query)
- We're using a BM25Retriever, which is a powerful text retrieval algorithm that doesn't require embeddings
- The method processes the query and returns the most relevant guest information

Step 3: Integrate the Tool with Alfred

Finally, let's bring everything together by creating our agent and equipping it with our custom tool:

```
from smolagents import CodeAgent, InferenceClientModel

# Initialize the Hugging Face model
model = InferenceClientModel()

# Create Alfred, our gala agent, with the guest info tool
alfred = CodeAgent(tools=[guest_info_tool], model=model)

# Example query Alfred might receive during the gala
response = alfred.run("Tell me about our guest named 'Lady Ada Lovelace'.")

print(" Alfred's Response:")
print(response)
```

Expected output:

```
Alfred's Response:
Based on the information I retrieved, Lady Ada Lovelace is an esteemed mathematician and friend. She is r \
enowned for her pioneering work in mathematics and computing, often celebrated as the first computer \
programmer due to her work on Charles Babbage's Analytical Engine. Her email address is ada.lovelace@ \
example.com.
```

What's happening in this final step:

- We initialize a Hugging Face model using the InferenceClientModel class
- We create our agent (Alfred) as a CodeAgent, which can execute Python code to solve problems
- We ask Alfred to retrieve information about a guest named "Lady Ada Lovelace"

```
from llama_index.core.agent.workflow import AgentWorkflow
from llama_index.llms.huggingface_api import HuggingFaceInferenceAPI

# Initialize the Hugging Face model
llm = HuggingFaceInferenceAPI(model_name="Qwen/Qwen2.5-Coder-32B-Instruct")

# Create Alfred, our gala agent, with the guest info tool
alfred = AgentWorkflow.from_tools_or_functions(
    [guest_info_tool],
    llm=llm,
)

# Example query Alfred might receive during the gala
response = await alfred.run("Tell me about our guest named 'Lady Ada Lovelace'.")

print(" Alfred's Response:")
print(response)
```

Expected output:

```
Alfred's Response:
Lady Ada Lovelace is an esteemed mathematician and friend, renowned for her pioneering work in mathematic \
```

s and computing. She is celebrated as the first computer programmer due to her work on Charles Babbage's Analytical Engine. Her email is ada.lovelace@example.com.

What's happening in this final step:

- We initialize a Hugging Face model using the `HuggingFaceInferenceAPI` class
- We create our agent (Alfred) as a `AgentWorkflow`, including the tool we just created
- We ask Alfred to retrieve information about a guest named "Lady Ada Lovelace"

```
from typing import TypedDict, Annotated
from langgraph.graph.message import add_messages
from langchain_core.messages import AnyMessage, HumanMessage, AIMessage
from langgraph.prebuilt import ToolNode
from langgraph.graph import START, StateGraph
from langgraph.prebuilt import tools_condition
from langchain_huggingface import HuggingFaceEndpoint, ChatHuggingFace

# Generate the chat interface, including the tools
llm = HuggingFaceEndpoint(
    repo_id="Qwen/Qwen2.5-Coder-32B-Instruct",
    huggingfacehub_api_token=HUGGINGFACEHUB_API_TOKEN,
)

chat = ChatHuggingFace(llm=llm, verbose=True)
tools = [guest_info_tool]
chat_with_tools = chat.bind_tools(tools)

# Generate the AgentState and Agent graph
class AgentState(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]

def assistant(state: AgentState):
    return {
        "messages": [chat_with_tools.invoke(state["messages"])],
    }

## The graph
builder = StateGraph(AgentState)

# Define nodes: these do the work
builder.add_node("assistant", assistant)
builder.add_node("tools", ToolNode(tools))

# Define edges: these determine how the control flow moves
builder.add_edge(START, "assistant")
builder.add_conditional_edges(
    "assistant",
    # If the latest message requires a tool, route to tools
    # Otherwise, provide a direct response
    tools_condition,
)
builder.add_edge("tools", "assistant")
alfred = builder.compile()

messages = [HumanMessage(content="Tell me about our guest named 'Lady Ada Lovelace'.")]
response = alfred.invoke({"messages": messages})

print(" Alfred's Response:")
print(response['messages'][-1].content)
```

Expected output:

Alfred's Response:
Lady Ada Lovelace is an esteemed mathematician and pioneer in computing, often celebrated as the first computer programmer due to her work on Charles Babbage's Analytical Engine.

What's happening in this final step:

- We initialize a Hugging Face model using the `HuggingFaceEndpoint` class. We also generate a chat interface and append the tools.
- We create our agent (Alfred) as a `StateGraph`, that combines 2 nodes (assistant, tools) using an edge
- We ask Alfred to retrieve information about a guest named "Lady Ada Lovelace"

Example Interaction

During the gala, a conversation might flow like this:

You: "Alfred, who is that gentleman talking to the ambassador?"

Alfred: quickly searches the guest database "That's Dr. Nikola Tesla, sir. He's an old friend from your university days. He's recently patented a new wireless energy transmission system and would be delighted to discuss it with you. Just remember he's passionate about pigeons, so that might make for good small talk."

```
{
  "name": "Dr. Nikola Tesla",
  "relation": "old friend from university days",
  "description": "Dr. Nikola Tesla is an old friend from your university days. He's recently patented a \
new wireless energy transmission system and would be delighted to discuss it with you. Just remember \
he's passionate about pigeons, so that might make for good small talk.",
  "email": "nikola.tesla@gmail.com"
}
```

Taking It Further

Now that Alfred can retrieve guest information, consider how you might enhance this system:

1. Improve the retriever to use a more sophisticated algorithm like sentence-transformers (<https://www.sbert.net/>)
1. Implement a conversation memory so Alfred remembers previous interactions
1. Combine with web search to get the latest information on unfamiliar guests
1. Integrate multiple indexes to get more complete information from verified sources

Now Alfred is fully equipped to handle guest inquiries effortlessly, ensuring your gala is remembered as the most sophisticated and delightful event of the century!

Try extending the retriever tool to also return conversation starters based on each guest's interests or background. How would you modify the tool to accomplish this?

When you're done, implement your guest retriever tool in the retriever.py file.

Building and Integrating Tools for Your Agent

Source file: [units/en/unit3/agent-rag/tools.mdx](#)

Building and Integrating Tools for Your Agent

In this section, we'll grant Alfred access to the web, enabling him to find the latest news and global updates. Additionally, he'll have access to weather data and Hugging Face hub model download statistics, so that he can make relevant conversation about fresh topics.

Give Your Agent Access to the Web

Remember that we want Alfred to establish his presence as a true renaissance host, with a deep knowledge of the world.

To do so, we need to make sure that Alfred has access to the latest news and information about the world.

Let's start by creating a web search tool for Alfred!

```
from smolagents import DuckDuckGoSearchTool

# Initialize the DuckDuckGo search tool
search_tool = DuckDuckGoSearchTool()

# Example usage
results = search_tool("Who's the current President of France?")
print(results)
```

Expected output:

The current President of France is Emmanuel Macron.

```
from llama_index.tools.duckduckgo import DuckDuckGoSearchToolSpec
from llama_index.core.tools import FunctionTool

# Initialize the DuckDuckGo search tool
tool_spec = DuckDuckGoSearchToolSpec()

search_tool = FunctionTool.from_defaults(tool_spec.duckduckgo_full_search)
# Example usage
response = search_tool("Who's the current President of France?")
print(response.raw_output[-1]['body'])
```

Expected output:

The President of the French Republic is the head of state of France. The current President is Emmanuel Ma \
cron since 14 May 2017 defeating Marine Le Pen in the second round of the presidential election on 7 \
May 2017. List of French presidents (Fifth Republic) N° Portrait Name ...

```
from langchain_community.tools import DuckDuckGoSearchRun

search_tool = DuckDuckGoSearchRun()
results = search_tool.invoke("Who's the current President of France?")
print(results)
```

Expected output:

Emmanuel Macron (born December 21, 1977, Amiens, France) is a French banker and politician who was electe \
d president of France in 2017...

Creating a Custom Tool for Weather Information to Schedule the Fireworks

The perfect gala would have fireworks over a clear sky, we need to make sure the fireworks are not cancelled due to bad weather.

Let's create a custom tool that can be used to call an external weather API and get the weather information for a given location.

For the sake of simplicity, we're using a dummy weather API for this example. If you want to use a real weather API, you could implement a weather tool that uses the OpenWeatherMap API, like in Unit 1.

```
from smolagents import Tool
import random

class WeatherInfoTool(Tool):
    name = "weather_info"
    description = "Fetches dummy weather information for a given location."
    inputs = {
        "location": {
            "type": "string",
            "description": "The location to get weather information for."
        }
    }
    output_type = "string"

    def forward(self, location: str):
        # Dummy weather data
        weather_conditions = [
            {"condition": "Rainy", "temp_c": 15},
            {"condition": "Clear", "temp_c": 25},
            {"condition": "Windy", "temp_c": 20}
        ]
        # Randomly select a weather condition
        data = random.choice(weather_conditions)
        return f"Weather in {location}: {data['condition']}, {data['temp_c']}°C"

# Initialize the tool
weather_info_tool = WeatherInfoTool()

import random
from llama_index.core.tools import FunctionTool

def get_weather_info(location: str) -> str:
    """Fetches dummy weather information for a given location."""
    # Dummy weather data
    weather_conditions = [
        {"condition": "Rainy", "temp_c": 15},
        {"condition": "Clear", "temp_c": 25},
        {"condition": "Windy", "temp_c": 20}
    ]
    # Randomly select a weather condition
    data = random.choice(weather_conditions)
```

```

        return f"Weather in {location}: {data['condition']}, {data['temp_c']}°C"

# Initialize the tool
weather_info_tool = FunctionTool.from_defaults(get_weather_info)

from langchain_core.tools import Tool
import random

def get_weather_info(location: str) -> str:
    """Fetches dummy weather information for a given location."""
    # Dummy weather data
    weather_conditions = [
        {"condition": "Rainy", "temp_c": 15},
        {"condition": "Clear", "temp_c": 25},
        {"condition": "Windy", "temp_c": 20}
    ]
    # Randomly select a weather condition
    data = random.choice(weather_conditions)
    return f"Weather in {location}: {data['condition']}, {data['temp_c']}°C"

# Initialize the tool
weather_info_tool = Tool(
    name="get_weather_info",
    func=get_weather_info,
    description="Fetches dummy weather information for a given location."
)

```

Creating a Hub Stats Tool for Influential AI Builders

In attendance at the gala are the who's who of AI builders. Alfred wants to impress them by discussing their most popular models, datasets, and spaces. We'll create a tool to fetch model statistics from the Hugging Face Hub based on a username.

```

from smolagents import Tool
from huggingface_hub import list_models

class HubStatsTool(Tool):
    name = "hub_stats"
    description = "Fetches the most downloaded model from a specific author on the Hugging Face Hub."
    inputs = {
        "author": {
            "type": "string",
            "description": "The username of the model author/organization to find models from."
        }
    }
    output_type = "string"

    def forward(self, author: str):
        try:
            # List models from the specified author, sorted by downloads
            models = list(list_models(author=author, sort="downloads", direction=-1, limit=1))

            if models:
                model = models[0]
                return f"The most downloaded model by {author} is {model.id} with {model.downloads:,} downloads."
            else:
                return f"No models found for author {author}."
        except Exception as e:
            return f"Error fetching models for {author}: {str(e)}"

# Initialize the tool
hub_stats_tool = HubStatsTool()

# Example usage
print(hub_stats_tool("facebook")) # Example: Get the most downloaded model by Facebook

```

Expected output:

The most downloaded model by facebook is facebook/esmfold_v1 with 12,544,550 downloads.

```

import random
from llama_index.core.tools import FunctionTool
from huggingface_hub import list_models

def get_hub_stats(author: str) -> str:
    """Fetches the most downloaded model from a specific author on the Hugging Face Hub."""
    try:
        # List models from the specified author, sorted by downloads
        models = list(list_models(author=author, sort="downloads", direction=-1, limit=1))

        if models:

```

```

        model = models[0]
        return f"The most downloaded model by {author} is {model.id} with {model.downloads:,} downloads."
    else:
        return f"No models found for author {author}."
except Exception as e:
    return f"Error fetching models for {author}: {str(e)}"

# Initialize the tool
hub_stats_tool = FunctionTool.from_defaults(get_hub_stats)

# Example usage
print(hub_stats_tool("facebook")) # Example: Get the most downloaded model by Facebook

```

Expected output:

The most downloaded model by facebook is facebook/esmfold_v1 with 12,544,550 downloads.

```

from langchain_core.tools import Tool
from huggingface_hub import list_models

def get_hub_stats(author: str) -> str:
    """Fetches the most downloaded model from a specific author on the Hugging Face Hub."""
    try:
        # List models from the specified author, sorted by downloads
        models = list(list_models(author=author, sort="downloads", direction=-1, limit=1))

        if models:
            model = models[0]
            return f"The most downloaded model by {author} is {model.id} with {model.downloads:,} downloads."
        else:
            return f"No models found for author {author}."
    except Exception as e:
        return f"Error fetching models for {author}: {str(e)}"

# Initialize the tool
hub_stats_tool = Tool(
    name="get_hub_stats",
    func=get_hub_stats,
    description="Fetches the most downloaded model from a specific author on the Hugging Face Hub."
)

# Example usage
print(hub_stats_tool.invoke("facebook")) # Example: Get the most downloaded model by Facebook

```

Expected output:

The most downloaded model by facebook is facebook/esmfold_v1 with 13,109,861 downloads.

With the Hub Stats Tool, Alfred can now impress influential AI builders by discussing their most popular models.

Integrating Tools with Alfred

Now that we have all the tools, let's integrate them into Alfred's agent:

```

from smolagents import CodeAgent, InferenceClientModel

# Initialize the Hugging Face model
model = InferenceClientModel()

# Create Alfred with all the tools
alfred = CodeAgent(
    tools=[search_tool, weather_info_tool, hub_stats_tool],
    model=model
)

# Example query Alfred might receive during the gala
response = alfred.run("What is Facebook and what's their most popular model?")

print("Alfred's Response:")
print(response)

```

Expected output:

```

Alfred's Response:
Facebook is a social networking website where users can connect, share information, and interact with others. The most downloaded model by Facebook on the Hugging Face Hub is ESMFold_v1.

from llama_index.core.agent.workflow import AgentWorkflow
from llama_index.llms.huggingface_api import HuggingFaceInferenceAPI

```

```
# Initialize the Hugging Face model
llm = HuggingFaceInferenceAPI(model_name="Qwen/Qwen2.5-Coder-32B-Instruct")
# Create Alfred with all the tools
alfred = AgentWorkflow.from_tools_or_functions(
    [search_tool, weather_info_tool, hub_stats_tool],
    llm=llm
)

# Example query Alfred might receive during the gala
response = await alfred.run("What is Facebook and what's their most popular model?")

print(" Alfred's Response:")
print(response)
```

Expected output:

```
Alfred's Response:
Facebook is a social networking service and technology company based in Menlo Park, California. It was fo \
unded by Mark Zuckerberg and allows people to create profiles, connect with friends and family, share \
photos and videos, and join groups based on shared interests. The most popular model by Facebook on \
the Hugging Face Hub is `facebook/esmfold_v1` with 13,109,861 downloads.

from typing import TypedDict, Annotated
from langgraph.graph.message import add_messages
from langchain_core.messages import AnyMessage, HumanMessage, AIMessage
from langgraph.prebuilt import ToolNode
from langgraph.graph import START, StateGraph
from langgraph.prebuilt import tools_condition
from langchain_huggingface import HuggingFaceEndpoint, ChatHuggingFace

# Generate the chat interface, including the tools
llm = HuggingFaceEndpoint(
    repo_id="Qwen/Qwen2.5-Coder-32B-Instruct",
    huggingfacehub_api_token=HUGGINGFACEHUB_API_TOKEN,
)

chat = ChatHuggingFace(llm=llm, verbose=True)
tools = [search_tool, weather_info_tool, hub_stats_tool]
chat_with_tools = chat.bind_tools(tools)

# Generate the AgentState and Agent graph
class AgentState(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]

def assistant(state: AgentState):
    return {
        "messages": [chat_with_tools.invoke(state["messages"])],
    }

## The graph
builder = StateGraph(AgentState)

# Define nodes: these do the work
builder.add_node("assistant", assistant)
builder.add_node("tools", ToolNode(tools))

# Define edges: these determine how the control flow moves
builder.add_edge(START, "assistant")
builder.add_conditional_edges(
    "assistant",
    # If the latest message requires a tool, route to tools
    # Otherwise, provide a direct response
    tools_condition,
)
builder.add_edge("tools", "assistant")
alfred = builder.compile()

messages = [HumanMessage(content="Who is Facebook and what's their most popular model?")]
response = alfred.invoke({"messages": messages})

print(" Alfred's Response:")
print(response['messages'][-1].content)
```

Expected output:

```
Alfred's Response:
Facebook is a social media company known for its social networking site, Facebook, as well as other servi \
ces like Instagram and WhatsApp. The most downloaded model by Facebook on the Hugging Face Hub is fac \
ebook/esmfold_v1 with 13,202,321 downloads.
```

Conclusion

By integrating these tools, Alfred is now equipped to handle a variety of tasks, from web searches to weather updates and model statistics. This ensures he remains the most informed and engaging host at the gala.

Try implementing a tool that can be used to get the latest news about a specific topic.

When you're done, implement your custom tools in the tools.py file.

Creating Your Gala Agent

Source file: [units/en/unit3/agentic-rag/agent.mdx](#)

Creating Your Gala Agent

Now that we've built all the necessary components for Alfred, it's time to bring everything together into a complete agent that can help host our extravagant gala.

In this section, we'll combine the guest information retrieval, web search, weather information, and Hub stats tools into a single powerful agent.

Assembling Alfred: The Complete Agent

Instead of reimplementing all the tools we've created in previous sections, we'll import them from their respective modules which we saved in the tools.py and retriever.py files.

If you haven't implemented the tools yet, go back to the tools and retriever sections to implement them, and add them to the tools.py and retriever.py files.

Let's import the necessary libraries and tools from the previous sections:

```
# Import necessary libraries
import random
from smolagents import CodeAgent, InferenceClientModel

# Import our custom tools from their modules
from tools import DuckDuckGoSearchTool, WeatherInfoTool, HubStatsTool
from retriever import load_guest_dataset
```

Now, let's combine all these tools into a single agent:

```
# Initialize the Hugging Face model
model = InferenceClientModel()

# Initialize the web search tool
search_tool = DuckDuckGoSearchTool()

# Initialize the weather tool
weather_info_tool = WeatherInfoTool()

# Initialize the Hub stats tool
hub_stats_tool = HubStatsTool()

# Load the guest dataset and initialize the guest info tool
guest_info_tool = load_guest_dataset()

# Create Alfred with all the tools
alfred = CodeAgent(
    tools=[guest_info_tool, weather_info_tool, hub_stats_tool, search_tool],
    model=model,
    add_base_tools=True, # Add any additional base tools
    planning_interval=3  # Enable planning every 3 steps
)

# Import necessary libraries
from llama_index.core.agent.workflow import AgentWorkflow
from llama_index.llms.huggingface_api import HuggingFaceInferenceAPI

from tools import search_tool, weather_info_tool, hub_stats_tool
from retriever import guest_info_tool
```


Now, let's combine all these tools into a single agent:

```
# Initialize the Hugging Face model
llm = HuggingFaceInferenceAPI(model_name="Qwen/Qwen2.5-Coder-32B-Instruct")

# Create Alfred with all the tools
alfred = AgentWorkflow.from_tools_or_functions(
    [guest_info_tool, search_tool, weather_info_tool, hub_stats_tool],
    llm=llm,
)

from typing import TypedDict, Annotated
from langgraph.graph.message import add_messages
from langchain_core.messages import AnyMessage, HumanMessage, AIMessage
from langgraph.prebuilt import ToolNode
from langgraph.graph import START, StateGraph
from langgraph.prebuilt import tools_condition
from langchain_huggingface import HuggingFaceEndpoint, ChatHuggingFace

from tools import DuckDuckGoSearchRun, weather_info_tool, hub_stats_tool
from retriever import guest_info_tool
```

Now, let's combine all these tools into a single agent:

```
# Initialize the web search tool
search_tool = DuckDuckGoSearchRun()

# Generate the chat interface, including the tools
llm = HuggingFaceEndpoint(
    repo_id="Qwen/Qwen2.5-Coder-32B-Instruct",
    huggingfacehub_api_token=HUGGINGFACEHUB_API_TOKEN,
)

chat = ChatHuggingFace(llm=llm, verbose=True)
tools = [guest_info_tool, search_tool, weather_info_tool, hub_stats_tool]
chat_with_tools = chat.bind_tools(tools)

# Generate the AgentState and Agent graph
class AgentState(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]

def assistant(state: AgentState):
    return {
        "messages": [chat_with_tools.invoke(state["messages"])],
    }

## The graph
builder = StateGraph(AgentState)

# Define nodes: these do the work
builder.add_node("assistant", assistant)
builder.add_node("tools", ToolNode(tools))

# Define edges: these determine how the control flow moves
builder.add_edge(START, "assistant")
builder.add_conditional_edges(
    "assistant",
    # If the latest message requires a tool, route to tools
    # Otherwise, provide a direct response
    tools_condition,
)
builder.add_edge("tools", "assistant")
alfred = builder.compile()
```

Your agent is now ready to use!

Using Alfred: End-to-End Examples

Now that Alfred is fully equipped with all the necessary tools, let's see how he can help with various tasks during the gala.

Example 1: Finding Guest Information

Let's see how Alfred can help us with our guest information.

```
query = "Tell me about 'Lady Ada Lovelace'"
response = alfred.run(query)

print(" Alfred's Response:")
print(response)
```

Expected output:

```
Alfred's Response:
Based on the information I retrieved, Lady Ada Lovelace is an esteemed mathematician and friend. She is renowned for her pioneering work in mathematics and computing, often celebrated as the first computer programmer due to her work on Charles Babbage's Analytical Engine. Her email address is ada.lovelace@example.com.

query = "Tell me about Lady Ada Lovelace. What's her background?"
response = await alfred.run(query)

print(" Alfred's Response:")
print(response.response.blocks[0].text)
```

Expected output:

```
Alfred's Response:
Lady Ada Lovelace was an English mathematician and writer, best known for her work on Charles Babbage's Analytical Engine. She was the first to recognize that the machine had applications beyond pure calculation.

response = alfred.invoke({"messages": "Tell me about 'Lady Ada Lovelace'"})

print(" Alfred's Response:")
print(response['messages'][-1].content)
```

Expected output:

```
Alfred's Response:
Ada Lovelace, also known as Augusta Ada King, Countess of Lovelace, was an English mathematician and writer. Born on December 10, 1815, and passing away on November 27, 1852, she is renowned for her work on Charles Babbage's Analytical Engine, a proposed mechanical general-purpose computer. Ada Lovelace is celebrated as one of the first computer programmers because she created a program for the Analytical Engine in 1843. She recognized that the machine could be used for more than mere calculation, envisioning its potential in a way that few did at the time. Her contributions to the field of computer science laid the groundwork for future developments. A day in October, designated as Ada Lovelace Day, honors women's contributions to science and technology, inspired by Lovelace's pioneering work.
```

Example 2: Checking the Weather for Fireworks

Let's see how Alfred can help us with the weather.

```
query = "What's the weather like in Paris tonight? Will it be suitable for our fireworks display?"
response = alfred.run(query)

print(" Alfred's Response:")
print(response)
```

Expected output (will vary due to randomness):

```
Alfred's Response:
I've checked the weather in Paris for you. Currently, it's clear with a temperature of 25°C. These conditions are perfect for the fireworks display tonight. The clear skies will provide excellent visibility for the spectacular show, and the comfortable temperature will ensure the guests can enjoy the outdoor event without discomfort.

query = "What's the weather like in Paris tonight? Will it be suitable for our fireworks display?"
response = await alfred.run(query)

print(" Alfred's Response:")
print(response)
```

Expected output:

```
Alfred's Response:
The weather in Paris tonight is rainy with a temperature of 15°C. Given the rain, it may not be suitable for a fireworks display.

response = alfred.invoke({"messages": "What's the weather like in Paris tonight? Will it be suitable for our fireworks display?"})

print(" Alfred's Response:")
print(response['messages'][-1].content)
```

Expected output:

```
Alfred's Response:
The weather in Paris tonight is rainy with a temperature of 15°C, which may not be suitable for your fireworks display.
```

Example 3: Impressing AI Researchers

Let's see how Alfred can help us impress AI researchers.

```
query = "One of our guests is from Qwen. What can you tell me about their most popular model?"
response = alfred.run(query)
```

```
print(" Alfred's Response:")
print(response)
```

Expected output:

```
Alfred's Response:
The most popular Qwen model is Qwen/Qwen2.5-VL-7B-Instruct with 3,313,345 downloads.
```

```
query = "One of our guests is from Google. What can you tell me about their most popular model?"
response = await alfred.run(query)
```

```
print(" Alfred's Response:")
print(response)
```

Expected output:

```
Alfred's Response:
The most popular model by Google on the Hugging Face Hub is google/electra-base-discriminator, with 28,546,752 downloads.
```

```
response = alfred.invoke({"messages": "One of our guests is from Qwen. What can you tell me about their most popular model?"})
```

```
print(" Alfred's Response:")
print(response['messages'][-1].content)
```

Expected output:

```
Alfred's Response:
The most downloaded model by Qwen is Qwen/Qwen2.5-VL-7B-Instruct with 3,313,345 downloads.
```

Example 4: Combining Multiple Tools

Let's see how Alfred can help us prepare for a conversation with Dr. Nikola Tesla.

```
query = "I need to speak with Dr. Nikola Tesla about recent advancements in wireless energy. Can you help me prepare for this conversation?"
response = alfred.run(query)
```

```
print(" Alfred's Response:")
print(response)
```

Expected output:

```
Alfred's Response:
I've gathered information to help you prepare for your conversation with Dr. Nikola Tesla.
```

Guest Information:

Name: Dr. Nikola Tesla

Relation: old friend from university days

Description: Dr. Nikola Tesla is an old friend from your university days. He's recently patented a new wireless energy transmission system and would be delighted to discuss it with you. Just remember he's passionate about pigeons, so that might make for good small talk.

Email: nikola.tesla@gmail.com

Recent Advancements in Wireless Energy:

Based on my web search, here are some recent developments in wireless energy transmission:

1. Researchers have made progress in long-range wireless power transmission using focused electromagnetic waves
2. Several companies are developing resonant inductive coupling technologies for consumer electronics
3. There are new applications in electric vehicle charging without physical connections

Conversation Starters:

1. "I'd love to hear about your new patent on wireless energy transmission. How does it compare to your original concepts from our university days?"
2. "Have you seen the recent developments in resonant inductive coupling for consumer electronics? What do you think of their approach?"
3. "How are your pigeons doing? I remember your fascination with them."

This should give you plenty to discuss with Dr. Tesla while demonstrating your knowledge of his interests and recent developments in his field.

```
query = "I need to speak with Dr. Nikola Tesla about recent advancements in wireless energy. Can you help me prepare for this conversation?"
response = await alfred.run(query)
```

```
print(" Alfred's Response:")
print(response)
```

Expected output:

```
Alfred's Response:
Here are some recent advancements in wireless energy that you might find useful for your conversation with Dr. Nikola Tesla:

1. Advancements and Challenges in Wireless Power Transfer: This article discusses the evolution of wireless power transfer (WPT) from conventional wired methods to modern applications, including solar space power stations. It highlights the initial focus on microwave technology and the current demand for WPT due to the rise of electric devices.

2. Recent Advances in Wireless Energy Transfer Technologies for Body-Interfaced Electronics: This article explores wireless energy transfer (WET) as a solution for powering body-interfaced electronics without the need for batteries or lead wires. It discusses the advantages and potential applications of WET in this context.

3. Wireless Power Transfer and Energy Harvesting: Current Status and Future Trends: This article provides an overview of recent advances in wireless power supply methods, including energy harvesting and wireless power transfer. It presents several promising applications and discusses future trends in the field.

4. Wireless Power Transfer: Applications, Challenges, Barriers, and the

response = alfred.invoke({"messages": "I need to speak with 'Dr. Nikola Tesla' about recent advancements in wireless energy. Can you help me prepare for this conversation?"})

print(" Alfred's Response:")
print(response['messages'][-1].content)
```

Expected output:

```
Based on the provided information, here are key points to prepare for the conversation with 'Dr. Nikola Tesla' about recent advancements in wireless energy:\n1. Wireless Power Transmission (WPT): Discuss how WPT revolutionizes energy transfer by eliminating the need for cords and leveraging mechanisms like inductive and resonant coupling.\n2. Advancements in Wireless Charging: Highlight improvements in efficiency, faster charging speeds, and the rise of Qi/Qi2 certified wireless charging solutions.\n3. 5G-Advanced Innovations and NearLink Wireless Protocol: Mention these as developments that enhance speed, security, and efficiency in wireless networks, which can support advanced wireless energy technologies.\n4. AI and ML at the Edge: Talk about how AI and machine learning will rely on wireless networks to bring intelligence to the edge, enhancing automation and intelligence in smart homes and buildings.\n5. Matter, Thread, and Security Advancements: Discuss these as key innovations that drive connectivity, efficiency, and security in IoT devices and systems.\n6. Breakthroughs in Wireless Charging Technology: Include any recent breakthroughs or studies, such as the one from Incheon National University, to substantiate the advancements in wireless charging.
```

Advanced Features: Conversation Memory

To make Alfred even more helpful during the gala, we can enable conversation memory so he remembers previous interactions:

```
# Create Alfred with conversation memory
alfred_with_memory = CodeAgent(
    tools=[guest_info_tool, weather_info_tool, hub_stats_tool, search_tool],
    model=model,
    add_base_tools=True,
    planning_interval=3
)

# First interaction
response1 = alfred_with_memory.run("Tell me about Lady Ada Lovelace.")
print(" Alfred's First Response:")
print(response1)

# Second interaction (referencing the first)
response2 = alfred_with_memory.run("What projects is she currently working on?", reset=False)
print(" Alfred's Second Response:")
print(response2)

from llama_index.core.workflow import Context

alfred = AgentWorkflow.from_tools_or_functions(
    [guest_info_tool, search_tool, weather_info_tool, hub_stats_tool],
    llm=llm
)

# Remembering state
ctx = Context(alfred)

# First interaction
```

```
response1 = await alfred.run("Tell me about Lady Ada Lovelace.", ctx=ctx)
print(" Alfred's First Response:")
print(response1)

# Second interaction (referencing the first)
response2 = await alfred.run("What projects is she currently working on?", ctx=ctx)
print(" Alfred's Second Response:")
print(response2)

# First interaction
response = alfred.invoke({"messages": [HumanMessage(content="Tell me about 'Lady Ada Lovelace'. What's he \
r background and how is she related to me?")]])

print(" Alfred's Response:")
print(response['messages'][-1].content)
print()

# Second interaction (referencing the first)
response = alfred.invoke({"messages": response["messages"] + [HumanMessage(content="What projects is she \
currently working on?")]])

print(" Alfred's Response:")
print(response['messages'][-1].content)
```

Notice that none of these three agent approaches directly couple memory with the agent. Is there a specific reason for this design choice ?

- smolagents: Memory is not preserved across different execution runs, you must explicitly state it using `reset=False`.
- LlamaIndex: Requires explicitly adding a context object for memory management within a run.
- LangGraph: Offers options to retrieve previous messages or utilize a dedicated `MemorySaver` (<https://langchain-ai.github.io/langgraph/tutorials/introduction/#part-3-adding-memory-to-the-chatbot>) component.

Conclusion

Congratulations! You've successfully built Alfred, a sophisticated agent equipped with multiple tools to help host the most extravagant gala of the century. Alfred can now:

1. Retrieve detailed information about guests
1. Check weather conditions for planning outdoor activities
1. Provide insights about influential AI builders and their models
1. Search the web for the latest information
1. Maintain conversation context with memory

With these capabilities, Alfred is ready to ensure your gala is a resounding success, impressing guests with personalized attention and up-to-date information.

Conclusion

Source file: [units/en/unit3/agent-rag/conclusion.mdx](#)

Conclusion

In this unit, we've learned how to create an agentic RAG system to help Alfred, our friendly neighborhood agent, prepare for and manage an extravagant gala.

The combination of RAG with agentic capabilities demonstrates how powerful AI assistants can become when they have:

- Access to structured knowledge (guest information)
- Ability to retrieve real-time information (web search)
- Domain-specific tools (weather information, Hub stats)
- Memory of past interactions

With these capabilities, Alfred is now well-equipped to be the perfect host, able to answer questions about guests, provide up-to-date information, and ensure the gala runs smoothly-even managing the perfect timing for the fireworks display!

<i>Now that you've built a complete agent, you might want to explore:</i>
<i>- Creating more specialized tools for your own use cases</i>
<i>- Implementing more sophisticated RAG systems with embeddings</i>
<i>- Building multi-agent systems where agents can collaborate</i>
<i>- Deploying your agent as a service that others can interact with</i>

Unit 4. Final Project

Create, Test, and Certify Your Agent

Instructional sections below follow the official English course order.

Introduction to the Final Unit

Source file: [units/en/unit4/introduction.mdx](#)

Welcome to the final Unit

[Image omitted from text-first adaptation: AI Agents Course thumbnail]

Welcome to the final unit of the course!

So far, you've built a strong foundation in AI Agents, from understanding their components to creating your own. With this knowledge, you're now ready to build powerful agents and stay up-to-date with the latest advancements in this fast-evolving field.

This unit is all about applying what you've learned. It's your final hands-on project, and completing it is your ticket to earning the course certificate.

What's the challenge?

You'll create your own agent and evaluate its performance using a subset of the GAIA benchmark (<https://huggingface.co/spaces/gaia-benchmark/leaderboard>).

To successfully complete the course, your agent needs to score 30% or higher on the benchmark. Achieve that, and you'll earn your Certificate of Completion, officially recognizing your expertise.

Additionally, see how you stack up against your peers! A dedicated Student Leaderboard (https://huggingface.co/spaces/agents-course/Students_leaderboard) is available for you to submit your scores and see the community's progress.

Heads Up: Advanced & Hands-On Unit

Please be aware that this unit shifts towards a more practical, hands-on approach. Success in this section will require more advanced coding knowledge and relies on you navigating tasks with less explicit guidance compared to earlier parts of the course.

Sounds exciting? Let's get started!

What is GAIA?

Source file: [units/en/unit4/what-is-gaia.mdx](#)

What is GAIA?

GAIA (<https://huggingface.co/papers/2311.12983>) is a benchmark designed to evaluate AI assistants on real-world tasks that require a combination of core capabilities-such as reasoning, multimodal understanding, web browsing, and proficient tool use.

It was introduced in the paper "GAIA: A Benchmark for General AI Assistants (<https://huggingface.co/papers/2311.12983>)".

The benchmark features 466 carefully curated questions that are conceptually simple for humans, yet remarkably challenging for current AI systems.

To illustrate the gap:

- Humans: ~92% success rate
- GPT-4 with plugins: ~15%
- Deep Research (OpenAI): 67.36% on the validation set

GAIA highlights the current limitations of AI models and provides a rigorous benchmark to evaluate progress toward truly general-purpose AI assistants.

GAIA's Core Principles

GAIA is carefully designed around the following pillars:

- Real-world difficulty: Tasks require multi-step reasoning, multimodal understanding, and tool interaction.
- Human interpretability: Despite their difficulty for AI, tasks remain conceptually simple and easy to follow for humans.
- Non-gameability: Correct answers demand full task execution, making brute-forcing ineffective.
- Simplicity of evaluation: Answers are concise, factual, and unambiguous-ideal for benchmarking.

Difficulty Levels

GAIA tasks are organized into three levels of increasing complexity, each testing specific skills:

- Level 1: Requires less than 5 steps and minimal tool usage.
- Level 2: Involves more complex reasoning and coordination between multiple tools and 5-10 steps.
- Level 3: Demands long-term planning and advanced integration of various tools.

[Image omitted from text-first adaptation: GAIA levels]

Example of a Hard GAIA Question

Which of the fruits shown in the 2008 painting "Embroidery from Uzbekistan" were served as part of the October 1949 breakfast menu for the ocean liner that was later used as a floating prop for the film "The Last Voyage"? Give the items as a comma-separated list, ordering them in clockwise order based on their arrangement in the painting starting from the 12 o'clock position. Use the plural form of each fruit.

As you can see, this question challenges AI systems in several ways:

- Requires a structured response format
- Involves multimodal reasoning (e.g., analyzing images)
- Demands multi-hop retrieval of interdependent facts:
- Identifying the fruits in the painting
- Discovering which ocean liner was used in The Last Voyage
- Looking up the breakfast menu from October 1949 for that ship
- Needs correct sequencing and high-level planning to solve in the right order

This kind of task highlights where standalone LLMs often fall short, making GAIA an ideal benchmark for agent-based systems that can reason, retrieve, and execute over multiple steps and modalities.

[Image omitted from text-first adaptation: GAIA capabilities plot]

Live Evaluation

To encourage continuous benchmarking, GAIA provides a public leaderboard hosted on Hugging Face, where you can test your models against 300 testing questions.

Check out the leaderboard here (<https://huggingface.co/spaces/gaia-benchmark/leaderboard>)

[Embedded media omitted from text-first adaptation: interactive media]

Want to dive deeper into GAIA?

- Read the full paper (<https://huggingface.co/papers/2311.12983>)
- Deep Research release post by OpenAI (<https://openai.com/index/introducing-deep-research/>)

- Open-source DeepResearch - Freeing our search agents (<https://huggingface.co/blog/open-deep-research>)

The Final Hands-On

Source file: [units/en/unit4/hands-on.mdx](#)

Hands-On

Now that you're ready to dive deeper into the creation of your final agent, let's see how you can submit it for review.

The Dataset

The Dataset used in this leaderboard consist of 20 questions extracted from the level 1 questions of the validation set from GAIA.

The chosen question were filtered based on the number of tools and steps needed to answer a question.

Based on the current look of the GAIA benchmark, we think that getting you to try to aim for 30% on level 1 question is a fair test.

[Image omitted from text-first adaptation: GAIA current status!]

The process

Now the big question in your mind is probably : "How do I start submitting ?"

For this Unit, we created an API that will allow you to get the questions, and send your answers for scoring. Here is a summary of the routes (see the live documentation (<https://agents-course-unit4-scoring.hf.space/docs>) for interactive details):

- GET /questions: Retrieve the full list of filtered evaluation questions.
- GET /random-question: Fetch a single random question from the list.
- GET /files/{task_id}: Download a specific file associated with a given task ID.
- POST /submit: Submit agent answers, calculate the score, and update the leaderboard.

The submit function will compare the answer to the ground truth in an EXACT MATCH manner, hence prompt it well ! The GAIA team shared a prompting example for your agent here (<https://huggingface.co/spaces/gaia-benchmark/leaderboard>) (for the sake of this course, make sure you don't include the text "FINAL ANSWER" in your submission, just make your agent reply with the answer and nothing else).

Make the Template Your Own!

To demonstrate the process of interacting with the API, we've included a basic template (https://huggingface.co/spaces/agents-course/Final_Assignment_Template) as a starting point.

Please feel free-and actively encouraged-to change, add to, or completely restructure it! Modify it in any way that best suits your approach and creativity.

In order to submit this templates compute 3 things needed by the API :

- Username: Your Hugging Face username (here obtained via Gradio login), which is used to identify your submission.
- Code Link (agent_code): the URL linking to your Hugging Face Space code (.../tree/main) for verification purposes, so please keep your space public.
- Answers (answers): The list of responses ({"task_id": ..., "submitted_answer": ...}) generated by your Agent for scoring.

Hence we encourage you to start by duplicating this template

(https://huggingface.co/spaces/agents-course/Final_Assignment_Template) on your own huggingface profile.

Check out the leaderboard here (https://huggingface.co/spaces/agents-course/Students_leaderboard)

A friendly note: This leaderboard is meant for fun! We know it's possible to submit scores without full verification. If we see too many high scores posted without a public link to back them up, we might need to review, adjust, or remove some entries to keep

the leaderboard useful. The leaderboard will show the link to your space code-base, since this leaderboard is for students only, please keep your space public if you get a score you're proud of.

[Embedded media omitted from text-first adaptation: interactive media]

Get Your Certificate Of Excellence

Source file: [units/en/unit4/get-your-certificate.mdx](https://huggingface.co/spaces/agents-course/Unit4-Final-Certificate)

Claim Your Certificate

If you scored above 30%, congratulations! You're now eligible to claim your official certificate.

Follow the steps below to receive it:

1. Visit the certificate page (<https://huggingface.co/spaces/agents-course/Unit4-Final-Certificate>).
1. Sign in with your Hugging Face account using the button provided.
1. Enter your full name. This is the name that will appear on your certificate.
1. Click "Get My Certificate" to verify your score and download your certificate.

[Image omitted from text-first adaptation: Congrats!]

Once you've got your certificate, feel free to:

- Add it to your LinkedIn profile
- Share it on X, Bluesky, etc.

Don't forget to tag [@huggingface](https://huggingface.co/huggingface) (<https://huggingface.co/huggingface>). We'd be super proud and we'd love to cheer you on!

If you have any issues with submission please open a discussion item on The certification community tab (<https://huggingface.co/spaces/agents-course/Unit4-Final-Certificate/discussions>).

Conclusion of the Course

Source file: [units/en/unit4/conclusion.mdx](https://huggingface.co/spaces/agents-course/Unit4-Final-Certificate/discussions)

Conclusion

Congratulations on finishing the Agents Course!

Through perseverance and dedication, you've built a solid foundation in the world of AI Agents.

But finishing this course is not the end of your journey. It's just the beginning: don't hesitate to explore the next section where we share curated resources to help you continue learning, including advanced topics like MCPs and beyond.

Thank you for being part of this course. We hope you liked this course as much as we loved writing it.

And don't forget: Keep Learning, Stay Awesome

What Should You Learn Now?

Source file: [units/en/unit4/additional-readings.mdx](https://huggingface.co/spaces/agents-course/Unit4-Final-Certificate/discussions)

And now? What topics I should learn?

Agentic AI is a rapidly evolving field, and understanding foundational protocols is essential for building intelligent, autonomous systems.

Two important standards you should get familiar with are:

- The Model Context Protocol (MCP)
- The Agent-to-Agent Protocol (A2A)

Model Context Protocol (MCP)

The Model Context Protocol (MCP) by Anthropic is an open standard that enables AI models to securely and seamlessly connect with external tools, data sources, and applications, making agents more capable and autonomous.

Think of MCP as a universal adapter, like a USB-C port, that allows AI models to plug into various digital environments without needing custom integration for each one.

MCP is quickly gaining traction across the industry, with major companies like OpenAI and Google beginning to adopt it.

Learn more:

- Anthropic's official announcement and documentation (<https://www.anthropic.com/news/model-context-protocol>)
- MCP on Wikipedia (https://en.wikipedia.org/wiki/Model_Context_Protocol)
- Blog on MCP (<https://huggingface.co/blog/Kseniase/mcp>)

Agent-to-Agent (A2A) Protocol

Google has developed the Agent-to-Agent (A2A) protocol as a complementary counterpart to Anthropic's Model Context Protocol (MCP).

While MCP connects agents to external tools, A2A connects agents to each other, paving the way for cooperative, multi-agent systems that can work together to solve complex problems.

Dive deeper into A2A:

- Google's A2A announcement (<https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/>)

Appendix: Apache License 2.0

This is the complete license text distributed with the pinned source repository.

Apache License Version 2.0, January 2004 <http://www.apache.org/licenses/>

TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

1. Definitions.

"License" shall mean the terms and conditions for use, reproduction, and distribution as defined by Sections 1 through 9 of this document.

"Licensor" shall mean the copyright owner or entity authorized by the copyright owner that is granting the License.

"Legal Entity" shall mean the union of the acting entity and all other entities that control, are controlled by, or are under common control with that entity. For the purposes of this definition, "control" means (i) the power, direct or indirect, to cause the direction or management of such entity, whether by contract or otherwise, or (ii) ownership of fifty percent (50%) or more of the outstanding shares, or (iii) beneficial ownership of such entity.

"You" (or "Your") shall mean an individual or Legal Entity exercising permissions granted by this License.

"Source" form shall mean the preferred form for making modifications, including but not limited to software source code, documentation source, and configuration files.

"Object" form shall mean any form resulting from mechanical transformation or translation of a Source form, including but not limited to compiled object code, generated documentation, and conversions to other media types.

"Work" shall mean the work of authorship, whether in Source or Object form, made available under the License, as indicated by a copyright notice that is included in or attached to the work (an example is provided in the Appendix below).

"Derivative Works" shall mean any work, whether in Source or Object form, that is based on (or derived from) the Work and for which the editorial revisions, annotations, elaborations, or other modifications represent, as a whole, an original work of authorship. For the purposes of this License, Derivative Works shall not include works that remain separable from, or merely link (or bind by name) to the interfaces of, the Work and Derivative Works thereof.

"Contribution" shall mean any work of authorship, including the original version of the Work and any modifications or additions to that Work or Derivative Works thereof, that is intentionally submitted to Licensor for inclusion in the Work by the copyright owner or by an individual or Legal Entity authorized to submit on behalf of the copyright owner. For the purposes of this definition, "submitted" means any form of electronic, verbal, or written communication sent to the Licensor or its representatives, including but not limited to communication on electronic mailing lists, source code control systems, and issue tracking systems that are managed by, or on behalf of, the Licensor for the purpose of discussing and improving the Work, but excluding communication that is conspicuously marked or otherwise designated in writing by the copyright owner as "Not a Contribution."

"Contributor" shall mean Licensor and any individual or Legal Entity on behalf of whom a Contribution has been received by Licensor and subsequently incorporated within the Work.

2. Grant of Copyright License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable copyright license to reproduce, prepare Derivative Works of, publicly display, publicly perform, sublicense, and distribute the Work and such Derivative Works in Source or Object form.

3. Grant of Patent License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable (except as stated in this section) patent license to make, have made, use, offer to sell, sell, import, and otherwise transfer the Work, where such license applies only to those patent claims licensable by such Contributor that are necessarily infringed by their Contribution(s) alone or by combination of their Contribution(s) with the Work to which such Contribution(s) was submitted. If You institute patent litigation against any entity (including a cross-claim or counterclaim in a lawsuit) alleging that the Work or a Contribution incorporated within the Work constitutes direct or contributory patent infringement, then any patent licenses granted to You under this License for that Work shall terminate as of the date such litigation is filed.

4. Redistribution. You may reproduce and distribute copies of the Work or Derivative Works thereof in any medium, with or without modifications, and in Source or Object form, provided that You meet the following conditions:

- (a) You must give any other recipients of the Work or Derivative Works a copy of this License; and
- (b) You must cause any modified files to carry prominent notices stating that You changed the files; and
- (c) You must retain, in the Source form of any Derivative Works that You distribute, all copyright, patent, trademark, and attribution notices from the Source form of the Work, excluding those notices that do not pertain to any part of the Derivative Works; and
- (d) If the Work includes a "NOTICE" text file as part of its distribution, then any Derivative Works that You distribute must include a readable copy of the attribution notices contained within such NOTICE file, excluding those notices that do not pertain to any part of the Derivative Works, in at least one of the following places: within a NOTICE text file distributed as part of the Derivative Works; within the Source form or documentation, if provided along with the Derivative Works; or, within a display generated by the Derivative Works, if and wherever such third-party notices normally appear. The contents of the NOTICE file are for informational purposes only and do not modify the License. You may add Your own attribution notices within Derivative Works that You distribute, alongside or as an addendum to the NOTICE text from the Work, provided that such additional attribution notices cannot be construed as modifying the License.

You may add Your own copyright statement to Your modifications and may provide additional or different license terms and conditions for use, reproduction, or distribution of Your modifications, or for any such Derivative Works as a whole, provided Your use, reproduction, and distribution of the Work otherwise complies with the conditions stated in this License.

5. Submission of Contributions. Unless You explicitly state otherwise, any Contribution intentionally submitted for inclusion in the Work by You to the Licensor shall be under the terms and conditions of this License, without any additional terms or conditions. Notwithstanding the above, nothing herein shall supersede or modify the terms of any separate license agreement you may have executed with Licensor regarding such Contributions.

6. Trademarks. This License does not grant permission to use the trade names, trademarks, service marks, or product names of the Licensor, except as required for reasonable and customary use in describing the origin of the Work and reproducing the content of the NOTICE file.

7. Disclaimer of Warranty. Unless required by applicable law or agreed to in writing, Licensor provides the Work (and each Contributor provides its Contributions) on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied, including, without limitation, any warranties or conditions of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A PARTICULAR PURPOSE. You are solely responsible for determining the appropriateness of using or redistributing the Work and assume any risks associated with Your exercise of permissions under this License.

8. Limitation of Liability. In no event and under no legal theory, whether in tort (including negligence), contract, or otherwise, unless required by applicable law (such as deliberate and grossly negligent acts) or agreed to in writing, shall any Contributor be liable to You for damages, including any direct, indirect, special, incidental, or consequential damages of any character arising as a result of this License or out of the use or inability to use the Work (including but not limited to damages for loss of goodwill, work stoppage, computer failure or malfunction, or any and all other commercial damages or losses), even if such Contributor has been advised of the possibility of such damages.

9. Accepting Warranty or Additional Liability. While redistributing the Work or Derivative Works thereof, You may choose to offer, and charge a fee for, acceptance of support, warranty, indemnity, or other liability obligations and/or rights consistent with this License. However, in accepting such obligations, You may act only on Your own behalf and on Your sole responsibility, not on behalf of any other Contributor, and only if You agree to indemnify, defend, and hold each Contributor harmless for any liability incurred by, or claims asserted against, such Contributor by reason of your accepting any such warranty or additional liability.

END OF TERMS AND CONDITIONS

APPENDIX: How to apply the Apache License to your work.

To apply the Apache License to your work, attach the following boilerplate notice, with the fields enclosed by brackets "[]" replaced with your own identifying information. (Don't include the brackets!) The text should be enclosed in the appropriate comment syntax for the file format. We also recommend that a file or class name and description of purpose be included on the same "printed page" as the copyright notice for easier identification within third-party archives.

Copyright [yyyy] [name of copyright owner]

Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at

<http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.